

ARM PrimeCell

Vectored Interrupt Controller (PL190)

Design Manual

ARM PrimeCell Vectored Interrupt Controller Design Manual

© Copyright ARM Limited 2000. All rights reserved.

Proprietary notice

ARM, the ARM powered logo, Thumb and StrongARM are registered trademarks of ARM Limited.

The ARM logo, PrimeCell, AMBA, Angel, ARMulator, EmbeddedICE, ModelGen, Multi-ICE, ARM7TDMI, ARM9TDMI, TDMI and STRONG are trademarks of ARM Limited.

All other products or services mentioned herein may be trademarks of their respective owners.

Neither the whole nor any part of the information contained in, or the product described in, this document may be adapted or reproduced in any material form except with the prior written permission of the copyright holder.

The product described in this document is subject to continuous developments and improvements. All particulars of the product and its use contained in this document are given by ARM Limited in good faith. However, all warranties implied or expressed, including but not limited to implied warranties or merchantability, or fitness for purpose, are excluded.

This document is intended only to assist the reader in the use of the product. ARM Limited shall not be liable for any loss or damage arising from the use of any information in this document, or any error or omission in such information, or any incorrect use of the product.

Document confidentiality status

This document is ARM Confidential (Distributed to ARM staff, product licensees & NDA signatories only).

Product status

The information in this document is Final **(information on a developed product)**

Web address

<http://www.arm.com>

Feedback

ARM Limited welcomes feedback on both the product, and the documentation.

Feedback on this document

If you have any comments on about this document, please send email to errata@arm.com giving:

- The document title
- The documents number
- The page number(s) to which your comments refer
- A concise explanation of your comments

General suggestion for additions and improvements are also welcome.

Feedback on the ARM PrimeCell Vectored Interrupt Controller

If you have any comments or suggestions about this product, please contact your supplier giving:

- The Product name
- A concise explanation of your comments.

Contents

1	ABOUT THIS DOCUMENT	1
1.1	Change control	1
1.1.1	Current status and anticipated changes	1
1.1.2	Change history	1
1.2	References	1
1.3	Terms and abbreviations	1
2	SCOPE	3
3	INTRODUCTION	3
4	VECTORED INTERRUPT CONTROLLER DESIGN OVERVIEW	4
4.1	Signal Description	4
4.2	Programmer's Model	5
4.2.1	Functional Mode Registers	5
4.2.2	Integration Test Mode Registers	6
4.2.3	Identification Registers	6
4.3	Block Partitioning	7
4.3.1	AHB Interface and Register Block	8
4.3.2	Vector Bank	8
4.3.3	Priority Control	9
4.3.4	Synchronization Block	10
5	VIC IMPLEMENTATION DETAILS	11
5.1	Design Hierarchy	11
5.2	VIC Top Level Module	11
5.3	AHB Interface and Register Block (VicAhbifReg)	11
5.3.1	Input Module	11
5.3.2	Output Module	12
5.4	Vector Bank (VicVectBank)	15
5.5	Priority Control (VicPriority)	16
5.6	Revision Designator block (VicRevAnd)	18
5.7	Synchronisation block (VicSynctoHCLK)	18
5.8	Integration Test Structures	19
5.8.1	Input Integration Test Harness	19

5.8.2	Output Integration Test Harness	19
5.7	Software Restrictions	20
6	VECTORED INTERRUPT CONTROLLER TEST DETAILS	21
6.1	VIC VHDL Verification Testbench	21
6.1.1	Unit Under Test	22
6.1.2	Bus Master Protocol Checker	23
6.1.3	AHB Master/Arbiter Model	23
6.1.4	Trickbox	23
6.1.5	Protocol Checker	24
6.1.6	Test Vectors	24
6.1.7	Generics	24
6.1.8	Running Simulations	25
6.2	VIC Verilog Verification Testbench	25
6.2.1	Unit Under Test	27
6.2.2	Bus Master Protocol Checker	28
6.2.3	AHB Master/Arbiter Model	28
6.2.4	Trickbox	28
6.2.5	Protocol Checker	29
6.2.6	Test Vectors	29
6.2.7	Parameters	29
6.2.8	Running Simulations	30
6.3	VIC Trickbox Description	30
6.3.1	Trickbox Signal Description	30
6.3.2	Trickbox Register Description	32
6.3.3	Trickbox functional description	37
6.4	Functional Tests	41
6.4.1	Reset Tests	41
6.4.2	Register Tests	43
6.4.3	Transfer Test	44
6.4.4	Error Response Test	45
6.4.5	Protection Mode Test	45
6.4.6	Protection Mode Interrupt Test	45
6.4.7	Basic Interrupt Tests	46
6.4.8	Special Interrupt Tests	73
6.5	Verification Test Vector Generation	78
7	VIC INTEGRATION TEST DETAILS	79
7.1	VIC VHDL Integration testbench	79
7.1.1	Unit Under Test	80
7.1.2	Test Vectors	80
7.1.3	Running Simulations	81
7.2	VIC Verilog Integration testbench	81
7.2.1	Unit Under Test	83
7.2.2	Test Vectors	83
7.2.3	Running Simulations	83

7.3	Integration Tests	84
7.3.1	Integration Testing of AHB pins	85
7.3.2	Integration Testing of intra-chip inputs	85
7.3.3	Integration Testing of intra-chip outputs	86
7.4	Integration Test Vector Generation	86

1 ABOUT THIS DOCUMENT

1.1 Change control

1.1.1 Current status and anticipated changes

This document has been reviewed and review comments have been incorporated.

1.1.2 Change history

Issue	Date	Change
A00	14 th June, 2000	First draft
A01	21 th June, 2000	Incorporated review changes
A02	31 st July, 2000	Incorporated review changes
A	8 th August, 2000	First Release

1.2 References

This document refers to the following documents.

Ref.	Doc No	Title
1	ARM DDI 0181	AHB Vectored Interrupt Controller Technical Reference Manual
2	PL190 INTM 0000	ARM PrimeCell Vectored Interrupt Controller (PL190) Integration Manual
3	ARM DUI 0006	AMBA Compliance Test Suite User Guide
4	ARM DDI 0138	Example AMBA System Technical Reference Manual
5	ARM DUI 0092	Example AMBA System User Guide
6	SCB00-QPRO-0008-A03	ARM PrimeCell Integration Vector Strategy

1.3 Terms and abbreviations

This document uses the following terms and abbreviations.

Term	Meaning
AMBA	Advanced Micro-controller Bus Architecture
AHB	Advanced High-performance Bus
ASIC	Application Specific Integrated Circuit
HDL	Hardware Description Language
TIC	Test Interface Controller

VHDL	VHSIC Hardware Description Language
VHSIC	Very High Speed Integrated Circuit
IRQ	ARM core interrupt.
ISR	Interrupt Service Routine.
FIQ	ARM core fast interrupt.
VIC	Vectored Interrupt Controller.
CSR	Current Service Register
Vectored Interrupt	An interrupt where the vector to the interrupt service routine is provided by hardware, normally the interrupt controller.

2 SCOPE

This document describes the design and implementation of the ARM PrimeCell VIC (Vectored Interrupt Controller). The design description is split between two sections – Section 4 and Section 5. The testbenches and the test programs for functional verification and integration testing are described in Section 6 and Section 7.

Section 4 presents an overview of the VIC design, describing the functional partitioning of the VIC and the signal interconnections. Section 5 presents detailed descriptions on the VIC design. Section 6 describes the testbenches and the test programs used to generate test vectors for functional verification of the VIC. Section 7 describes the testbenches and the test programs used to generate test vectors for Integration Testing of the VIC.

3 INTRODUCTION

The VIC is a part of a library of reusable IP blocks, which can be easily integrated into a typical ASIC design flow.

For further information on this block refer to the ARM PrimeCell Vectored Interrupt Controller (PL190) Technical Reference Manual [Ref.1]. The ARM PrimeCell Vectored Interrupt Controller (PL190) Integration Manual [Ref.2] describes how the block may be integrated into a typical industry standard ASIC design flow.

The AMBA Compliance Test Suite User Guide [Ref.3] is a source for further information about AMBA bus compliance testbenches.

The Example AMBA System Technical Reference Manual [Ref.4] and User Guide [Ref.5] describe an example micro-controller based on an ARM processor and the low-power, generic design methodology of AMBA. Further information can also be found on use of TICTalk test vectors for production test.

4 VECTORED INTERRUPT CONTROLLER DESIGN OVERVIEW

The Vectored Interrupt Controller (VIC) is an AMBA slave device that provides a low latency interrupt path from peripherals to the ARM. Interrupt latency is reduced by providing Vectored Interrupt support for high priority interrupt sources. By providing an AHB interface, register access latency is minimised.

4.1 Signal Description

The following tables summarise the VIC signal list. **Table 4.1-1** lists the AHB interface signals, while **Table 4.1-2** lists the module-specific functionality related signals. **Table 4.1-3** lists the Scan Test signals.

Name	Type	Source / Destination	Description
HCLK	In	Clock Source	AHB clock
HRESETn	In	Reset Controller	AHB reset (active low)
HADDR[11:2]	In	Master	This is part of the AHB address bus, which is used for decoding the internal address space.
HSELVIC	In	Decoder	When HIGH, this signal indicates that VIC has been selected.
HTRANS	In	Master	This is connected to HTRANS[1]. When HIGH, this signal indicates that the current transfer is a Sequential or Non-Sequential and when LOW indicates an Idle or Busy transfer.
HPROT	In	Master	This is connected to HPROT[1]. When HIGH, this signal indicates Supervisor Access and when LOW, User Access.
HSIZE[2:0]	In	Master	Indicates AHB bus transfer size. The VIC supports only 32-bit accesses.
HWRITE	In	Master	AHB bus write signal, When HIGH, this signal indicates a write into the VIC and when LOW, a read from the VIC
HWDATA[31:0]	In	Master	Data is written into the VIC on this bus.
HRDATA[31:0]	Out	Slave	Data is read out of the VIC via this bus.
HREADYIN	In	External Slave	AHB bus transfer done indication generated by either the VIC or by an alternate slave.
HREADYOUT	Out	Slave	AHB bus transfer done indication generated by the VIC.
HRESP[1:0]	Out	Slave	AHB bus transfer response generated by the VIC.

Table 4.1-1 AHB interface signal descriptions

Name	Type	Source / Destination	Description
VICINTSOURCE [31:0]	In	Peripheral(s)	Interrupt source input from Peripheral(s)
nVICFIQIN	In	External interrupt controller	Daisy chained FIQ input
nVICIRQIN	In	External interrupt controller	Daisy chained IRQ input
VICVECTADDRIN [31:0]	In	External interrupt controller	Daisy chained interrupt address input

nVICIRQ	Out	ARM / Interrupt Controller	Interrupt request to the ARM processor or to the next Interrupt Controller in the Daisy chain.
nVICFIQ	Out	ARM / Interrupt Controller.	Fast Interrupt request to the ARM processor or to the next Interrupt Controller in the Daisy chain.
VICVECTADDR0UT [31:0]	Out	Interrupt controller	Interrupt address to the next Interrupt Controller in the Daisy chain.

Table 4.1-2 Block-specific functional signal descriptions.

Name	Type	Source / Destination	Description
SCANENABLE	In	Test Controller	Scan test enable.
SCANINHCLK	In	Test Controller	Scan Input signal for HCLK domain.
SCANOUTHCLK	Out	Test Controller	Scan Output signal for HCLK domain.

Table 4.1-3 Scan Test signal descriptions.

4.2 Programmer's Model

The VIC Functional mode registers are shown in **Table 4.2-1** and the Integration Test Mode registers are shown in **Table 4.2-2**. The VIC Identification registers are shown in **Table 4.2-3**. The base address of the VIC is implementation dependent. Although for best performance the VIC should reside at 0xffff_f000. The registers located at offsets 0x300, 0x304, 0x308, 0x30C and 0x310 are reserved for integration testing and are not used during normal operation. For a detailed description of the registers refer to the ARM PrimeCell Vectored Interrupt Controller (PL190) Technical Reference Manual [Ref. 1].

4.2.1 Functional Mode Registers

Address	Type	Read/Write Width	Reset value	Name	Description
Base + 0x000	R	32/00	0x00	VICIRQStatus	IRQ Status after masking
Base + 0x004	R	32/00	0x00	VICFIQStatus	FIQ Status after masking
Base + 0x008	R	32/00	x	VICRawIntr	Interrupt Status prior to masking
Base + 0x00C	R/W	32/32	0x00	VICIntSelect	Select IRQ or FIQ interrupts
Base + 0x010	R/W	32/32	0x00	VICIntEnable	Interrupt Enable
Base + 0x014	W	00/32	-	VICIntEnClear	Interrupt Enable Clear
Base + 0x018	R/W	32/32	0x00	VICSoftInt	To generate Software interrupts
Base + 0x01C	W	00/32	-	VICSoftIntClear	Software Interrupt Clear
Base + 0x020	R/W	1/1	0x00	VICProtection	Protection Enable Register
Base + 0x030	R/W	32/32	x	VICVectAddr	Interrupt Vector address
Base + 0x034	R/W	32/32	0x00	VICDefVectAddr	Default Vector address
Base + 0x100	R/W	32/32	0x00	VICVectAddr0	Interrupt vector address 0
Base + 0x104-0x13C	R/W	32/32	0x00	VICVectAddr1-15	Interrupt vector address N [N = 1 to 15]
Base + 0x200	R/W	6/6	0x00	VICVectCntl0	Interrupt vector control 0

Base + 0x204-0x23C	R/W	6/6	0x00	VicVectCntl1-15	Interrupt vector control N [N = 1 to 15]
Base + 0xFE0	R	8/0	0x90	VICPeriphID0	Peripheral ID register Bits 7:0
Base + 0xFE4	R	8/0	0x11	VICPeriphID1	Peripheral ID register Bits 15:8
Base + 0xFE8	R	8/0	0x04	VICPeriphID2	Peripheral ID register Bits 23:16
Base + 0xFEC	R	8/0	0x00	VICPeriphID3	Peripheral ID register Bits 31:24
Base + 0xFF0	R	8/0	0x0D	VICPCellID0	PrimeCell ID register Bits 7:0
Base + 0xFF4	R	8/0	0xF0	VICPCellID1	PrimeCell ID register Bits 15:8
Base + 0xFF8	R	8/0	0x05	VICPCellID2	PrimeCell ID register Bits 23:16
Base + 0xFFC	R	8/0	0xB1	VICPCellID3	PrimeCell ID register Bits 31:24

Table 4.2-1 Functional Mode Registers Memory Map

4.2.2 Integration Test Mode Registers

Address	Type	Width	Reset value	Name	Description
Base + 0x300	R/W	1/1	0x00	VICITCR	Integration Test Control Register
Base + 0x304	R/W	2/2 [7:6]	x	VICITIP1	To indicate and control nVICIRQIN and nVICFIQIN
Base + 0x308	R/W	32/32	x	VICITIP2	To indicate and control VICVECTADDRIN
Base + 0x30C	R	2/0 [7:6]	x	VICITOP1	To indicate status of FIQ and IRQ
Base + 0x310	R	32/00	x	VICITOP2	To indicate status of VICVECTADDRROUT

Table 4.2-2 Integration Test Mode Registers Memory Map

4.2.3 Identification Registers

Address	Type	Read/Write Width	Reset value	Name	Description
Base + 0xFE0	R	8/0	0x90	VICPeriphID0	Peripheral ID register Bits 7:0
Base + 0xFE4	R	8/0	0x11	VICPeriphID1	Peripheral ID register Bits 15:8
Base + 0xFE8	R	8/0	0x04	VICPeriphID2	Peripheral ID register Bits 23:16
Base + 0xFEC	R	8/0	0x00	VICPeriphID3	Peripheral ID register Bits 31:24
Base + 0xFF0	R	8/0	0x0D	VICPCellID0	PrimeCell ID register Bits 7:0
Base + 0xFF4	R	8/0	0xF0	VICPCellID1	PrimeCell ID register Bits 15:8
Base + 0xFF8	R	8/0	0x05	VICPCellID2	PrimeCell ID register Bits 23:16
Base + 0xFFC	R	8/0	0xB1	VICPCellID3	PrimeCell ID register Bits 31:24

Table 4.2-3 Identification Registers Memory Map

Note : The 'x' in the 'Reset Value' Column of **Table 4.2-1** and **Table 4.2-2** indicate that the reset values are unknown. This is due to the fact that these registers are fed by external inputs [nVICIRQIN, nVICFIQIN and VICVECTADDRIN]. The actual values in these registers will hence depend on the values present on the external inputs.

4.3 Block Partitioning

The Vectored Interrupt Controller block diagram is shown in **Figure 4.3-1**. The design contains the following sub-blocks.

- AHB Interface and Register Block
- Vector Bank [instantiated 16 times]
- Interrupt Priority Control
- Synchronisation Block

The design of these sub-blocks is detailed in the subsequent sections.

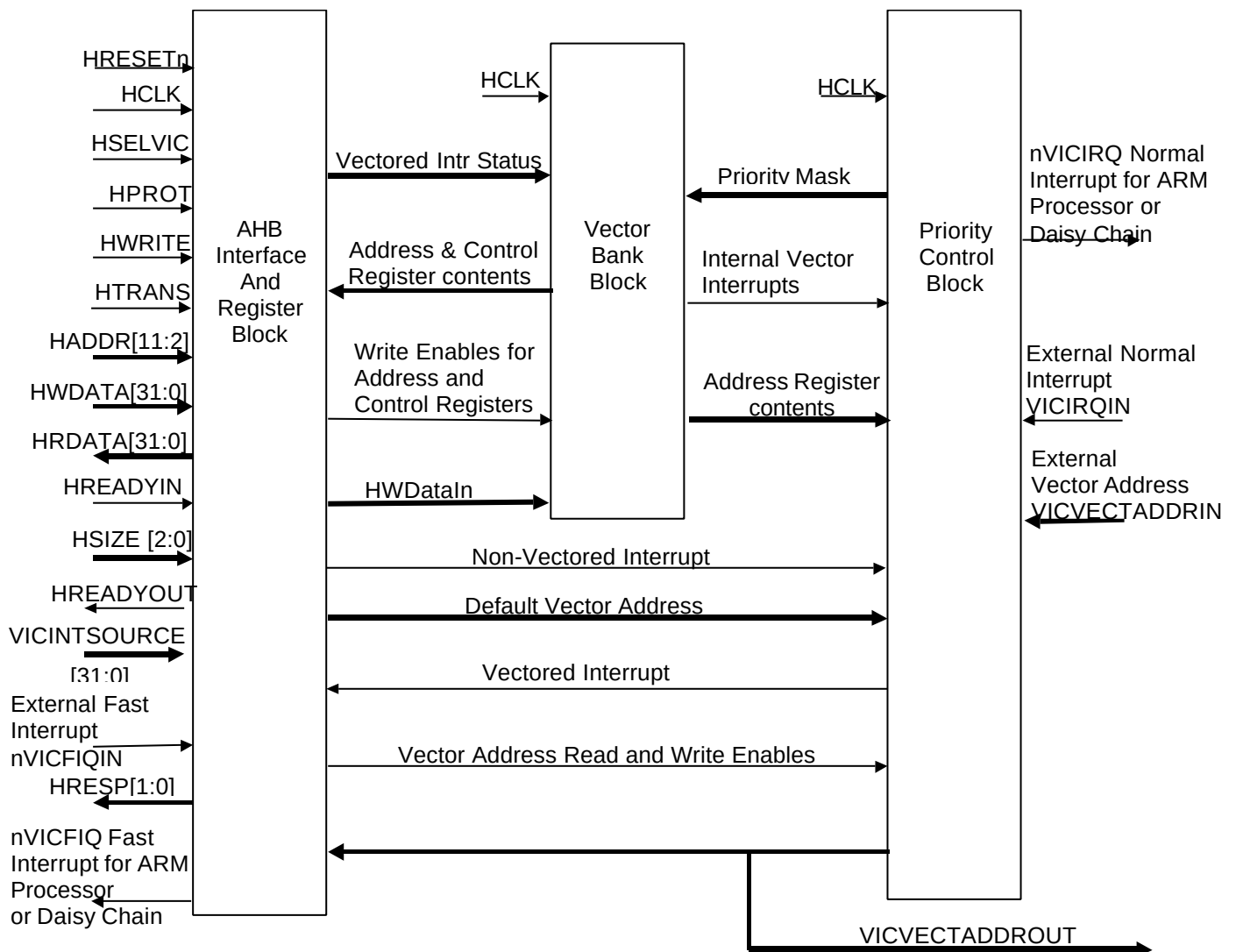


Figure 4.3-1 Vectored Interrupt Controller Block diagram

4.3.1 AHB Interface and Register Block

This block implements the AHB Slave interface and the VIC registers. The AHB interface deals with reads and writes to registers from the AHB. A simplified block diagram of the AHB interface and Register block is shown in **Figure 4.3-2**.

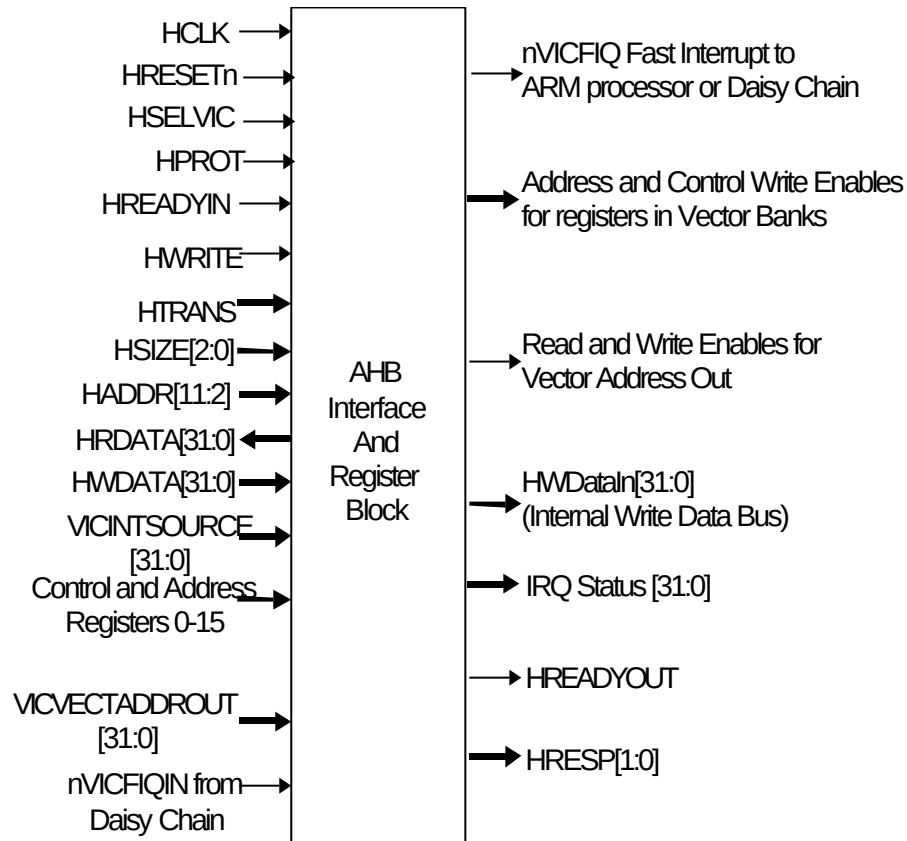


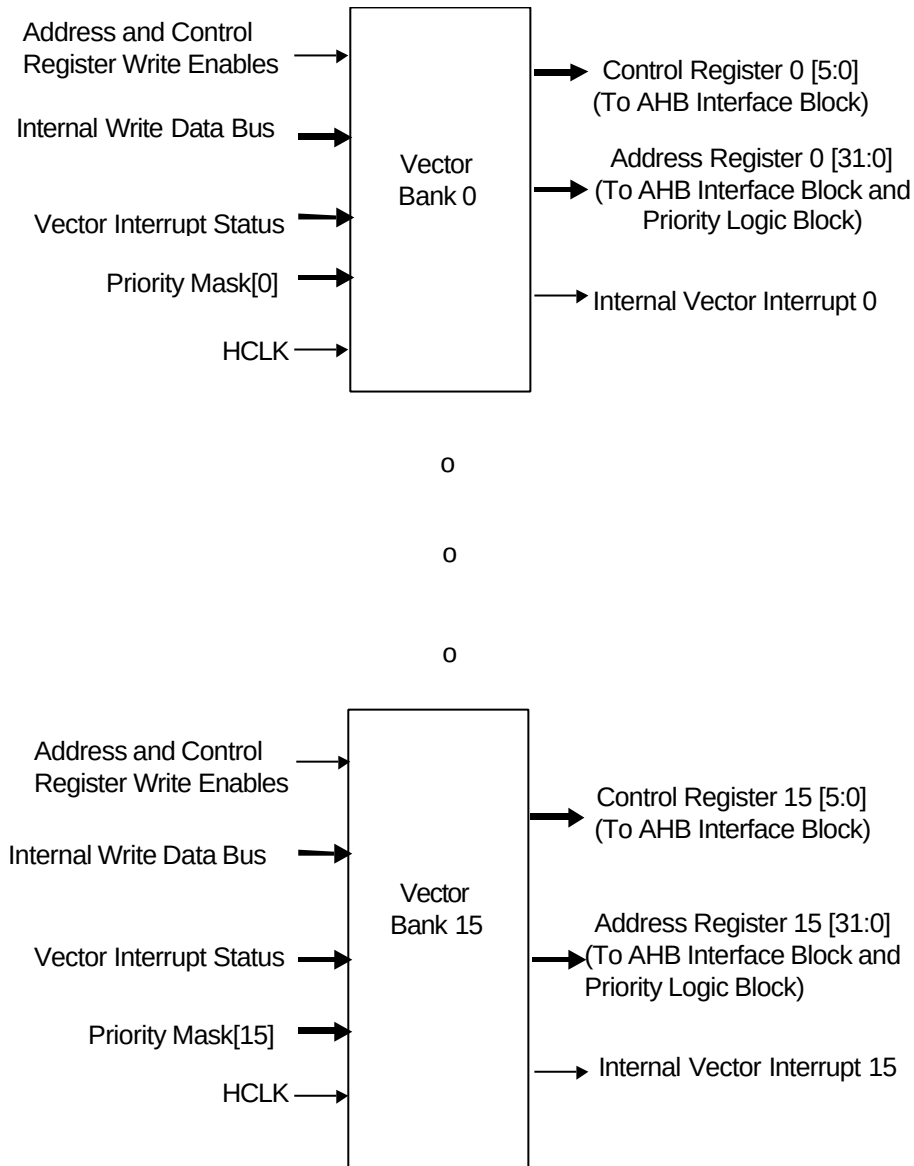
Figure 4.3-2 AHB Interface and Register Block

This block decodes the accesses to the various registers of the VIC and generates write enables for all the writable registers. The internal databus for the VIC, `HWDatInt[31:0]`, is generated by gating the input data bus, `HWDATA[31:0]` with `HSELVIC` and `HWRITE`. The AHB interface and register block also implements the output read data multiplexer.

4.3.2 Vector Bank

Each Vector Bank module contains a Vector Control register and a Vector Address register. The Vector Bank generates an internal Vectored Interrupt. The Vector Bank module is instantiated 16 times with Vector Bank 0 having the highest priority and Vector bank 15 having the least.

The block diagram of the Vector Banks is shown in **Figure 4.3-3**.

**Figure 4.3-3 Vector Banks**

4.3.3 Priority Control

This block prioritises VectorBank requests [0-15], the external interrupt request and the non-vectored interrupt request. The VectorBank requests have the highest priority followed by the non-vectored interrupt request. The external interrupt request has the least priority. The Vectored Interrupts are further prioritised with VectorBank request 0 having the highest priority and VectorBank request 15 having the least. The Priority Controller generates the PriorityMask vector used by Vector Banks and Non-Vectored Interrupt. The Priority Controller also routes the Vector address of the currently active highest priority interrupt into the VICVectAddr register. **Figure 4.3-4** shows the Priority Control Block.

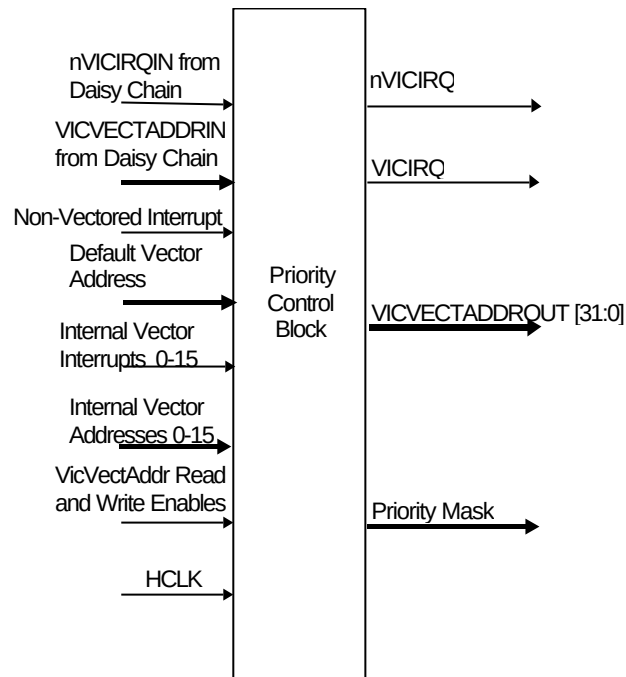


Figure 4.3-4 Priority Control Block

4.3.4 Synchronization Block

This block is used to double synchronize VICIRQStatus , VICFIQStatus, VICRawIntr registers to posedge of HCLK. **Figure 4.3-5** shows the block diagram.

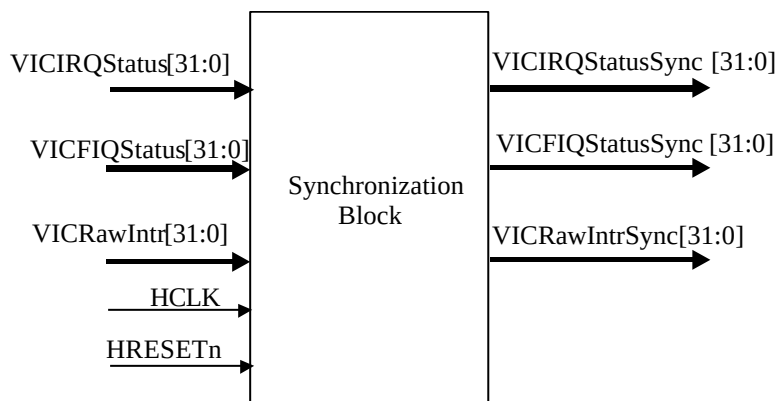


Figure 4.3-5 Synchronization Block

5 VIC IMPLEMENTATION DETAILS

5.1 Design Hierarchy

The hierarchy in the VIC is shown in **Figure 5.1-1**. A short description about the functionality of each of the sub-modules is also given within brackets.

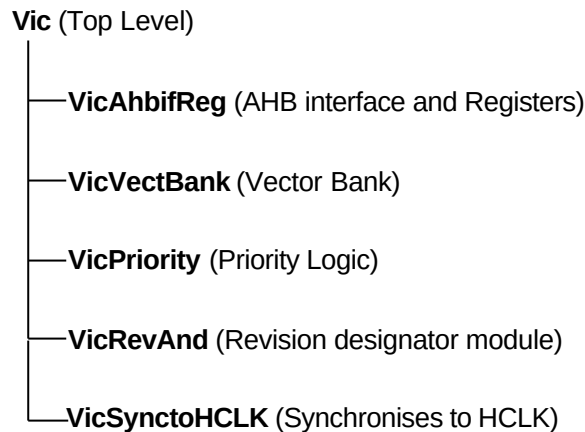


Figure 5.1-1 VIC Design Hierarchy

5.2 VIC Top Level Module

The top-level module is a structural block which instantiates and interconnects the main functional blocks in the VIC.

5.3 AHB Interface and Register Block (VicAhbifReg)

The AHB interface and Register block consists of the input and output module. The AHB interface deals with reads and writes to the VIC registers from the AHB Interface. Power saving is performed by gating the address bus, HADDR[11:2] with HSELVIC, and the write databus, HWDATA[31:0] with HSELVIC and HWRITE to generate HAddrGated[11:2] and HWDatIn[31:0] respectively.

5.3.1 Input Module

This section of the AHB write interface generates decodes for writes to all the writable registers in the device. Each register is clocked by HCLK and is written to when the appropriate write enable is HIGH. The write enable terms are generated from a combinatorial address decode of latched address (IntHAddr) and WrEn. WrEn is generated by ANDing latched HTRANS (IntHTrans), HSELVIC (IntHSELVic), HWRITE (IntHWrite), HREADYIN (IntHReadyIn) and the AccessEn signals.

The AccessEn signal is set when HSIZE indicates a word access and

- The Protection Control bit is cleared OR
- The Protection Control bit is set and HPROT indicates a Privilege mode access.

The block diagram in **Figure 5.3-1** illustrates the structure of the input module. In this block diagram, the write decodes and enables are generated in the AHB interface block. Most of the registers are located with this block itself. However some registers are physically located in the Vector Banks or in the Priority control module.

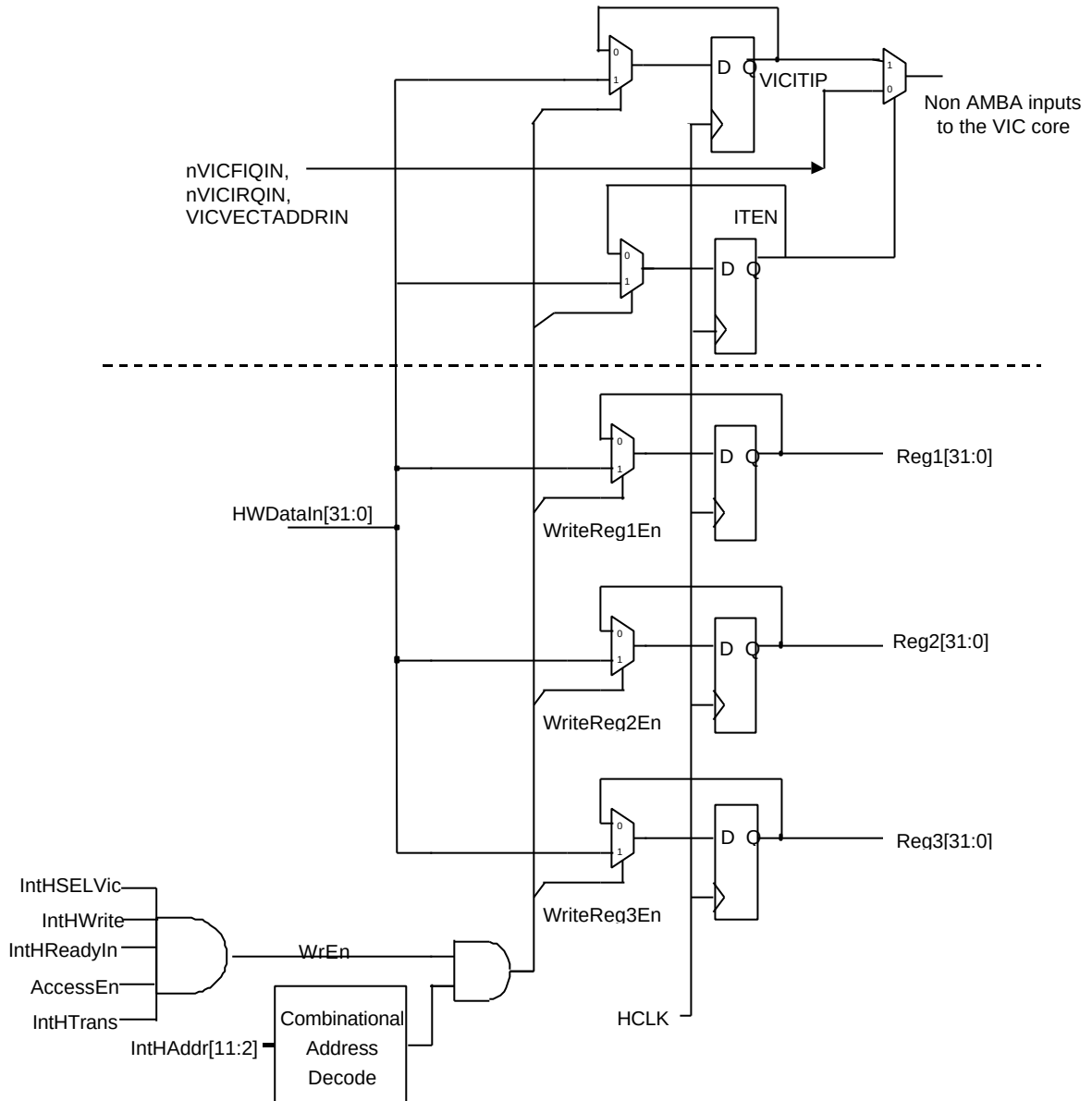


Figure 5.3-1 Input Module

5.3.2 Output Module

The output module in the AHB interface consists of a multiplexer for selecting the various sources of read data to be driven onto the read databus. A combinatorial address decode of latched address (`IntHAddr`) is used to select which of the internal register sources are driven onto the HRDATA bus.

The output multiplexer is left such that, by default, zeros are driven on to the data bus. This ensures that when Non-Privilege mode accesses or Non-Word accesses are attempted, the VIC returns all zeros on the bus. A block diagram of the output module is shown in **Figure 5.3-2**. The registers shown in this block diagram are distributed among different blocks while the output multiplexer is located in the AHB interface block.

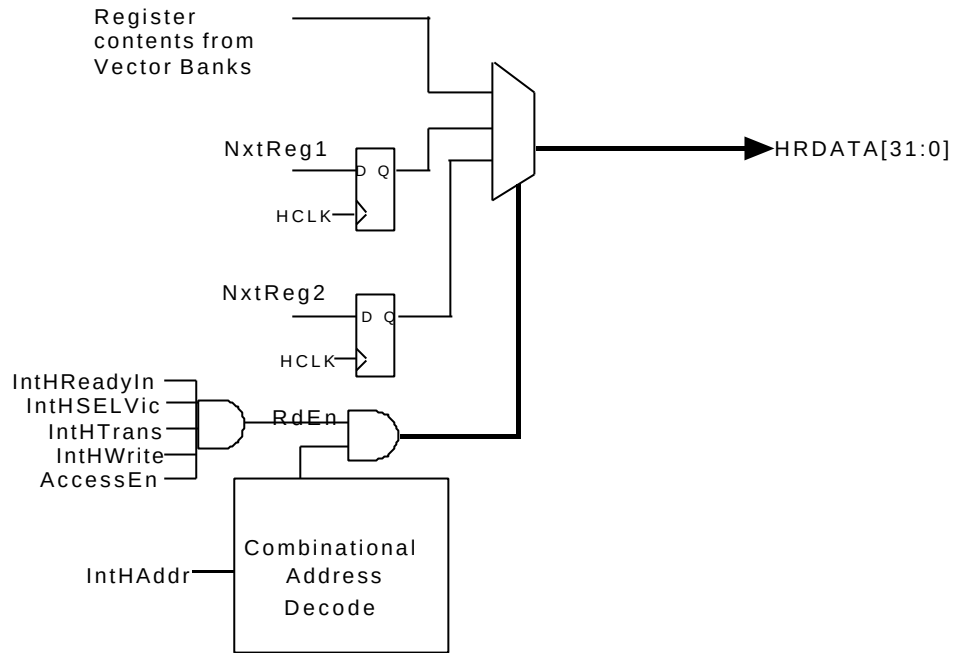


Figure 5.3-2 Output module

The AHB Interface Block also generates two enables for the Priority Logic block (PriorWrEn, PriorRdEn). The PriorWrEn is asserted whenever there is an AHB write to the VICVectAddr register. PriorRdEn is asserted whenever there is an AHB read to the VICVectAddr register.

Interrupt Status Generation

The AHB Interface block generates the VICRawIntr, VICIRQStatus and the VICFIQStatus vectors. The VICINTSOURCE is OR-ed with VICSoftInt to generate VICRawIntr. The VICRawIntr vector is AND-ed with VICIntEnable. The output is routed to VICFIQStatus if the corresponding bit in the VICIntSelect register is set, otherwise it is routed to the VICIRQStatus. When the ITEN bit in the VICITCR register is set, zeroes are routed to the Raw Interrupt generation logic, else the external VICINTSOURCE is routed. This ensures that during integration testing, the interrupt generation logic is controlled purely by the VICSoftInt register with no contribution from the VICINTSOURCE inputs.

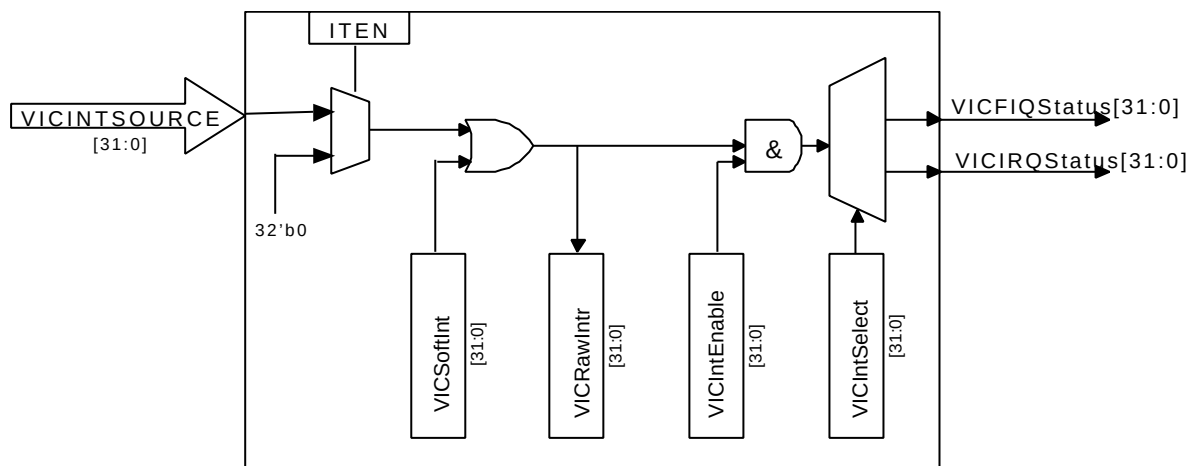


Figure 5.3-3 Interrupt Status Generation

The combinational versions of VICIRQStatus, VICFIQStatus and VICRawIntr registers are used for generating other internal signals while the synchronised versions are routed to the HRDATA multiplexer.

Fast Interrupt Generation

All the VICFIQStatus bits and the inverted version of the External Fast interrupt (from the daisy chain) nVICFIQIN are OR-ed together to create the VICFIQ signal. This feeds bit 6 of the VICITOP1 register. The VICFIQ signal is inverted to generate the active low Fast Interrupt output, nVICFIQ.

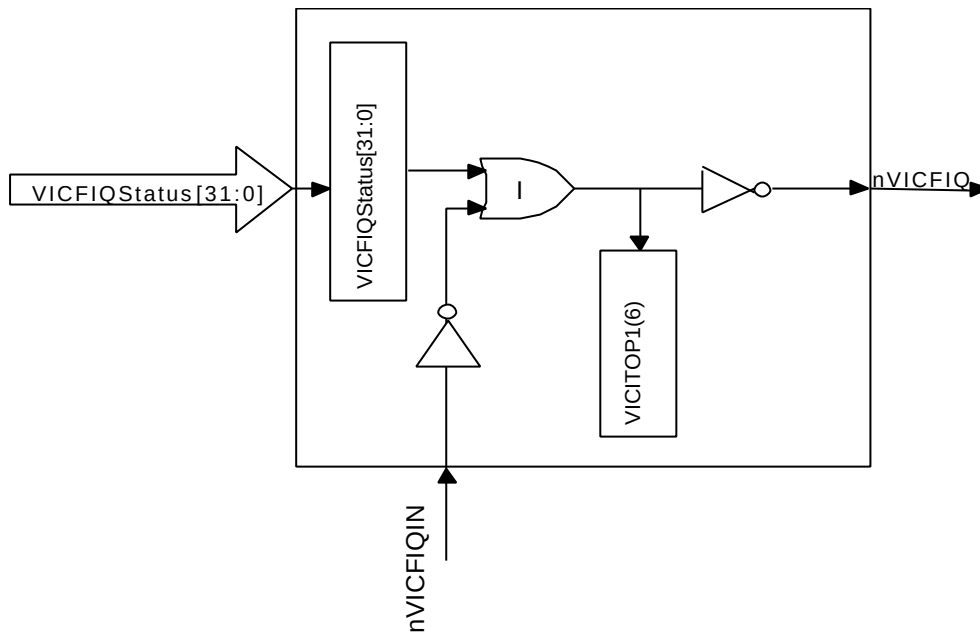


Figure 5.3-4 FIQ Interrupt Generation

Non Vectored Interrupt Generation

The Non-Vectored Interrupt (NonVectIrqCo) is generated by OR-ing all the VICIRQStatus bits.

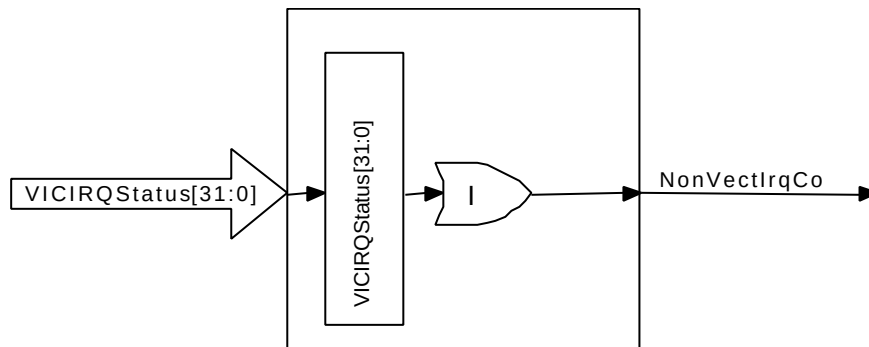


Figure 5.3-5 Non Vectored Interrupt Generation

Interrupt Enable Logic

The enable register (VICIntEnable) has a dual mechanism for setting and clearing the enable bits. This allows VICIntEnable bits to be set or cleared independently, with no knowledge of the other bits in the enable register. When writing to the VICIntEnable register, each data bit that is HIGH sets the corresponding bit in the VICIntEnable register. All other bits of the enable register are unaffected. Conversely, when writing to the VICIntEnClear register, each data bit that is HIGH clears the corresponding bit in the VICIntEnable register while leaving other bits unaffected.

Software Interrupt Logic

The SoftInt register (VICSoftInt) also has a dual mechanism for setting and clearing the enable bits. This allows VICSoftInt bits to be set or cleared independently, with no knowledge of the other bits in the enable register. When writing to the VICSoftInt register, each data bit that is HIGH sets the corresponding bit in the VICSoftInt register. All other bits of the enable register are unaffected. Conversely, when writing to the VICSoftIntClear register, each data bit that is HIGH clears the corresponding bit in the VICSoftInt register while leaving other bits unaffected.

Response Generation

The VIC returns a Zero wait state OK response if the incoming transfer is a SEQ or a NSEQ and the access is a Word access or if the incoming transfer is a BUSY or an IDLE. If the transfer is a SEQ or a NSEQ with a Non-Word access type, the VIC returns a two cycle ERROR response. During the first cycle of the ERROR response, HREADYOUT is driven low. In the second cycle, HREADYOUT is driven high to terminate the cycle. If the VIC is not being accessed, it returns a Zero wait state OK response.

5.4 Vector Bank (VicVectBank)

Each Vector Bank module contains a Vector Control register (VICVectCntl) and a Vector Address register (VICVectAddr). The VICIRQStatus[31:0] is compared with the IntSource field of the Vector Control register (VICVectCntl). If a match is found and the Enable bit is set, the Vector Bank raises an internal Vectored Interrupt (VectIrq). This is further qualified by the PriorityMask bit generated by the VicPriority module. This mask bit ensures that a Vector Bank raises an interrupt only if it is currently the highest priority requestor.

The Vector Bank module is instantiated 16 times with Vector Bank 0 having the highest priority and Vector bank 15 having the least.

The block diagram of the Vector Bank is shown in **Figure 5.4-1**

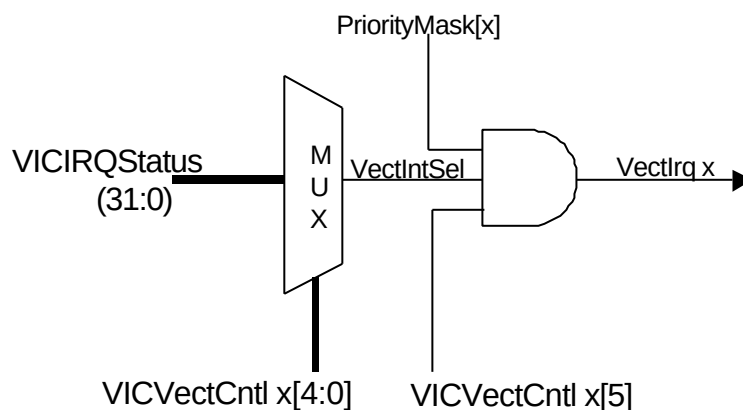


Figure 5.4-1 Vector Bank

5.5 Priority Control (VicPriority)

This block prioritises VectorBank requests (Vectlrq0Co – Vectlrq15Co), the external interrupt request (nVICIRQIN) and the non-vectored interrupt request (NonVectlrqCo). The VectorBank requests have the highest priority followed by the non-vectored interrupt request. The external interrupt request has the least priority. The Vectored Interrupts are further prioritised with VectorBank request, Vectlrq0Co, having the highest priority and VectorBank request, Vectlrq15Co, having the least. The Priority Controller generates the PriorityMask vector(17 bit). Bits [15:0] of this PriorityMask vector are routed to the Vector Banks [with each bank getting a single PriorityMask bit] allowing each Bank to raise a request only if it is the current highest priority source. PriorityMask[16] is used for prioritising NonVectlrq. If no interrupt is being serviced, the PriorityMask enables all interrupts. The Priority Controller also routes the clocked Vector address of the currently active highest priority interrupt into the VICVectAddr register and to the VICVECTADDRROUT output.

nVICIRQ generation

All the 16 Vectored Interrupts (Vectlrq0-15), the inverted version of the external interrupt request (nVICIRQIN) and the Non-vectored interrupt request (NonVectlrqCo) are logically ORed to create the normal interrupt request. This is inverted to create the active low nVICIRQ signal. To ensure that the Vectored interrupts have the highest priority, the external interrupt and the non-vectored interrupts are qualified with bit 16 of the PriorityMask vector. PriorityMask[15] is high only when there are no active Vectored interrupts and PriorityMask[16] is high when there are no active Vectored interrupts and non-vectored interrupts. The pseudocode is as follows:

```
PriorNonvect  <= NonVectlrqCo && PriorityMask[16];
```

```
VICIRQ <= logical OR[ Vectlrq[0 to 15], PriorNonvect, (not nVICIRQIN and PriorityMask[16])
```

```
nVICIRQ <= not VICIRQ;
```

VICVECTADDRROUT generation

The Priority logic block generates the VICVECTADDRROUT (vector address of currently active highest priority interrupt) as shown in the following pseudocode. NxtVicAddrOut is assigned VICDefVectAddr for NonVectlrq interrupt and when there is no active interrupt. NxtVicAddrOut is clocked on the rising edge of HCLK to create VICVECTADDRROUT.

```
NxtVicAddrOut  <= VICVectAddr0      when (Vectlrq0Co = '1')
                else
                    VICVectAddr1      when (Vectlrq1Co = '1')
                -
                -
                else
                    VICVectAddr15     when (Vectlrq15Co = '1')
                else
                    VICDefVectAddr     when (PriorNonvect = '1')
                else
                    VICVECTADDRIN     when ((nVICIRQIN = '0') and PriorityMask(16))
                else
                    VICDefVectAddr;
```

Current Service Register

The Priority Logic Block contains a 17 bit register, CurrServ, which acts like a stack. Whenever a read to the VICVectAddr register is detected on the AHB (PriorRdEn is high), the highest priority interrupt currently active is OR-ed with the contents of the CurrServ register. Whenever a write to the VICVectAddr is detected on the AHB (PriorWrEn is high), the lowest numbered set bit in the CurrServ register is cleared. If both PriorRdEn and PriorWrEn are deasserted then CurrServ retains its old value. The pseudocode for determining the currently active interrupt is as follows:

```

ActiveIrq <= "00000000000000001" when (VectIrq0Co = '1')
    else
        "00000000000000010" when (VectIrq1Co = '1')
        -
        -
        -
    else
        "01000000000000000" when (VectIrq15Co = '1')
    else
        "10000000000000000" when (PriorNonvect = '1')
    else
        "00000000000000000";

```

The pseudocode for the CurrServ Register implementation is as follows:

```

NxtCurrServ <= CurrServ;
if ( PriorRdEn = '1') then
    NxtCurrServ <= ActiveIrq or CurrServ;
elsif ( PriorWrEn = '1') then
    if (CurrServ(0) = '1') then
        NxtCurrServ(0) <= '0';
    elsif (CurrServ(1) = '1') then
        NxtCurrServ(1) <= '0';
    -
    -
    -
elsif (CurrServ(15) = '1') then
    NxtCurrServ(15) <= '0';

```

```

elsif (CurrServ(16) = '1') then
    NxtCurrServ(16) <= '0';
else
    NxtCurrServ <= "0000000000000000";
end if;
end if;

```

Priority Mask generation

The Priority Logic also generates PriorityMask (a 17 bit vector) which is used by the Vector Bank blocks and non-vectored interrupt for enabling only interrupts which have a higher priority than the current interrupt being serviced. The PriorityMask is generated by finding the lowest numbered set bit in CurrServ register and disabling all the lower priority interrupts. If no bit is set, then all the interrupts are enabled. The pseudocode is as follows:

```

If (CurrServ(0) = '1') then
    PriorityMask <= "00000000000000000";
elsif (CurrServ(1) = '1') then
    PriorityMask <= "00000000000000001";
-
-
-
elsif (CurrServ(15) = '1') then
    PriorityMask <= "00111111111111111";
elsif (PriorNonvect) = '1') then
    PriorityMask <= "01111111111111111";
else
    PriorityMask <= "11111111111111111";
end if;

```

5.6 Revision Designator block (VicRevAnd)

This module contains a single AND gate to be used as a place-holder cell to mark the Revision of the controller. The 2 input pins are tied off at the top level of the hierarchy. These "TieOffs" can be identified during layout and re-wired to "VDD" or "VSS" if needed.

5.7 Synchronisation block (VicSynctoHCLK)

This module is used to double synchronize VICIRQStatus, VICFIQStatus and VICRawIntr registers to the HCLK domain. The VICINTSOURCE input to the VIC is an asynchronous input that has combinational paths to the above-mentioned registers. Hence, double synchronization of these registers is done for the Read path.

5.8 Integration Test Structures

This section describes the test structures for supporting the Integration test strategy.

Integration testing is supported by the VICITCR [Integration Test Control Register], the VICITIP1/2 [Integration Test Input Registers] and the VICITOP1/2 [Integration Test Output Registers]. The ITEN bit in the VICITCR register is used to force the multiplexers on the non-AMBA inputs of the VIC to use the test values.

5.8.1 Input Integration Test Harness

Figure 5.8-1 explains the implementation details of the input integration test harness. The ITEN bit is used as the control bit for the multiplexer which is used in the read path of the intra-chip inputs. If the ITEN control bit is de-asserted, the intra-chip inputs [VICINTSOURCE, nVICIRQIN, nVICFIQIN, VICVECTADDRIN] are routed; else the stored register values are driven on the internal lines.

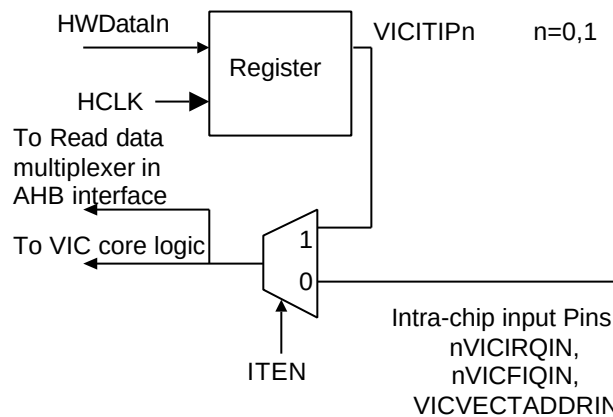


Figure 5.8-1 Input Integration Test harness

5.8.2 Output Integration Test Harness

Figure 5.8-2 explains the implementation details of the output integration test harness.

VICITOP1 reflects the pre-inverted values on the nVICIRQ and nVICFIQ lines. VICITOP2 reflects values on the VICVECTADDRROUT lines.

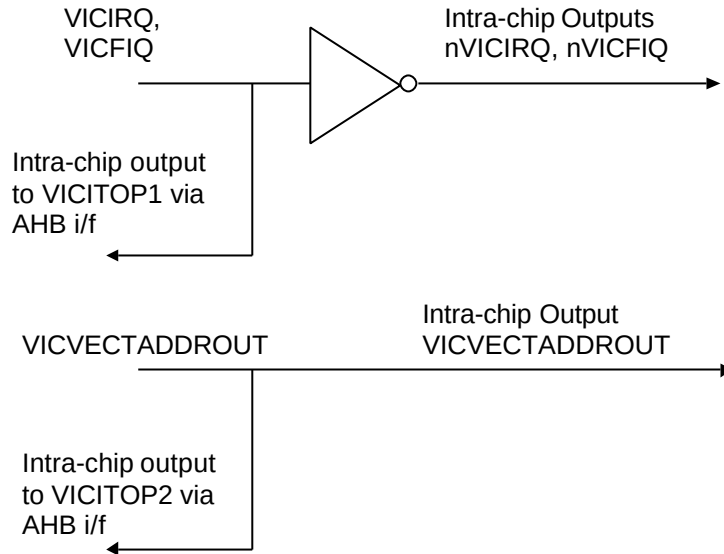


Figure 5.8-2 Output Integration Test harness

5.7 Software Restrictions

The software should ensure the following:

- Any AHB write access which causes a change in the VICRawIntr, VICIRQStatus or VICFIQStatus registers should NOT be immediately followed by a read to these registers. At least one IDLE cycle should separate the Read from the Write. This is due to the fact that the VIC is a Zero wait state peripheral. It takes two clocks for the write to update internal registers. Due to the pipelining of the AHB and since the VIC does not insert wait states, a read access immediately following the write will return the previous register values.
- Any AHB write access which modifies the VICProtection register should NOT be immediately followed by another access to the VIC. At least one IDLE cycle should separate the Write from the subsequent access.

6 VECTORED INTERRUPT CONTROLLER TEST DETAILS

The VIC Functional Verification testbench is an AHB BusTalk testbench. The VHDL and Verilog BusTalk testbench files reside in the **verification/vhdl** and **verification/verilog** directories respectively. The test vectors that are applied to the BusTalk testbenches are in the **verification/bustest** directory.

6.1 VIC VHDL Verification Testbench

The ARM PrimeCell VIC VHDL test world uses a testbench which is based on a bus compliance testbench from the AMBA Compliance Test Suite. For further information refer to the ARM Micropack AMBA Compliance Test Suite User Guide.

The testbenches are designed to be used with most common VHDL simulators, and they are suited to AMBA system designers who want to ensure the portability of their blocks to any other AMBA designs. This simplifies the ASIC integration task and exchange of peripherals between projects.

The testbench is “programmable”, meaning that a description of the transfers to be performed on the **Unit Under Test** (UUT) is given without a need to modify the test bench implementation, hence the “generic” property. This transfer description takes the form of a text file that can be read by the specific test bench to which it is being targeted.

Figure 6.1-1 illustrates the Bustalk VHDL testbench for running test vectors.

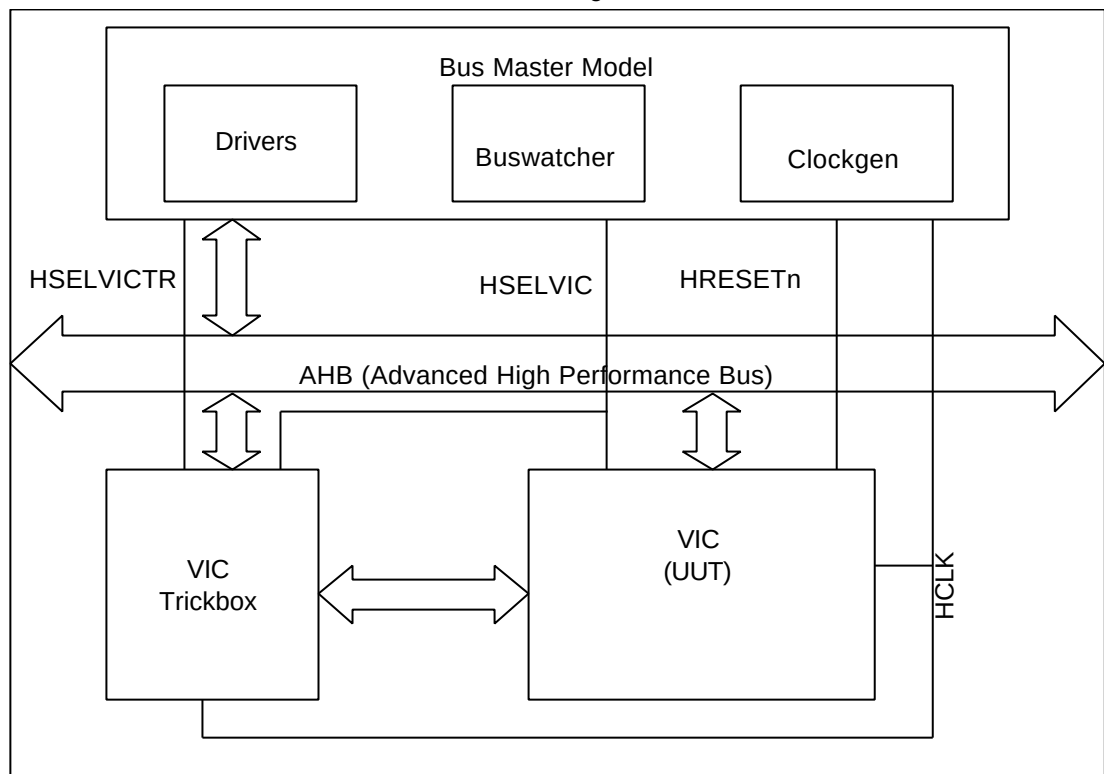


Figure 6.1-1 VHDL Functional Verification Testbench

Figure 6.1-2 illustrates the hierarchy for the VHDL testbench.

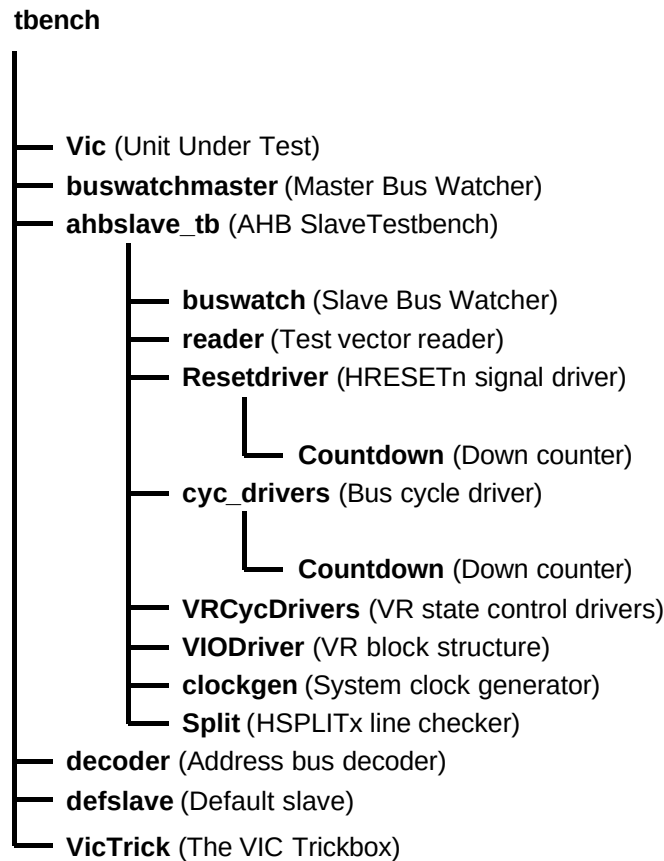


Figure 6.1-2 VHDL Functional Verification Testbench hierarchy

The testbench, **tbench.vhd** is used for functional testing of the Vectored Interrupt Controller (VIC). This block instantiates the VIC (Unit Under Test), the behavioural model of the trickbox, the behavioural model of the AHB Master/Arbiter, the decoder, the default slave and the master bus watcher.

All the non-AMBA inputs to the VIC are driven by the trickbox. All the Scan input pins are tied low to keep them from interfering with the test process.

The default test frequency of HCLK is set to 100MHz. This is the same frequency to which the VIC was constrained during synthesis. The HCLK frequency and AHB signal timing budgets are defined in the **timing.vhd** file.

6.1.1 Unit Under Test

The Unit Under Test may be one of the following

- **VIC VHDL RTL**
- **VIC scan-inserted VHDL netlist from VHDL Source**

One out of these two versions may be referenced by creating a link [named **uut**] in the **verification/vhdl** directory to the corresponding source directory. [Note that this link will be created automatically by the simulation makefiles].

6.1.2 Bus Master Protocol Checker

The testbench instantiates the master bus watcher, `buswatchmaster.vhd` which checks the protocol of master accesses initiated via the test code. This module implements the protocol checks for the AHB master's output signals and will flag warning and error messages if any timing or protocol violations are detected during simulations.

6.1.3 AHB Master/Arbiter Model

The UUT is stimulated primarily by AHB Master/Arbiter behavioural model. The AHB Master/Arbiter block issues commands on the AHB bus under program control.

The AHB Master/Arbiter model, `ahbslave_tb.vhd` instantiates the buswatcher, the reader, the reset driver, the cycle drivers, the VR cycle driver, the split checker and the clock generator. The reader block reads the test vectors from the .bif file (from the ***verification/bustest/invec*** directory) one line at a time and drives the information about the bus cycle in the form of packets. If the information driven by the reader indicates that the current cycle is a reset cycle, the reset driver drives the HRESETn signal with the correct timings. The `cyc_drivers` drive the address, control and data signals for each AHB cycle type, i.e. HSR and HSW etc. The `VRCycdrivers` selects the appropriate VR registers for a read or write operation. The buswatch module implements the protocol checks for the AHB slave's output signals and flags warning and error messages on detection of mismatches. The Split module verifies that the HMASTER number is asserted on the HSPLITx lines on the expected clock. The clockgen module generates the system clock, HCLK.

The reader reads test vectors from the `infile.bif` residing in the ***verification/bustest/invec*** directory.

Decoder

This module decodes the addresses and generates the HSELx signals for the slaves. It can generate the select signals for a maximum of 16 slaves.

The UUT (VIC) is connected to HSEL[0] and the Trickbox is connected to HSEL[1].

Any access to the address range

0x50000000 to 0x5FFFFFFF

will access the UUT via the HSEL[0] select line.

Accesses to the address range

0xA0000000 to 0xAFFFFFFF

will access the Trickbox via the HSEL[1] select line.

Accesses to any address outside the above ranges will select the Default Slave.

Default Slave

This module drives the response when an access is detected to an unmapped region. The Default slave drives its HREADY output high, when other slaves are not selected. It drives a zero wait state OK response for IDLE and BUSY transfers and a two cycle ERROR response for an NSEQ or SEQ transfer access to an unmapped address.

6.1.4 Trickbox

The functionality of the VIC is verified via the Trickbox. The Trickbox is a programmable AHB slave designed to drive stimulus on the non-AHB input terminals of the VIC and sample its non-AHB outputs. The Trickbox implements a mirrored model of the VIC within it and continuously compares the locally generated outputs with the actual outputs generated by the VIC.

6.1.5 Protocol Checker

The testbench includes a protocol checker which reports any protocol or timing violations during accesses to any AHB slave. Timing parameters can be set using the timing.vhd file in the **verification/vhdl/tbench** directory. The buswatch module residing in the **verification/vhdl/buswatcher** directory checks the Read/Write databus timings and reports any timing violations.

6.1.6 Test Vectors

The test vectors for the verification of the VIC are coded into separate C files, in the **verification/bustest** directory, depending on the logical region of the VIC which these vectors are testing. Make options have been provided to use all the test vectors together or to use them individually.

6.1.7 Generics

The VIC testbench provides some configurability options using generics. All these generics except **SuppressOnReset** are configurable through the tbench.vhd file. **SuppressOnReset** is configurable through the ahbslave_tb.vhd file in the **verification/vhdl/tbench** directory.

XonSig

XonSig is used in the cyc_drivers.vhd file that resides in **verification/vhdl/bid**.

This generic is used to eliminate the 'X's that appear on the address and control signals when these signals are not being actively driven. If **XonSig** is set to **0** in the top level of the testbench, these signals will go to '0' when the corresponding line driver is inactive. If set to **1**, the signals go to 'X' when the line driver is idle.

Verbosity

Verbosity is used in the reset_driver.vhd and the cyc_drivers.vhd which reside in **verification/vhdl/bid**. It is also used in the buswatch.vhd file that resides in **verification/vhdl/buswatcher** and the VIODrivers.vhd which reside in **verification/vhdl/vr**.

This generic is used to control the verbosity of the transcript generated during simulation. If **Verbosity** is set to **1**, every command issued will have a corresponding message in the transcript file. If **Verbosity** is set to **0**, a message is issued only if an error occurs on a read.

HaltOnMismatch

This is used in the cyc_drivers.vhd, which resides in **verification/vhdl/bid**, the buswatch.vhd which resides in **verification/vhdl/buswatcher** and the VIODriver.vhd which resides in **verification/vhdl/vr**.

This generic is used to stop the simulation if an error occurs. If set to **1**, the simulation halts after an error is reported. If set to **0**, the error is flagged, but the simulation continues.

Databuswidth

Databuswidth is used in the cyc_drivers.vhd, which resides in **verification/vhdl/bid**.

This generic controls the width of the AHB databus. For the VIC testbench this is set to **32**.

SuppressOnReset

SuppressOnReset is used in the buswatch.vhd, which resides in **verification/vhdl/buswatcher**.

This generic is used to suppress all protocol checks on the slave's output signals when the HRESETn signal is asserted. If **SuppressOnReset** is TRUE, the set-up and hold timing violations on these signals are not flagged when HRESETn is asserted. If **SuppressOnReset** is set to FALSE, the set-up and hold timing violations are flagged irrespective of HRESETn. When **SuppressOnReset** is FALSE the slave's output signals are not checked for 'X'es when HRESETn is asserted.

6.1.8 Running Simulations

The user's guide gives step by step instructions for running simulations using the VIC test vectors on the VHDL source code and the VHDL netlist.

Run time logs for the RTL simulations are:

- `verification/vhdl/Vic_rtl/Vic_rtl_modelsim.log`

Run time logs for the netlist simulations are:

- `verification/vhdl/Vic_scaninsert_vhdl_net_max/Vic_scaninsert_vhdl_net_max_modelsim.log`

The code coverage results are:

- `verification/vhdl/Vic_rtl/vnavigator.summary.`

6.2 VIC Verilog Verification Testbench

The ARM PrimeCell VIC Verilog test world uses a testbench which is based on a bus compliance testbench from the AMBA Compliance Test Suite. For further information refer to the ARM Micropack AMBA Compliance Test Suite User Guide.

The test bench is designed to be used with most common Verilog simulators, and it is suited to AMBA system designers who want to ensure the portability of their blocks to any other AMBA designs. This simplifies the ASIC integration task and exchange of peripherals between projects.

The test bench is "programmable", meaning that a description of the transfers to be performed on the **Unit Under Test** (UUT) is given without a need to modify the test bench implementation, hence the "generic" property. This transfer description takes the form of a text file that can be read by the specific test bench to which it is being targeted.

Figure 6.2-1 illustrates the BusTalk Verilog testbench for simulating test vectors.

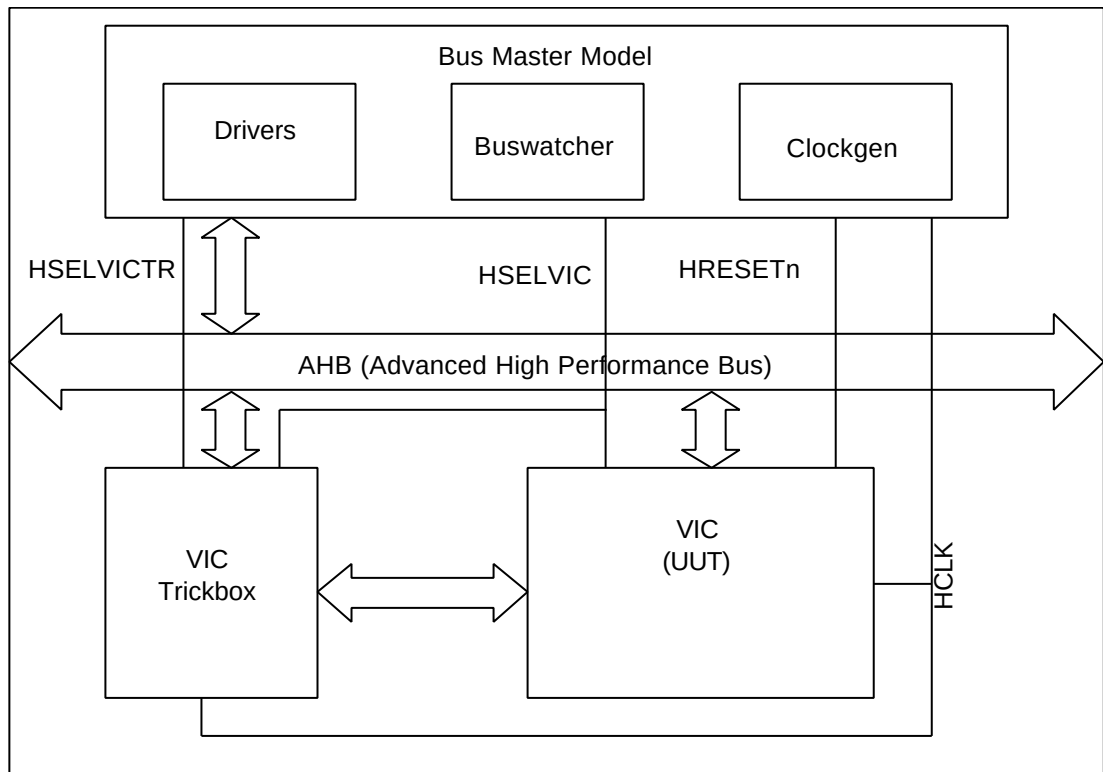


Figure 6.2-1 Verilog Functional Verification Testbench

Figure 6.2-2 illustrates the hierarchy for the functional Verilog testbench.

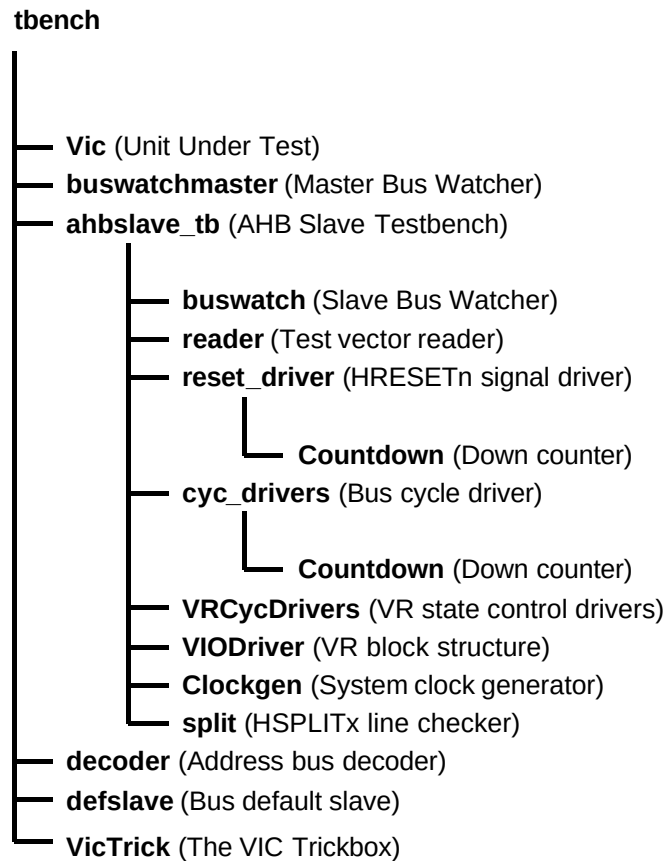


Figure 6.2-2 Verilog Functional Verification Testbench hierarchy

The testbench, **tbench.v** is used for functional testing of the Vectored Interrupt Controller (VIC). This block instantiates the VIC (Unit Under Test), the behavioural model for the AHB masters/arbiters, the behavioural model of the trickbox, the decoder, the default slave and the master bus watcher.

All the non-AMBA inputs to the VIC are driven by the trickbox. All the Scan input pins are tied low to keep them from interfering with the test process.

The default test frequency of HCLK is set to 100MHz. This is the same frequency to which the VIC was constrained during synthesis. The HCLK frequency and AHB signal timing budgets are defined in the timing.v file.

6.2.1 Unit Under Test

The Unit Under Test may be one of the following

- **VIC Verilog RTL**
- **VIC scan-ready Verilog netlist from Verilog Source**
- **VIC non-scan Verilog netlist from VHDL Source**

One out of these two versions may be referenced by creating a link [named **uut**] in the **verification/verilog** directory to the corresponding source directory. [Note that this link will be created automatically by the simulation makefiles].

6.2.2 Bus Master Protocol Checker

The testbench instantiates the master bus watcher, `buswatchmaster.v` which checks the protocol of master accesses initiated via the test code. This module implements the protocol checks for the AHB master's output signals and will flag warning and error messages if any timing or protocol violations are detected during simulations.

6.2.3 AHB Master/Arbiter Model

The UUT is stimulated primarily by AHB Master/Arbiter behavioural model. The AHB Master/Arbiter block issues commands on the AHB bus under program control.

The AHB Master/Arbiter model, `ahbslave_tb.v` instantiates the buswatcher, the reader, the reset driver, the cycle drivers, the VR cycle driver, the split checker and the clock generator. The reader block reads the test vectors from the .sim file (from the ***verification/bustest/invec*** directory) one line at a time and drives the information about the bus cycle in the form of packets. If the information driven by the reader indicates that the current cycle is a reset cycle, the reset driver drives the HRESETn signal with the correct timings. The `cyc_drivers` drive the address, control and data signals for each AHB cycle type, i.e. HSR and HSW etc. The `VRCycdrivers` selects the appropriate VR registers for a read or write operation. The buswatch module implements the protocol checks for the AHB slave's output signals and flags warning and error messages on detection of mismatches. The split module verifies that the HMASTER number is asserted on the HSPLITx lines on the expected clock. The Clockgen module generates the system clock, HCLK.

The reader reads test vectors from the `bif.sim` residing in the ***verification/bustest/invec*** directory.

Decoder

This module decodes the addresses and generates the HSELx signals for the slaves. It can generate the select signals for a maximum of 16 slaves.

The UUT (VIC) is connected to HSEL[0] and the Trickbox is connected to HSEL[1].

Any access to the address range

0x50000000 to 0x5FFFFFFF

will access the UUT via the HSEL[0] select line.

Accesses to the address range

0xA0000000 to 0xAFFFFFFF

will access the Trickbox via the HSEL[1] select line.

Accesses to any address outside the above ranges will select the Default Slave.

Default Slave

This module drives the response when an access is detected to an unmapped region. The Default slave drives its HREADY output high, when other slaves are not selected. It drives a zero wait state OK response for IDLE and BUSY transfers and a two cycle ERROR response for an NSEQ or SEQ transfer access to an unmapped address.

6.2.4 Trickbox

The functionality of the VIC is verified via the Trickbox. The Trickbox is a programmable AHB slave designed to drive stimulus on the non-AHB input terminals of the VIC and sample its non-AHB outputs. The Trickbox implements a mirrored model of the VIC within it and continuously compares the locally generated outputs with the actual outputs generated by the VIC.

6.2.5 Protocol Checker

The testbench includes a protocol checker which reports any protocol or timing violations during accesses to any AHB slave. Timing parameters can be set using the `timing.v` file in the **`verification/verilog/tbench`** directory. The buswatch module residing in the **`verification/verilog/buswatcher`** directory checks the Read/Write databus timings and reports any timing violations.

6.2.6 Test Vectors

The test vectors for the verification of the VIC are coded into separate C files, in the **`verification/bustest`** directory, depending on the logical region of the VIC which these vectors are testing. Make options have been provided to use all the test vectors together or to use them individually.

6.2.7 Parameters

The VIC testbench provides some configurability options via parameters. All the parameters are configurable through the `tbench.v` file.

XonSig

XonSig is used in the `cyc_drivers.v` file that resides in **`verification/verilog/bid`**.

This parameter is used to eliminate the 'X's that appear on the address and control signals when these signals are not being actively driven. If **XonSig** is set to **0** in the top level of the testbench, these signals will go to '0' when the corresponding line driver is inactive. If set to **1**, the signals go to 'X' when the line driver is idle.

Verbosity

Verbosity is used in the `reset_driver.v` and the `cyc_drivers.v` which reside in **`verification/verilog/bid`**. It is also used in the `buswatch.v` that resides in **`verification/verilog/buswatcher`** and the `VIODrivers.v` which reside in **`verification/verilog/vr`**.

This parameter is used to control the verbosity of the transcript generated during simulation. If **Verbosity** is set to **1**, every command issued will have a corresponding message in the transcript file. If **Verbosity** is set to **0**, a message is issued only if an error occurs on a read.

HaltOnMismatch

This is used in the `cyc_drivers.v`, which resides in **`verification/verilog/bid`**, the `buswatch.v` which resides in **`verification/verilog/buswatcher`** and the `VIODriver.v` which resides in **`verification/verilog/vr`**.

This parameter is used to stop the simulation if an error occurs. If set to **1**, the simulation halts after an error is reported. If set to **0**, the error is flagged but the simulation continues.

SuppressOnReset

SuppressOnReset is used in the `buswatch.v`, which resides in **`verification/verilog/buswatcher`**.

This parameter is used to suppress all protocol checks on the slave's output signals when the HRESETn signal is asserted. If **SuppressOnReset** is TRUE, the set-up and hold timing violations on these signals are not flagged when HRESETn is asserted. If **SuppressOnReset** is set to FALSE, the set-up and hold timing violations are flagged irrespective of HRESETn. When **SuppressOnReset** is FALSE the slave's output signals are not checked for 'X'es when HRESETn is asserted.

TestMode

TestMode is used in the `cyc_drivers.v`, which resides in **`verification/verilog/bid`**.

If the **TestMode** parameter is set, the testbench is configured to run in Big Endian mode. By default this parameter is cleared.

Databuswidth

Databuswidth is used in the `cyc_drivers.v`, which resides in **verification/verilog/bid**.

This parameter controls the width of the AHB databus. For the VIC testbench this is set to **32**.

6.2.8 Running Simulations

The user's guide gives step by step instructions for running simulations using the VIC test vectors on the Verilog source code and the Verilog netlist.

Run time logs for the RTL simulations are:

- **verification/verilog/Vic_rtl/Vic_rtl_modelsim.log**
- **verification/verilog/Vic_rtl/Vic_rtl_LDV.sim**

Run time logs for the netlist simulations are:

- **verification/verilog/Vic_noscan_vhdl_net_min/Vic_noscan_vhdl_net_min_modelsim.log**
- **verification/verilog/Vic_scanready_verilog_net_typ/Vic_scanready_verilog_net_typ_LDV.log**

The code coverage results are:

- **verification/verilog/Vic_rtl/vnavigator.summary**

6.3 VIC Trickbox Description

The Trick Box is a programmable AHB slave, designed to drive stimulus on the non-AHB input terminals of the VIC and sample the non-AHB outputs of the same. The TrickBox also provides a means of protocol verification.

The Trickbox core logic mirrors the functionality of the VIC. This block generates outputs, which are to be compared with those from the VIC. The Trickbox continuously monitors the output from the VIC and flags error messages whenever a mismatch is found. The status of the interrupts generated by the VIC can also be read back via the Trickbox.

The following sections detail the functionality of the trickbox.

6.3.1 Trickbox Signal Description

The VIC TrickBox Signals are listed in **Table 6.3-1** and **Table 6.3-2**.

Name	Type	Source / Destination	Description
HCLK	Input	From Test Bench	AHB clock. This clock times all bus transfers. All signal timings are referenced to the rising edge of HCLK.
HRESETn	Input	From Test Bench	Bus reset signal, Active Low
HADDR[11:2]	Input	From Test Bench	AHB address bus.
HTRANS	Input	From Test Bench	Indicates the type of the current transfer. The VIC Trickbox does not support the BUSY transfer type. This signal is to be connected to HTRANS[1].
HSIZE[2:0]	Input	From Test Bench	Indicates the size of the current transfer. The VIC Trickbox and the VIC support only WORD transfer size.

HPROT	Input	From Test Bench	Protection Control Signal. This indicates that the transfer to the mirrored registers is a supervisor mode or user mode access. This signal is to be connected to HPROT[1].
HWRITE	Input	From Test Bench	AHB transfer direction signal, indicates a write access when HIGH, read access when LOW.
HRDATA[31:0]	Output	To Test Bench	AHB read data bus. It is used to transfer data from bus slaves to the master during read operations.
HWDATA[31:0]	Input	From Test Bench	AHB write data bus. During write operations, HWDATA is used to transfer data from the bus master to the slaves.
HREADYIN	Input	External Slave(s)	The slave will start the transfer only if HREADYIN is HIGH.
HREADYOUT	Output	From Trickbox	HREADYOUT along with an OK response indicates the successful completion of a data transfer.
HRESP[1:0]	Output	From Trickbox	The bus transfer response. The Trickbox returns only the OK response.
HSELVICTR	Input	From Test Bench	Trickbox select signal from the decoder. This select signal is used for Trickbox-specific register access.
HSELVIC	Input	From Test Bench	This select signal is used by the Trickbox to access registers that are common to the VIC and the Trickbox.

Table 6.3-1 AHB signal descriptions

Name	Type	Source/Destination	Description
nVICFIQ	Input	From VIC	nVICFIQ output of the VIC. The value on this input is continuously compared with the internally generated nVICFIQ.
nVICIRQ	Input	From VIC	nVICIRQ output of the VIC. The value on this input is continuously compared with the internally generated nVICIRQ.
VICVECTADDRROUT [31:0]	Input	From VIC	VICVECTADDRROUT output of the VIC. The value on this input is continuously compared with the internally generated VectAddrOut.
VICINTSOURCE [31:0]	Output	To VIC	Interrupt Requests to the VIC are raised via the VICINTSOURCE output of the Trickbox.
nVICFIQIN	Output	To VIC	nVICFIQIN Daisy Chain signal to the VIC.
nVICIRQIN	Output	To VIC	nVICIRQIN Daisy Chain signal to the VIC.
VICVECTADDRIN [31:0]	Output	To VIC	VICVECTADDRIN Daisy chain vector to the VIC.

Table 6.3-2 Non-AHB signal descriptions

6.3.2 Trickbox Register Description

Register Memory Map

Address (Offset from VIC Base)	Type	Width (R/W)	Reset Value	Name	Description
0x00C	R/W	32/32	0x00	VICTrIntSelect	Mirrored version of the VICIntSelect register.
0x010	R/W	32/32	0x00	VICTrIntEnable	Mirrored version of the VICIntEnable register.
0x014	W	00/32	-	VICTrIntEnClear	Mirrored version of the VICIntEnableClear register.
0x018	R/W	32/32	0x00	VICTrSoftInt	Mirrored version of the VICSoftInt register.
0x01C	W	00/32	-	VICTrSoftIntClear	Mirrored version of the VICSoftIntClear register.
0x020	R/W	1/1	0x00	VICTrProtection	Mirrored version of the VICProtection register.
0x030	R/W	32/32	-	VICTrVectAddr	Mirrored version of the VICVectorAddr register.
0x034	R/W	32/32	0x00	VICTrDefVectAddr	Mirrored version of the VICDefaultVectAddr register.
0x100-0x13C	R/W	32/32	0x00	VICTrVectAddr0-15	Mirrored version of the VICVectorAddr0-15 registers.
0x200-0x23C	R/W	6/6	0x00	VICTrVectCntl0-15	Mirrored version of the VICVectorCntl0-15 registers.

Note: A write to the VIC register will automatically update its mirrored register as well. If these mirrored VIC registers are accessed using the VIC Base address, the Trickbox returns zeros on the HRDATA bus. However, if these registers are accessed using the Trickbox base address (instead of the VIC Base address), their current contents are reflected onto the HRDATA bus.

Table 6.3-3 VIC-mirrored registers

Address (Offset from TrickboxBase)	Type	Width (bits)	Reset Value	Name	Description
0x000	R/W	3	0x00	VICTrTCR	Test mode select register. Bits [2:0] are used to enable warning messages.
0x004	R/W	32	0x00	VICTrIntSource	Source register for the VICINTSOURCE inputs to the VIC
0x008	R	2	0b11	VICTrStatus	Status of the nVICFIQ and nVICIRQ output pins from the VIC
0x024	R	32	0x00	VICTrVectAddrOut	Status register for the VICVECTADDRROUT output from the VIC.
0x028	R/W	2	0b11	VICTrIntIn	Source register for the nVICFIQIN and

					nVICIRQIN inputs to the VIC
0x02C	R/W	32	0x00	VICTrVectAddrIn	Source Register for the VICVECTADDRIN input to the VIC.
0x038	R/W	32	0x55555555	VICTrWaitStReg	An accessible location with Non-zero waitstates. This has been included to verify the HREADYIN going LOW.

Table 6.3-4 VIC Trickbox-specific registers

A few of the VIC's registers are mirrored in the Trickbox. The HSELVIC signal of the trickbox (same as that connected to the HSELVIC input of the VIC) is used to access the registers common to both the Trickbox and the VIC. The HSELVICTR signal of the Trickbox is used to access Trickbox specific registers. Hence, all the programmable registers in the Trickbox, common to both the VIC and the Trickbox would be selected for write when the corresponding register in the VIC is being written to. It is ensured that the Trickbox does not return data on reads to these registers. Additionally, it does not return responses to bus transfers for which HSELVIC is asserted. This avoids conflict on the AHB databus.

The Base address of the Trickbox is not fixed and depends on the system implementation. However, the offset of any particular register from the Base address is fixed.

A detailed description of each of the VIC Trickbox registers follows:

VICTrIntSelect: Mirrored Interrupt Select Register [32] (+0x00C)

The VICTrIntSelect register selects the interrupt request to generate FIQ or IRQ. Since this is a mirrored register, the changes in the corresponding register in the VIC will be reflected here as well.

Bits	Name	Reset	Description
31-0	VICTrIntSelect	0x00	Read/Write register used to select whether the corresponding interrupt request generates a FIQ or an IRQ interrupt. To select a FIQ, the corresponding bit should be set to 1. For an IRQ, the bit should be 0.

Table 6.3-5 VICTrIntSelect Register

VICTrIntEnable: Mirrored Interrupt Enable Register [32] (+0x010)

This register is used to mask all interrupt requests except the External interrupt via the Daisy chain. An enable bit value of 1 indicates that the interrupt is enabled. Write accesses to the corresponding register in the VIC changes the contents of this register.

When writing to this register, each data bit that is HIGH causes the corresponding bit in the enable register to be set. Data bits that are LOW have no effect on the corresponding bit in the register.

Bits	Name	Reset	Description
31-0	VICTrIntEnable	0x00	Register used for masking interrupt requests.

Table 6.3-6 VICTrIntEnable Register

VICTrIntEnClear: Mirrored Interrupt Enable Clear Register [32] (+0x014)

The VICTrIntEnClear is used to clear the VICTrIntEnable register. When writing to this register, each data bit that is HIGH causes the corresponding bit in the enable register to be cleared. This is a mirrored register and will reflect the changes in the corresponding VIC register.

Bits	Name	Reset	Description
31-0	VICTrIntEnClear	-	Write only register used for clearing VICTrIntEnable.

Table 6.3-7 VICTrIntEnClear Register**VICTrSoftInt: Mirrored Software Interrupt Register [32] (+0x018)**

This read/write register is used to generate Software interrupts. A value of 1 in any bit position of the VICTrSoftInt register generates an internal software interrupt. This is a VIC-mirrored register.

When writing to this register, each data bit that is HIGH causes the corresponding bit in the VICTrSoftInt register to be set. Data bits that are LOW have no effect on the corresponding bit in the register.

Bits	Name	Reset	Description
31-0	VICTrSoftInt	0x00	Software interrupt generation.

Table 6.3-8 VICTrSoftInt Register**VICTrSoftIntClear: Mirrored Software Interrupt Clear Register [32] (+0x01C)**

The VICTrSoftIntClear write only register is used to clear the VICTrSoftInt Register. It is a VIC-mirrored register.

When writing to this register, each data bit that is HIGH causes the corresponding bit in the VICTrSoftInt register to be cleared. Data bits that are LOW have no effect on the corresponding bits in the VICTrSoftInt register.

Bits	Name	Reset	Description
31-0	VICTrSoftIntClear	-	Software Interrupt clear.

Table 6.3-9 VICTrSoftIntClear Register**VICTrProtection: Mirrored Protection Register [1] (+0x020)**

The VICTrProtection read/write register is a VIC- mirrored register.

When the bit is set, only privileged mode accesses are allowed to the other mirrored registers.

Bits	Name	Reset	Description
0	VICTrProtection	0	Protection Bit.

Table 6.3-10 VICTrProtection Register

VICTrVectAddr: Mirrored Vector Address Register [32] (+0x030)

VICTrVectAddr is a read/write register which contains the interrupt service routine vector address of the currently active interrupt. Writing to this register clears the currently active Vectored interrupt. A Read will indicate that the handler for the currently active Vectored interrupt has been entered.

Bits	Name	Reset	Description
31-0	VICTrVectAddr	-	Contains the ISR Vector Address of the currently active interrupt.

*Table 6.3-11 VICTrVectAddr Register***VICTrDefVectAddr: Mirrored Default Vector Address Register [32] (+0x034)**

This read/write register contains the default ISR vector address.

Bits	Name	Reset	Description
31-0	VICTrDefVectAddr	0x00	Contains default vector address.

*Table 6.3-12 VICTrDefVectAddr Register***VICTrVectAddr0-15: Mirrored Vector Address 0-15 Registers of Vector Bank [32] (+0x100-0x13C)**

The VICTrVectAddr0-15 registers are read/write registers which contain ISR vector addresses.

Bits	Name	Reset	Description
31-0	VICTrVectAddr0-15	0x00	Contains ISR vector addresses.

*Table 6.3-13 VICTrVectAddr0-15 Registers***VICTrVectCntl0-15: Mirrored Vector Interrupt Control Registers [6] (+200-0x23C)**

The VICTrVectCntl0-15 registers are read/write registers which are used to select which interrupt source to use for the vectored interrupt.

Bits	Name	Reset	Description
5	EnVectIRQ	0x00	Enables Vectored IRQ.
4-0	IntSource	0x00	Interrupt Source select. Any of the 32 sources may be chosen.

*Table 6.3-14 VICTrVectCntl0-15 Registers***VICTrTCR: Trickbox Test Control Register [3] (+0x000)**

This read/write register is used as the test enable register. Bits in this register correspond to each output of the VIC. If a bit is HIGH, the corresponding output from the VIC and that from the internal model are compared.

Bits	Name	Reset	Description
0	FIQCompEn	0	nVICFIQ compare enable.
1	IRQCompEn	0	nVICIRQ compare enable.
2	VectAdCompEn	0	VectAddrOut comparison enable.

Table 6.3-15 VICTrTCR Register

VICTrIntSource: Interrupt Source Register [32] (+0x020)

This read/write register is used as the interrupt source for the VIC.

Bits	Name	Reset	Description
31-0	VICTrIntSource	0x00	Interrupt source for the VIC.

Table 6.3-16 VICTrIntSource Register**VICTrStatus: Interrupt Status Register [2] (+0x024)**

VICTrStatus is a read only register used to read the nVICFIQ and nVICIRQ output pins of the VIC.

Bits	Name	Reset	Description
1	nVICFIQ	0b1	nVICFIQ output pin status.
0	nVICIRQ	0b1	nVICIRQ output pin status.

Table 6.3-17 VICTrStatus Register**VICTrVectAddrOut: Vector Address Status Register [32] (+0x028)**

The VICTrVectAddrOut is a read-only register which is used for reading the VICVECTADDRROUT bus from the VIC.

Bits	Name	Reset	Description
31-0	VICTrVectAddrOut	0x00	Status register for the VICVECTADDRROUT output from the VIC.

Table 6.3-18 VICTrVectAddrOut Register**VICTrIntIn: External Interrupt Input Register [2] (+0x02C)**

The VICTrIntIn register controls the nVICFIQIN and nVICIRQIN inputs of the VIC.

Bits	Name	Reset	Description
0	nVICFIQIN	0b1	nVICFIQIN input to the VIC.
1	nVICIRQIN	0b1	nVICIRQIN input to the VIC.

Table 6.3-19 VICTrIntIn Register**VICTrVectAddrIn: External Interrupt Vector Address Input Register [32] (+0x030)**

The VICTrVectAddrIn register controls the VICVECTADDRIN input bus to the VIC.

Bits	Name	Reset	Description
31-0	VICTrVectAddrIn	0x00	VICVECTADDRIN input to the VIC

Table 6.3-20 VICTrVectAddrIn Register

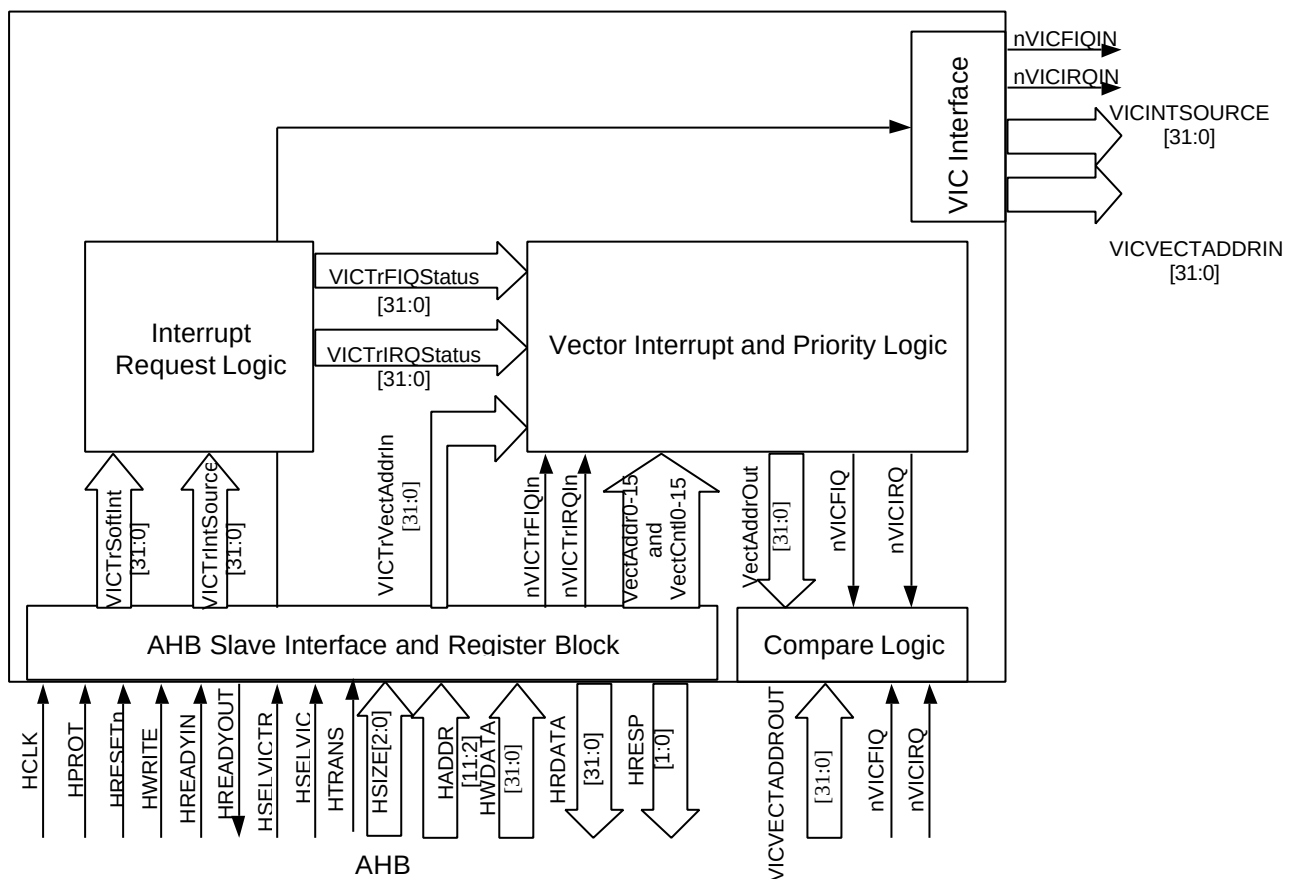
VICTrWaitStReg: External Interrupt Vector Address Input Register [32] (+0x038)

The VICTrWaitStReg register inserts 10 wait states when it is accessed. This Non-Zero wait state register has been included to verify the HREADYIN of the VIC going LOW.

Bits	Name	Reset	Description
31-0	VICTrWaitStReg	0x55555555	An accessible location with Non-zero waitstates. Returns 0x55555555 when it is read.

Table 6.3-21 VICTrWaitStReg Register**6.3.3 Trickbox functional description**

The primary function of the TrickBox is to drive the non-AHB inputs to the VIC and check the non-AHB outputs from the VIC. The basic blocks of the trickbox are illustrated in **Figure 6.3-1**.

**Figure 6.3-1 VIC Trickbox Block Diagram****AHB Interface and Register Block**

The Trickbox interfaces to the external world via the AHB interface. A basic AHB slave interface is supported to provide access to the internal registers. It does not support the Retry or Split responses. The AHB interface sub-module generates read and write decodes for accesses to internal registers of the Trickbox. The register block stores data written or to be read across the AHB interface. Most of the registers are READ/WRITE; those which are READ

ONLY or WRITE ONLY are specified explicitly. WRITE ONLY registers are mere addresses rather than a register. WRITE accesses to these 'registers' cause changes in some other specific registers.

Interrupt Request Logic

The Interrupt Request Logic generates the FIQ and IRQ status after masking. It is implemented as shown in **Figure 6.3-2**. This block receives the interrupt requests and software interrupts from the Register block and combines them to generate the status. VICTrIntEnable and VICTrIntSelect register contents are used for masking the raw interrupts.

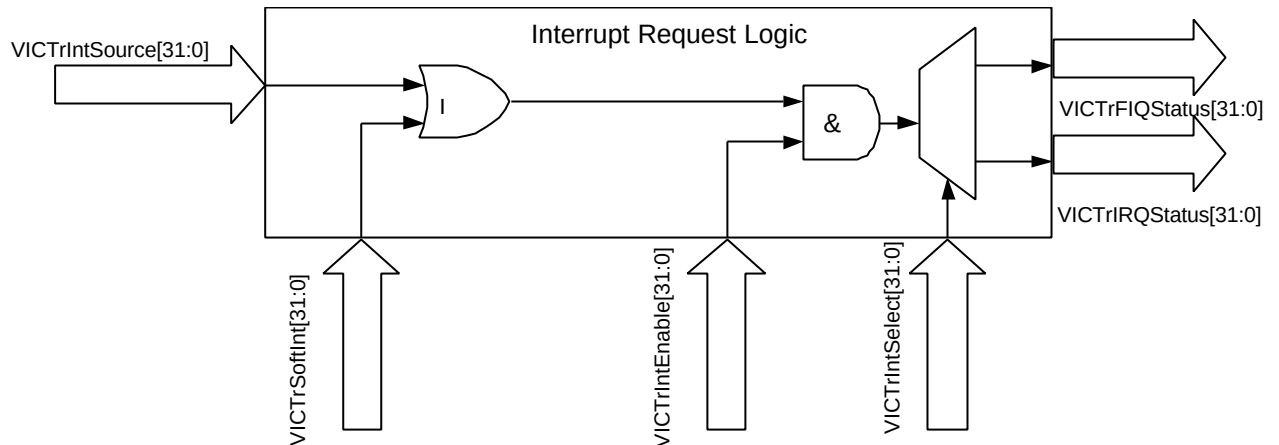


Figure 6.3-2 Interrupt Request Logic

Vector Interrupt and Priority Logic

This block (VicTrVectBank) generates the nVICTrFIQ, nVICTrIRQ and VICTrVectAddrOut signals which are to be compared with the nVICFIQ, nVICIRQ and VICVECTADDROUT outputs from the VIC. During testing, the mirrored registers in the Trickbox are loaded with the same data as the corresponding registers in the VIC.

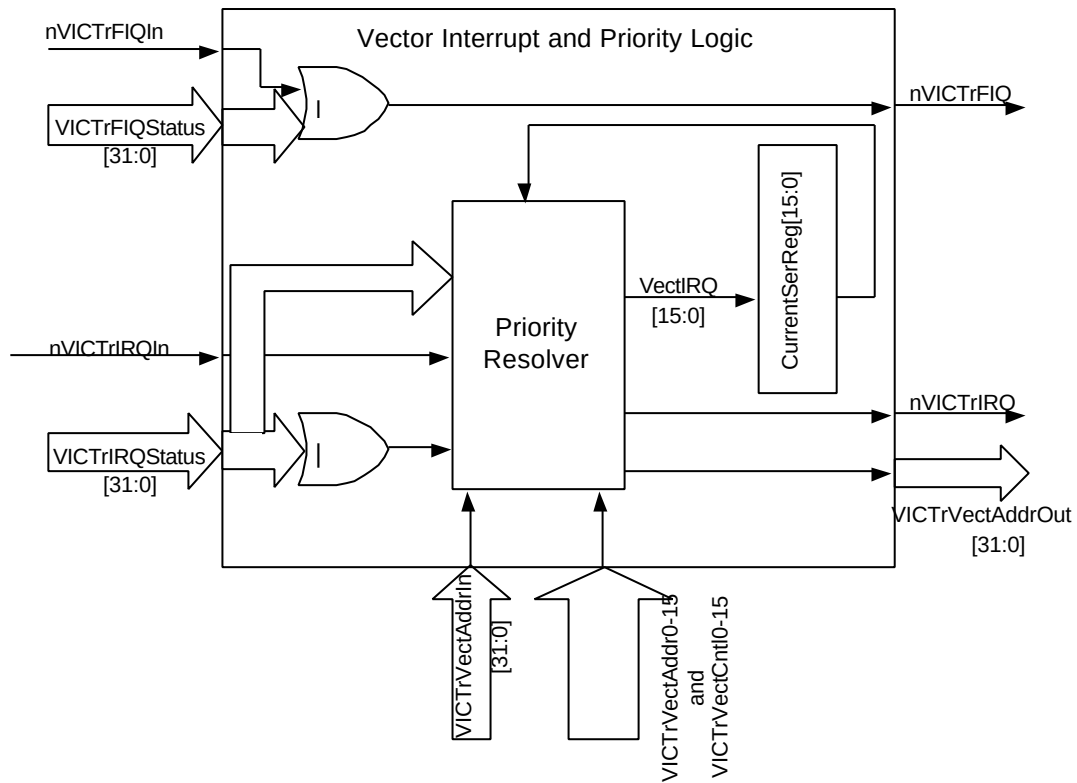


Figure 6.3-3 Vector Interrupt and Priority Logic

The VicTrVectBank module resolves the highest priority interrupt from the currently active interrupts and masks all others from raising interrupts. It masks itself from going active again during the ISR execution. A more detailed block diagram of the Priority Resolver is shown in **Figure 6.3-4**. The Current Service Register [CSR] holds the status of the currently pending interrupts. A read access to the Vector Address register sets the bit corresponding to the currently active highest priority IRQ in the CSR and a write access clears the same.

The active VectIRQ outputs and ActiveNonVectIRQ from the Priority Resolver is clocked into the CSR during read access to the Vector Address register and the CSR bits are fed back to the Resolver logic for masking it from further assertion. Appropriate Vector Address is driven to the VICTrVectAddr register.

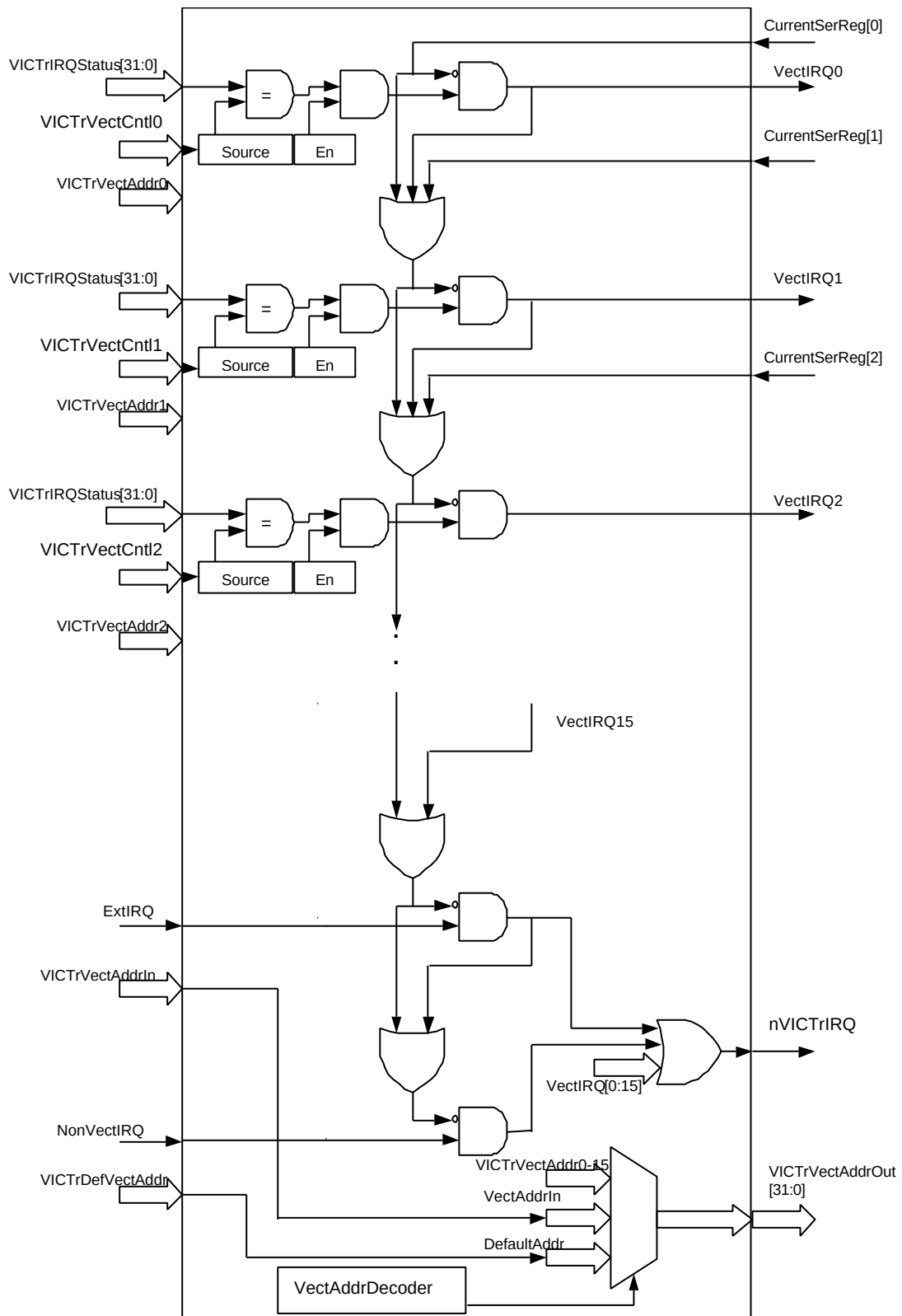


Figure 6.3-4 Priority Resolver Logic

Compare Logic

The Compare Logic (VicTrProtChkr) compares the locally generated outputs with the corresponding signals from the VIC. Bits [2:0] of the VICTrTCR register controls the duration for which the compare logic is active. When these bits are HIGH, the Trickbox compares the outputs at every rising edge of the HCLK. Whenever a mismatch in the corresponding output is found, it flags error messages.

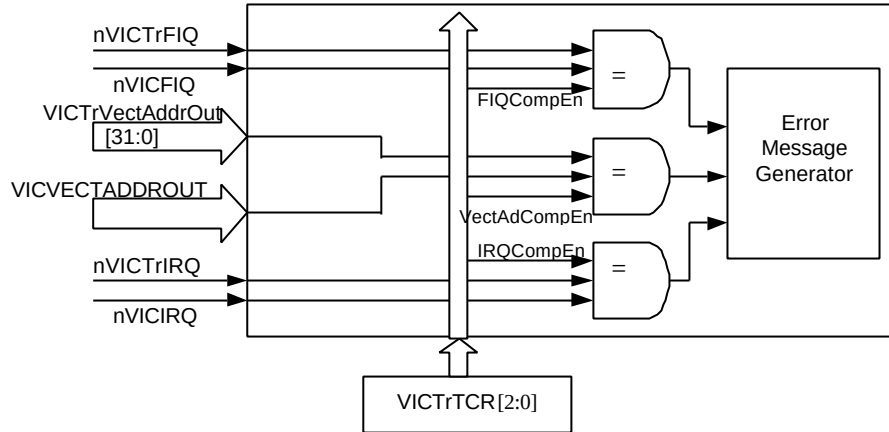


Figure 6.3-5 Compare Logic

6.4 Functional Tests

The following sections describe the test cases to validate the VIC.

It is to be noted that the majority of the Interrupt tests rely heavily on the on-line comparisons done in the Trickbox. This method has advantage that the compare accuracy can be done at nano-second level if need be [in the actual Trickbox implementation, the compare granularity is kept to an HCLK period to eliminate false mismatches during netlist simulations]. The number of test vectors required to prove functionality is much lesser than a method which predominantly uses reads of the VIC and Trickbox registers.

6.4.1 Reset Tests

This test is invoked by the ResetTests() function call.

After a reset, each of the following registers are read to check if they contain the specified reset value.

VIC Register Name	Reset Value
VICIRQStatus	0x00000000
VICFIQStatus	0x00000000
VICRawInterrupt ¹	0x00000000
VICIntSelect	0x00000000
VICIntEnable	0x00000000
VICSoftInt	0x00000000
VICProtection	0b0
VICVectAddr ²	0x00000000

VICDefVectAddr	0x00000000
VICITCR	0x00000000
VICITIP1 ³	0x000000C0
VICITIP2 ⁴	0x00000000
VICITOP1 ⁵	0x00000000
VICITOP2 ⁶	0x00000000
VICVectAddr0-15	0x00000000
VICVectCntl0-15	Bits [5:0]:- 0b000000
VICPeriphID0	0x00000090
VICPeriphID1	0x00000011
VICPeriphID2	0x00000004
VICPeriphID3	0x00000000
VICPCellID0	0x0000000D
VICPCellID1	0x000000F0
VICPCellID2	0x00000005
VICPCellID3	0x000000B1

Table 6.4-1 Reset values of VIC registers

1. Note that the VICRawIntr is the logical OR of the VICINTSOURCE and the VICSoftInt bits. The value reflected in the VICRawIntr register will depend on the values driven on the external VICIntSource pins since the VICSoftInt register is reset to 0x00000000. The reset value tests assume that the source of VICINTSOURCE (the Trickbox in the functional verification world) drives these pins to 0x00000000 after reset. If the source of the VICINTSOURCE drives these to a value other than 0x00000000, the test code will have to be modified accordingly.
2. Note that the VICVectAddr is derived from the internally generated Vector Address and the external Daisy Chained Vector Address (VICVECTADDRIN). The value reflected in the VICVectAddr register will depend on the values driven on the external VICVECTADDRIN and nVICIRQIN pins since the internally generated Vector Address will be reset to 0x00000000. The reset value tests assume that the source of VICVECTADDRIN and nVICIRQIN (the Trickbox in the functional verification world) drives these pins to 0x00000000 and 1 respectively after reset. If the source of the VICVECTADDRIN and nVICIRQIN drives these to a value other than 0x00000000, the test code will have to be modified accordingly.
3. Note that the VICITIP1 is derived from the internally generated Interrupts and external Daisy Chained nVICIRQIN and nVICFIQIN interrupts. The value reflected in the VICITIP1 register will depend on the values driven on the external nVICIRQIN and nVICFIQIN pins since the internally generated Interrupts will be reset to 0x00000000. The reset value tests assume that the source of nVICIRQIN and nVICFIQIN (the Trickbox in the functional verification world) drives these pins inactive after reset. If the source of the nVICIRQIN and nVICFIQIN drives these to their active values after reset, the test code will have to be modified accordingly.
4. Note that the VICITIP2 is derived from the internally generated Vector Address and the external Daisy Chained Vector Address (VICVECTADDRIN). The value reflected in the VICITIP2 register will depend on the values driven on the external VICVECTADDRIN pins since the internally generated Vector Address will be reset to 0x00000000. The reset value tests assume that the source of VICVECTADDRIN (the Trickbox in the functional verification world) drives these pins to 0x00000000 after reset. If the source of the

VICVECTADDRIN drives these to a value other than 0x00000000, the test code will have to be modified accordingly.

5. *Note that the VICITOP1 is derived from the internally generated Interrupts and the external Daisy Chained nVICIRQIN and nVICFIQIN interrupts. The value reflected in the VICITOP1 register will depend on the values driven on the external nVICIRQIN and nVICFIQIN pins since the internally generated Interrupts will be reset to 0x00000000. The reset value tests assume that the source of nVICIRQIN and nVICFIQIN (the Trickbox in the functional verification world) drives these pins inactive after reset. If the source of the nVICIRQIN and nVICFIQIN drives these to their active values after reset, the test code will have to be modified accordingly.*
6. *Note that the VICITOP2 is derived from the internally generated Vector Address and the external Daisy Chained Vector Address (VICVECTADDRIN). The values reflected in the VICITOP2 register will depend on the values driven on the external VICVECTADDRIN and nVICIRQIN pins since the internally generated Vector Address will be reset to 0x00000000. The reset value tests assume that the source of VICVECTADDRIN and nVICIRQIN (the Trickbox in the functional verification world) drives these pins to 0x00000000 after reset. If the source of the VICVECTADDRIN drives these to a value other than 0x00000000, the test code will have to be modified accordingly.*

Initially HRESETn is asserted and the registers VICIRQStatus, VICFIQStatus, VICRawInterrupt, VICIntSelect, VICIntEnable, VICSoftInt, VICProtection, VICVectAddr, VICDefVectAddr, VICITCR, VICITIP1, VICITIP2, VICITOP1, VICITOP2, VICVectAddr0-15, VICVectCntl0-15, VICPeriphID0, VICPeriphID1, VICPeriphID2, VICPeriphID3, VICPCellID0, VICPCellID1, VICPCellID2 and VICPCellID3 are read back to verify that the reset values are returned.

6.4.2 Register Tests

This test is invoked by the RegisterTests() function call.

Read-Write Registers

The data patterns 0x55555555, 0x00000000, 0xFFFFFFFF and 0xAAAAAAAA are written to each of the Read-Write registers. After each write, the register contents are read back to see if it contains the written value for the registers VICIntSelect, VICDefVectAddr, VICVectAddr0-15, VICVectCntl0-15 and VICITCR. If the returned values are the same as the expected values, then the register is declared to be writeable and readable and the test passes.

Prior to the write to the VICVectAddr register, the VICDefVectAddr is cleared. This is done because the value in the VICVectAddr is actually a reflection of the VICDefVectAddr when no interrupts are active. Hence, after writing a pattern in the VICVectAddr register, the data expected while reading is zero (the content of the VICDefVectAddr is at zero at this point).

For testing the VICProtection register, a write and a read is performed to it. If the HPROT[1] is low (indicating a user access) during the write-read operation, then this value is temporarily stored and the HPROT[1] is made high for the write-read access. If the pattern read is the same as that was written then the register is declared to be writeable and readable and the test passes.

To test the VICITIP registers, the VIC is configured in integration test mode by setting the ITEN bit in the VICITCR. A data pattern is written into the VICITIP1 register and read back from the 6th and 7th bits. If the returned values are the same as the expected value, then the register is declared to be writeable and readable and the test passes. Similarly the patterns written in the VICITIP2 should be returned while reading back. The VICITIP1 and VICITIP2 registers are cleared before proceeding further. Now, the VIC is configured back into normal mode by clearing the ITEN bit in the VICITCR. At this moment, the pattern coming in through the VICVECTADDRIN is reflected in the VICITIP2. Similarly, the values on the nVICIRQIN and nVICFIQIN inputs will be reflected in the 6th and 7th bits of the VICITIP1 register. The contents of the VICVECTADDRIN are made zero by writing to the VICTrVectAddrIn register of the Trickbox. The nVICIRQIN and nVICFIQIN are set (to indicate no external interrupts) by writing into the VICTrIntIn register. Now, different data patterns are written into the VICITIP1 and VICITIP2 registers. But, the content of these registers should not change. Now, the contents of VICTrVectAddrIn and VICTrIntIn are changed. This change should be reflected in the VICITIP1 and VICITIP2 registers.

To test the VICIntEnable and VICSoftInt registers, a different method is adopted as their contents can be altered by writing into their respective clear registers as well. As the reset value of the VICIntEnable and the VICSoftInt is ZERO, initially writing a pattern of 0x55555555 will return the same pattern on reads. Now a pattern of 0x99999999 is written and the pattern 0xDDDDDDDD is expected while reading. Then, a pattern of 0x22222222 is written and a pattern of 0xFFFFFFFF is expected while reading.

Now, the Write-only registers, VICIntEnClear and VICSoftIntClear can be used to clear the bits in the enable registers. Data pattern 0x33333333 is written into the respective clear registers and the pattern 0xCCCCCCCC is expected from the enable registers. Data pattern 0x55555555 is written into the clear registers and the pattern 0x88888888 is expected from the enable registers. Data pattern 0x88888888 is written into the clear registers and the pattern 0x00000000 is expected from the enable registers.

Read-only Registers

For read-only registers the data patterns 0x55555555, 0x00000000, 0xFFFFFFFF, 0xAAAAAAAA are written and after each write, read back to see if the default values (reset values) for the registers VICIRQStatus, VICFIQStatus, VICRawIntr, VICITOP1, VICPeriphID0, VICPeriphID1, VICPeriphID2, VICPeriphID3, VICPCellID0, VICPCellID1, VICPCellID2 and VICPCellID3 are returned. If the default values are returned on reads, these registers are read-only and the test passes.

Since, the VICITOP2 register indicates the status of the VICVectAddrROUT, the content of the VICDefVectAddr should be changed before any write into the VICITOP2. While reading this register back after the write, the data returned should be its reset value.

6.4.3 Transfer Test

This test is invoked by the TransferTests() function call.

Burst Transfer

A sequence of writes is performed to the VICVectAddr0-15 and VICVectCntl0-15 registers in a burst. Then the registers are read back in a burst with the expected data same as the written data. If all the registers return the expected data, the test passes.

In addition to this, burst transfers are also done in between the Error Response test, Interrupt test and the Protection Mode test. In the Error Response test, the VICVectCntl0-15 registers are accessed in a burst with HSIZE indicating a half-word transfer type. In the Interrupt test, the status registers and the VICRawIntr are read in a burst. Similarly, the initialisation of the vector and control registers are done using a burst access. In the Protection Mode test, all the burst transfers performed in this function are repeated with the Protection bit set.

BUSY Transfer

A burst write is performed to the VICVectAddr0-15 and VICVectCntl0-15 registers with BUSY cycles added in between. Then, a burst read is performed on these registers with BUSY cycles inserted. The points at which the BUSY cycles are inserted during the burst is varied. If the expected data is the same as the data read, then the test passes.

In addition to this, BUSY transfers are also performed in between the Interrupt test and the Protection Mode test. In the Interrupt test, BUSY cycles are added between the burst write access to the VICVectAddr0-15 registers. In the Protection Mode test, all the BUSY transfers performed in this function are repeated with the Protection bit set.

IDLE Transfer

A burst write is performed on the VICVectAddr0-15 and VICVectCntl0-15 registers with IDLE cycles added in between. Then, a burst read is performed on these registers with IDLE cycles added at some other points. If the expected data is the same as the data read, then the test passes.

In addition to this, IDLE transfers are also performed in between the Interrupt test and the Protection Mode test. In the Interrupt test, IDLE cycles are added between the burst write access to the VICVectAddr0-15 registers. In the Protection Mode test, all the BUSY transfers performed in this function are repeated with the Protection bit set.

6.4.4 Error Response Test

This test is invoked by the ErrorRespTests() function call.

A write access is performed to all the registers with the HSIZE as Byte or Half-Word. A burst access is performed to the VICVectCntl0-15 registers with HSIZE as Half-Word. If, in all cases an Error response is returned, the test passes.

6.4.5 Protection Mode Test

This test is invoked by the ProtectionTests() function call.

The functionality being tested is that when the VIC is in 'User' mode, both 'Supervisor' and 'User' mode accesses should be possible and when the VIC is in 'Protected' mode, only 'Supervisor' accesses should be possible.

The VIC is configured in 'Protected' mode by setting the Protection bit high. Now, a write-read test is performed in the 'Supervisor' mode. Data pattern 0x55555555 is written into the VICIntSelect, VICDefVectAddr, VICVectCntl0-15, VICVectAddr0-15, VICITCR, VICVectAddr, and VICProtection registers. The data read back should be exactly the same as that was written. Before writing to VICVectAddr, the content of VICDefVectAddr is cleared. Hence, the data value read from the VICVectAddr should be ZERO.

The read-only registers VICIRQStatus, VICFIQStatus, VICRawIntr, VICITOP1, VICITOP2, VICPeriphID0-3 and VICPCellID0-3 are also written with 0x55555555, but the expected value while reading is their reset value.

While testing the VICITIP registers, the ITEN bit of the VICITCR is enabled and then disabled.

After the above test, the VIC is accessed in 'User' mode. A write-read access is performed to all the above registers. The data pattern written is 0xAAAAAAAA. This time the expected value while reading is ZERO. Now, it has to be proved that the above access in the 'User' mode didn't change the content of any of the registers. Hence, again a 'Supervisor' mode read access is done on all the registers. The read-only registers should return their reset value and the read-write registers should return the previously written pattern, 0x55555555.

A 'User' mode burst access is done to all the VICVectAddr0-15 and VICVectCntl0-15 registers. The pattern read back should be ZERO. BUSY cycles are added in between the burst access and a ZERO value is expected while reading. Similarly, IDLE cycles are added in between the burst access and a ZERO value is expected while reading.

Half-word or Byte write accesses are done on the VICVectAddr, VICProtection and some of the control and address registers. The expected response is an ERROR response.

6.4.6 Protection Mode Interrupt Test

This test is invoked by the ProtModeIntTests() function call.

This test verifies that 'User' mode reads or writes to the VICVectAddr register do not affect the Priority Hardware. The test flow is as follows:

- Interrupt Source 1 - VICINTSOURCE[1] - is configured to raised an IRQ via Vector Bank 1.
- The Default Vector Address is programmed as 0x12345678
- Vector Bank 1 is programmed with a Vector Address of 0x11111111.
- Generate the interrupt via the Trickbox. The VICINTSOURCE is 0x00000002 (i.e., 0b0000_0000_0000_0000_0000_0000_0000_0010). Interrupt source 1 is active.
- The Trickbox compare logic verifies that the nVICIRQ signal is LOW.
- The Trickbox compare logic verifies that the VICVECTADDRROUT lines should contain 0x11111111.
- The Protection mode bit is set in the VICProtection register.

- A User mode read access is performed to the VICVectAddr register in the VIC. If the access were a protected mode access or if the Protection mode bit were cleared, this read should have resulted in the masking of the currently active nVICIRQ output. However, since the current access is in User mode, bit 1 of the CurrentService register should NOT get set. The nVICIRQ output should continue to remain LOW. The Trickbox compare logic verifies that the nVICIRQ line is indeed low. It also verifies that the Vector Address corresponding to Vector Bank 1 (0x11111111) continues to be present on the VICVECTADDROUT lines. This proves that the User mode read has had no effect on the internal priority mask.
- Now a Protected mode read access is performed to the VICVectAddr register in the VIC. This should cause bit 1 of the CurrentService register to be set, causing the nVICIRQ line to go inactive. The VICVECTADDROUT should now reflect the Default Vector Address since no interrupts are active. The trickbox compare logic verifies that the interrupt line status and the vector address values are as expected.
- All interrupt sources are cleared (VICINTSOURCE = 0x00000000)
- The trickbox compare logic continues to verify that the nVICIRQ is HIGH and the vector address is the default vector address.
- Interrupt source 2 is raised via Vector Bank 2 (VICINTSOURCE = 0x00000004)
- Vector Bank 2 is programmed with a Vector Address of 0x22222222.
- The Trickbox compare logic continues to verify that the nVICIRQ is HIGH and the vector address is the default vector address. This is because Vector Bank 2 is at a lower priority than Vector Bank 1. Since bit 1 of the CurrentService register is still set, Vector Bank 2 is unable to raise an IRQ.
- A User mode write access is performed to the VICVectAddr register. Bit 1 in the CurrentService register should continue to remain set. Consequently the priority mask should also remain the same. The trickbox compare logic verifies that the nVICIRQ line remains inactive and the default vector continues to get driven on the VECTADDROUT lines.
- A Protected mode write access is performed to the VICVectAddr register. Bit 1 in the CurrentService register should get cleared. Since the CurrentService register does not have any set bits in it, all Vector Banks will be unmasked. The pending Vector Bank 2 interrupt will be propagated on to the nVICIRQ line along with its corresponding Vector Address (0x22222222). The Trickbox compare logic verifies that the values are as expected.
- The Protection bit is reset using a protected mode access.
- The VICVectAddr register is read, masking the active Interrupt source 2.
- VICINTSOURCE is written with 0x00000000.
- The VICVectAddr register is written, clearing the set bit in the CurrentService register.

The Trickbox compare logic continuously monitors the VICVECTADDROUT, nVICIRQ and nVICFIQ output lines, flagging error messages if there are any mismatches.

6.4.7 Basic Interrupt Tests

The Interrupt tests are invoked by the UserOneIntTests(), TwoFourIntTests() and EightAllIntTests() function calls.

These tests create various combinations of interrupting conditions. Interrupt functionality is verified by a combination of on-line checking by the Trickbox and by selective reads of Status registers.

The overall flow of the Interrupt test suite as well as functionality of key support functions is described:

- Program the VICIntEnable, VICSoftInt, VICIntSelect, VICTrIntIn, VICProtection, VICTrVectAddrIn, VICVectAddr0-15 and VICVectCntl0-15 registers. This is done by the functions InitValue() and Initialise(). When the function Initialise() is called with the EnVectIRQ argument as 1, the Vector bank is enabled; else when EnVectIRQ is 0, the Vector bank is disabled. Tests are done with 32 different combinations of these register settings.
- A variable D_TrIntSource is assigned a value equivalent to the number of interrupts to be applied. For e.g, if it is a one interrupt test, then the binary format of the value assigned to the D_TrIntSource will have exactly one of the

bits as 1. The function `ApplyIntr()` is used to write this value into the register `VICTrIntSource` of the Trickbox (or the `VICSoftInt` of the VIC). When the argument of the function `ApplyIntr()` is 0, `VICTrIntSource` is used and when it is 1, `VICSoftInt` is used.

- The function `AllIntService()` is used to service and clear all the interrupts in the order of their priorities.
- The function `ReadStatus()` reads the Status registers viz., `VICIRQStatus`, `VICFIQStatus` and `VICRawIntr` expecting the desired value. A variable called CSR is written with the information about the number of interrupt sources raising an interrupt via the vector bank. Then, according to the value in the register `VICFIQStatus`, the function `FIQServ()` services and clears all the interrupt sources requesting the FIQ interrupt, beginning from the 0th position. Servicing, in this context, means reading all the status registers. Then, the next higher priority interrupts are the Vectored IRQs. For this, the content of the CSR is read. If it indicates presence of any Vectored IRQ, then the function `VectIRQServ()` is invoked, servicing and clearing all the interrupts raising Vectored IRQs in the order of their priority. Here, servicing is described as the time between a read from the `VICVectAddr` to a write to the `VICVectAddr`. In between this period, all the status registers are read. Now, the Non-Vectored IRQs are serviced and cleared. For this the `VICIRQStatus` register is checked. Again all the Non-Vectored IRQs are serviced and cleared from the 0th position. For this, the function `NonVectIRQServ()` is invoked. Here also servicing is the time between a read from the `VICVectAddr` to a writer to the `VICVectAddr`.

Tests are performed applying one, two, four, eight, sixteen and thirty-two interrupts at a time via the Trickbox or the `VICSoftInt` register. All these tests employ the procedure explained above. Apart from these some specific interrupt tests are performed and they are described in **Section 6.4.8 Special Interrupt Tests**.

There are three types of interrupts: Non-Vectored FIQ (maximum 32 in number), Vectored IRQ (maximum 16 in number) and Non-Vectored IRQ (maximum 32 in number). Note that the external interrupt is not considered here. So, a total of 3 different types of interrupt requests are possible in the One-Interrupt case. Of these all the 3 types have been covered. For the Two-Interrupt case, 6 different types of interrupt requests are possible, all of which are covered. For the Four-Interrupt case, all of the 15 different types of interrupt requests are covered. For the Eight-Interrupt case, 27 of the possible 45 types of interrupt requests are covered. For the Sixteen-Interrupt case, 24 out of possible 153 types are covered. For the Thirty-two interrupts case, 12 out of the possible 561 types are covered.

One Interrupt Test

This test is invoked by the `UserOneIntTests()` function call. This test can be divided into 3 categories:

- Raising of a FIQ.
- Raising of a Vectored IRQ.
- Raising of a Non-Vectored IRQ.

In all these categories, the VIC is tested with an active external interrupt as well. In some cases, certain interrupts are disabled by resetting the corresponding bit in the `VICIntEnable` register. In all, 1024 tests are performed for the One-Interrupt case. Each of the 3 categories is illustrated with one example each:

Program the following initialising values in the following registers.

<code>VICIntEnable</code>	<code>0xFFFFFFFF</code>	All the interrupts are enabled
<code>VICSoftInt</code>	<code>0x00000000</code>	None of the interrupts can occur through the <code>VICSoftInt</code> register
<code>VICIntSelect</code>	<code>0xAAAAAAAA</code>	Decides which of the sources will raise a FIQ
<code>VICTrIntIn</code>	<code>0x00000003</code>	Indicates no external interrupts
<code>VICTrVectAddrIn</code>	<code>0x00000000</code>	External Vector Address
<code>VICProtection</code>	<code>0x00000000</code>	VIC is in the 'User' mode
<code>VICVectAddr0</code>	<code>0x0000A000</code>	Address for Vector bank 0 is <code>0x0000A000</code>

VICVectAddr1	0x0000A004	Address for Vector bank 1 is 0x0000A004
...	...	...
VICVectAddr15	0x0000A03C	Address for Vector bank 15 is 0x0000A0C4
VICVectCntl0	0x00000028	Interrupt source 8 will raise a vectored interrupt request
VICVectCntl1	0x00000029	Interrupt source 9 will raise a vectored interrupt request
...	...	...
VICVectCntl15	0x00000037	Interrupt source 23 will raise a vectored interrupt request

Category 1 - FIQ

In this illustration, the value 0x00000008 is written into the VICTrIntSource. This is done by invoking the function `ApplyIntr()` with the argument as 0. This indicates that source 3 is raising an interrupt. Since, the bit 3 of the VICIntSelect is 1, it will be a FIQ interrupt.

To test the VIC the following have to be performed:

- Check the Status registers for the expected values. This marks the ISR entry.
- Check that the nVICFIQ pin is low.
- Clear the bit raising the interrupt in the VICTrIntSource.
- Again check the Status registers.

To perform the above tasks, the function `AllIntService()` is invoked. This function is used to service and clear all the interrupts in the order of their priorities. When the argument of this function is 0, it indicates the interrupts are being raised by the VICTrIntSource register of the Trickbox. When the interrupts are raised via the VICSoftInt register, the argument is 1.

Since, the status registers are clocked out by the VIC, due to the AHB pipelining and the double synchronisation of the status registers, the change in these registers are visible on the read data bus only after two clock cycles. Hence, in the function `AllIntService()` and the `ReadStatus()`, IDLE cycles are added to compensate for this two clocks discrepancy.

Now, the `ReadStatus()` function is called. Here the VICIRQStatus is read expecting a pattern of 0x00000000. The VICFIQStatus is read expecting a pattern of 0x00000008. Pattern 0x00000008 is expected from the VICRawIntr register. To read these three registers, a burst access is done.

Next, the function `CurServReg()` is invoked. The details of the interrupt sources raising vectored interrupts are stored in a 16-bit variable called CSR. The value stored in the CSR should be 0x0000.

Now, the VICFIQStatus is checked. If there is a non-zero bit, it indicates that there is an active FIQ interrupt. Hence, the function `FIQServ()` is accessed with the argument zero (indicating that it is from the VICTrIntSource). This function clears all the FIQ interrupts. By comparing with the VICFIQStatus register, it is possible to determine which bits of the VICTrIntSource are currently requesting a FIQ. Now, beginning from the 0th position, all the FIQ interrupt sources are cleared by writing 0s in the corresponding positions in the VICTrIntSource register of the Trickbox.

In the present illustration, the status registers are read. Expected data from the VICIRQStatus, VICFIQStatus and VICRawIntr are 0x00000000, 0x00000008 and 0x00000008 respectively. The bits of the VICFIQStatus are read from the 0th position. As soon as, there is a 1 (here, it is the 3rd bit), the VICITOP1 is read with the expected value as 0x00000040. Now, the 3rd bit of the VICTrIntSource is cleared. The 0th bit of the VICTrIntIn is checked to find any pending external FIQ.

Now, the VICFIQStatus register is checked for any other active FIQs. In this illustration, no FIQs should be pending.

The VICIRQStatus, is checked for any IRQ interrupts.

Now, the Status registers are read again expecting 0x00000000, 0x00000000 and 0x00000000 from the VICIRQStatus, VICFIQStatus and VICRawIntr registers respectively.

Category 2 - Vectored IRQ

In this illustration the value 0x00001000 is written into the VICTrIntSource. This is done by invoking the function ApplyIntr() with the argument as 0. This indicates that source 12 is raising an interrupt. Since bit 12 of the VICIntSelect is 0, it will be an IRQ interrupt. Now, the content of VICVectCntl4 is 0x0000002C. The last six bits are 0b101100. This indicates that the interrupt source 12 will raise a vectored interrupt via the 4th vector bank. Hence, the vector address should be the content of the register VICVectAddr4 i.e., 0x0000A010.

To test the VIC the following have to be performed:

- Check the Status registers for expected values.
- Check that nVICIRQ pin has gone low.
- Read the VICVectAddr register expecting the required vector address. This marks the beginning of the interrupt service routine.
- Clear the bit raising the interrupt in the VICTrIntSource.
- Perform a write access to the VICVectAddr. This marks the end of the interrupt service routine.
- Again check the Status registers.

To perform the above tasks, the function AllIntService() is invoked. This function is used to service and clear all the interrupts in the order of their priorities. When the argument of this function is 0, it indicates that the interrupts are being raised by the VICTrIntSource register of the Trickbox. When the interrupts are raised via the VICSoftInt register, the argument is 1.

Now the ReadStatus() function is called. Here the VICIRQStatus is read expecting a pattern of 0x00001000. The VICFIQStatus is read expecting a pattern of 0x00000000. Pattern 0x00001000 is expected from the VICRawIntr register. To read these three registers, a burst access is done.

Next, the function CurServReg() is invoked. The details of the interrupt sources raising vectored interrupts are stored in a 16-bit variable called CSR. Here, the interrupt is raised via the 4th vector bank. Hence, the 4th bit of the CSR should be set. Thus, the value stored in the CSR should be 0x0010.

The VICFIQStatus, is checked for any FIQ interrupts.

Now, the VICIRQStatus is checked. If there is a non-zero bit, it indicates that there is an active IRQ interrupt. Now the value of CSR decides whether the IRQ is vectored or non-vectored. Since, the CSR is non-zero (indicating the presence of an active vectored IRQ), the function VectIRQServ() is accessed with the argument zero (indicating that it is from the VICTrIntSource). This function clears all the IRQ interrupts in the order of their priorities. The priority of the interrupts goes on decreasing when we move from the 0th to the 15th bit in the CSR. The CSR is read from the 0th bit. As soon as any non-zero bit is seen, the interrupt service routine is started. The routine consists of reading the VICITOP registers, address register and then clearing the interrupt and writing to the address register. At the same time the content of the CSR is also changed.

In the present illustration, the status registers are read and the expected data from the VICIRQStatus, VICFIQStatus and VICRawIntr are 0x00001000, 0x00000000 and 0x00001000 respectively. The bits of the CSR are read from the 0th position. As soon as there is a 1 (here, it is the 4th bit), the VICITOP1 is read with the expected value as 0x00000060. The VICITOP2 is read expecting the vector address, 0x0000A010. The VICVectAddr is read expecting the address value as 0x0000A010. This indicates the beginning of the interrupt service routine. Now, the interrupt is to be cleared. For this, the bit 12 of the VICTrIntSource is made 0. Now, a write access is done on the VICVectAddr to mark the end of the interrupt service routine. The content of the CSR is updated to 0x0000.

Now, the function `NonVectIRQServ()` is called if the content of the CSR is zero. This function services and clears all the Non-Vectored IRQ interrupts. It checks the `VICIRQStatus` first. Since, the `VICIRQStatus` is zero in this case, this function is not executed.

Now, the Status registers are read again expecting 0x00000000, 0x00000000 and 0x00000000 from the `VICIRQStatus`, `VICFIQStatus` and `VICRawIntr` registers, respectively.

Non-Vectored IRQ

In this illustration the value 0x10000000 is written into the `VICTrIntSource`. This is done by invoking the function `ApplyIntr()` with the argument as 0. This indicates that source 28 is raising an interrupt. Since, bit 28 of the `VICIntSelect` is 0, it will be a IRQ interrupt. Since none of the `VICVectCntl` registers have the content 0x0000003C (interrupt source = 28), it indicates that the interrupt source 28 will raise a non-vectored interrupt. Hence, the vector address should be the content of the register `VICDefVectAddr` i.e., 0x12345678.

To test the VIC, the following have to be performed:

- Check the Status registers for expected values.
- Check that the `nVICIRQ` pin had gone low.
- Read the `VICVectAddr` expecting the required vector address. This marks the ISR entry.
- Clear the bit raising the interrupt in the `VICTrIntSource`.
- Perform a write access to the `VICVectAddr`. This marks the end of the interrupt service routine.
- Again check the Status registers.

To perform the above tasks, the function `AllIntService()` is invoked. This function is used to service and clear all the interrupts in the order of their priorities. When the argument of this function is 0, it indicates that the interrupts are being raised by the `VICTrIntSource` register of the Trickbox. When the interrupts are raised via the `VICSoftInt` register, the argument is 1.

Now the `ReadStatus()` function is called. The `VICIRQStatus` is read expecting a pattern of 0x10000000. The `VICFIQStatus` is read expecting a pattern of 0x00000000. The pattern 0x10000000 is expected from the `VICRawIntr` register. To read these three registers, a burst access is done.

Next, the function `CurServReg()` is invoked. The details of the interrupt sources raising vector interrupts are stored in a 16-bit variable called CSR. The value stored in the CSR should be 0x0000.

The `VICFIQStatus`, is checked for any FIQ interrupts.

Now, the `VICIRQStatus` is checked. If there is a non-zero bit, it indicates that there is an active IRQ interrupt. Now the value of CSR decides whether the IRQ is vectored or non-vectored. Since, the CSR is zero (indicating the absence of any active vectored IRQ), the function `NonVectIRQServ()` is accessed with the argument zero (indicating that it is from the `VICTrIntSource`). This function clears all the non vectored IRQ interrupts. The `VICIRQStatus` is read from the 0th bit. As soon as any non-zero bit is detected, the `VICITOP` registers are read. Then the `VICVectAddr` is read. Now, the bit which is raising this interrupt is cleared in the `VICTrIntSource`. Then, a write access is conducted on the `VICVectAddr` to indicate the end of the service routine.

In the present illustration, the status registers are read. Expected data from the `VICIRQStatus`, `VICFIQStatus` and `VICRawIntr` are 0x10000000, 0x00000000 and 0x10000000 respectively. The bits of the `VICIRQStatus` are read from the 0th position. As soon as there is a 1 (here, it is the 28th bit), the `VICITOP1` is read with the expected value as 0x00000060. The `VICITOP2` is read expecting the default vector address, 0x12345678. The `VICVectAddr` is read expecting the default address value of 0x12345678. Now, the interrupt is to be cleared. For this bit 28 of the `VICTrIntSource` is made 0. Then, the value 0x00000000 is written in the `VICVectAddr` register.

Now, the Status registers are read again expecting 0x00000000, 0x00000000 and 0x00000000 from the `VICIRQStatus`, `VICFIQStatus` and `VICRawIntr` registers, respectively.

If in all the above cases, the VIC performs as desired, the test passes.

In addition to the above basic test sequences, the following variants are also tested for the 3 categories:

- External interrupt is raised
- Tests are performed with the interrupt enable bit cleared in the VICVectCntl0-15 registers.
- Tests are performed with the interrupt disabled by clearing various combinations of bits in the VICIntEnable register.

Two Interrupt Test

This test is invoked by the TwoFourIntTests() function call. This test can be divided into 6 categories:

- Raising of two FIQs.
- Raising of two Vectored IRQs.
- Raising of two Non-Vectored IRQs.
- Raising of one FIQ and one Vectored IRQ.
- Raising of one FIQ and one Non-Vectored IRQ.
- Raising of one Vectored IRQ and one Non-Vectored IRQ.

In all these categories, the VIC is tested with the external interrupt also. In some cases certain interrupts are disabled by resetting the bit in the VICIntEnable register. In all, 512 tests are performed for the Two-Interrupt case. One category is illustrated with an example. In this example, the external interrupt is also active.

Program the following initialising values in the following registers.

VICIntEnable	0xFFFFFFFF	All the interrupts are enabled
VICSoftInt	0x00000000	None of the interrupts can occur through the VICSoftInt register
VICIntSelect	0x12345678	Decides which of the sources will raise an FIQ
VICTrIntIn	0x00000000	Indicates both the external interrupts
VICTrVectAddrIn	0x33333333	External Vector Address
VICProtection	0x00000000	VIC is in the 'User' mode
VICVectAddr0	0x0000A000	Address for Vector bank 0 is 0x0000A000
VICVectAddr1	0x0000A004	Address for Vector bank 1 is 0x0000A004
...	...	...
VICVectAddr15	0x0000A03C	Address for Vector bank 15 is 0x0000A0C4
VICVectCntl0	0x0000003D	Interrupt source 29 will raise a vectored interrupt request
VICVectCntl1	0x0000003E	Interrupt source 30 will raise a vectored interrupt request
VICVectCntl2	0x0000003F	Interrupt source 31 will raise a vectored interrupt request
VICVectCntl3	0x00000020	Interrupt source 0 will raise a vectored interrupt request
VICVectCntl4	0x00000021	Interrupt source 1 will raise a vectored interrupt request
...	...	...
VICVectCntl15	0x00000037	Interrupt source 13 will raise a vectored interrupt request

Two Vectored IRQs and both the external IRQs

In this illustration, the value 0xC0000000 is written into the VICTrIntSource register. This is done by invoking the function ApplyIntr() with the argument as 0. This indicates that sources 30 and 31 are raising interrupts. Since, the bits 30 and 31 of the VICIntSelect are 0, they will raise an IRQ interrupt. Now, the contents of VICVectCntl1 and VICVectCntl2 are 0x0000003E and 0x0000003F. The last six bits are 0b111110 and 0b111111. This indicates that the interrupt sources 30 and 31 will raise a vectored interrupt via the 1st and 2nd vector banks. Hence, the vector address should be the content of the registers VICVectAddr1 and VICVectAddr2 i.e., 0x0000A004 and 0x0000A008. Here, the priority of the interrupt from source 30 is greater than that from source 31.

To test the VIC the following have to be performed:

- Check the Status registers for expected values.
- Check that the nVICIRQ and nVICFIQ pins had gone low.
- Clear the bit raising the external FIQ. This is the 0th bit of VICTrIntIn.
- Again check the Status registers for expected values.
- Check that the nVICIRQ pin had gone low.
- Read the VICVectAddr register expecting the content of the VICVectAddr1. This marks the beginning of the interrupt service routine of the first interrupt.
- Clear the bit raising the first interrupt (bit 30) in the VICTrIntSource.
- Perform a write access to the VICVectAddr. This marks the end of the interrupt service routine of the first interrupt.
- Again check the Status registers for expected values.
- Check the nVICIRQ pin for a low.
- Read the VICVectAddr expecting the content of the VICVectAddr2. This marks the beginning of interrupt service routine of the second interrupt.
- Clear the bit raising the second interrupt (bit 31) in the VICTrIntSource.
- Perform a write access to the VICVectAddr. This marks the end of the interrupt service routine of the second interrupt.
- Again check the Status registers. They should contain zeroes.
- Check the nVICIRQ for a low.
- Read the VICVectAddr expecting the content of the VICTrVectAddrIn.
- Clear the bit raising the external IRQ. This is the 1st bit of VICTrIntIn.

To perform the above tasks, the function AllIntService() is invoked. This function is used to service and clear all the interrupts in the order of their priorities. When the argument of this function is 0, it indicates that the interrupts are being raised by the VICTrIntSource register of the Trickbox. When the interrupts are raised via the VICSoftInt register, the argument is 1.

Now the ReadStatus() function is called. Here the VICIRQStatus is read expecting a pattern of 0xC0000000. The VICFIQStatus is read expecting a pattern of 0x00000000. The pattern 0xC0000000 is expected from the VICRawIntr register. To read these three registers, a burst access is done.

Next, the function CurServReg() is invoked. The details of the interrupt sources raising vectored interrupts are stored in a 16-bit variable called CSR. Here, the interrupts are raised via the 1st and 2nd vector bank. Hence, the 1st and 2nd bits of the CSR should be set. Thus, the value stored in the CSR should be 0x0006.

The VICFIQStatus register is checked for any FIQ interrupts. The 0th bit of the VICTrIntIn is checked for any external FIQ. Now, the FIQServ() function is called. In this function, the Status registers are read. The expected values are

0xC0000000, 0x00000000 and 0xC0000000 from the VICIRQStatus, VICFIQStatus and VICRawIntr registers. The 0th bit in the VICTrIntIn is set to indicate that the external FIQ is cleared.

Now, the VICIRQStatus is checked. If there is a non-zero bit, it indicates that there is an active IRQ interrupt. Now the value of CSR decides whether the IRQ is vectored or non-vectored. Since, the CSR is non-zero (indicating the presence of active vectored IRQs), the function VectIRQServ() is accessed with the argument zero (indicating that it is from the VICTrIntSource). This function clears all the IRQ interrupts in the order of their priorities. The priority of the interrupts goes on decreasing when we move from 0 to 15th bit in the CSR. The CSR is read from the 0th bit. As soon as any non-zero bit is read, the interrupt service routine is started. The routine consists of reading the VICITOP registers, the address register and then clearing the interrupt and writing to the address register. At the same time the content of the CSR is also changed.

In the present illustration, the status registers are read and the expected data from the VICIRQStatus, VICFIQStatus and VICRawIntr are 0xC0000000, 0x00000000 and 0xC0000000 respectively. The bits of the CSR are read from the 0th position. As soon as there is a 1 (here, it is the 1st bit), the VICITOP1 is read with the expected value as 0x00000060. The VICITOP2 is read expecting the vector address, 0x0000A004. The VICVectAddr is read expecting the address value as 0x0000A004. This indicates the beginning of the interrupt service routine. Now, the interrupt is to be cleared. For this bit 30 of the VICTrIntSource is made 0. Now, a write access is done to the VICVectAddr register to mark the end of the interrupt service routine. The content of the CSR is updated to 0x0004.

Now, the status registers are read again. Expected data from the VICIRQStatus, VICFIQStatus and VICRawIntr are 0x80000000, 0x00000000 and 0x80000000 respectively. The next bits of the CSR are read. As soon as there is a 1 (here, it is the 2nd bit), the VICITOP1 is read with the expected value as 0x00000060. The VICITOP2 is read expecting the vector address, 0x0000A008. The VICVectAddr is read expecting the address value as 0x0000A008. This indicates the beginning of the interrupt service routine. Now, the interrupt is to be cleared. For this, the bit 31 of the VICTrIntSource is made 0. Now, a write access is done to the VICVectAddr register to mark the end of the interrupt service routine. The content of the CSR is cleared.

Now, the status registers are read expecting 0x00000000.

Now, the function NonVectIRQServ() is called if the content of the CSR is zero. This function services and clears all the Non-Vectored IRQ interrupts. It checks the VICIRQStatus, first. Since, the VICIRQStatus is zero in this case, this function is not executed.

The VICTrIntIn is checked for any external IRQ. If the 1st bit is 0, the external interrupt has to be cleared. For this, the 1st bit of the VICTrIntIn is cleared.

Now, the Status registers are read again expecting 0x00000000, 0x00000000 and 0x00000000 from the VICIRQStatus, VICFIQStatus and VICRawIntr registers, respectively.

The other categories of the Two interrupts case are tested in a similar fashion.

Four Interrupt Test

This test is invoked by the TwoFourIntTests() function call. This test can be divided into 15 categories:

- Raising of four FIQs.
- Raising of four Vectored IRQs.
- Raising of four Non-Vectored IRQs.
- Raising of three FIQs and one Vectored IRQ.
- Raising of three FIQs and one Non-Vectored IRQ.
- Raising of two FIQs and two Vectored IRQs.
- Raising of two FIQs, one Vectored IRQ and one Non-Vectored IRQ.
- Raising of two FIQs and two Non-Vectored IRQs.
- Raising of one FIQ and three Vectored IRQs.

- Raising of one FIQ, two Vectored IRQs and one Non-Vectored IRQ.
- Raising of one FIQ, one Vectored IRQ and two Non-Vectored IRQs.
- Raising of one FIQ and three Non-Vectored IRQs.
- Raising of three Vectored IRQs and one Non-Vectored IRQ.
- Raising of two Vectored IRQs and two Non-Vectored IRQs.
- Raising of one Vectored IRQ and three Non-Vectored IRQs.

In all these categories, the VIC is tested with the external interrupt also. In some cases, certain interrupts are disabled by resetting the bit in the VICIntEnable register. In all, 256 tests are performed for the Four-Interrupt case. One category is illustrated with an example.

Program the following initialising values in the following registers.

VICIntEnable	0xFFFFFFFF	All the interrupts are enabled
VICSoftInt	0x00000000	None of the interrupts can occur through the VICSoftInt register
VICIntSelect	0x55555555	Decides which of the sources will raise a FIQ
VICTrIntIn	0x00000003	Indicates both the external interrupts
VICTrVectAddrIn	0x00000000	External Vector Address
VICProtection	0x00000000	VIC is in the 'User' mode
VICVectAddr0	0x0000A000	Address for Vector bank 0 is 0x0000A000
VICVectAddr1	0x0000A004	Address for Vector bank 1 is 0x0000A004
...	...	...
VICVectAddr15	0x0000A0C4	Address for Vector bank 15 is 0x0000A0C4
VICVectCntl0	0x0000002A	Interrupt source 10 will raise a vector interrupt request
VICVectCntl1	0x0000002B	Interrupt source 11 will raise a vector interrupt request
...	...	...
VICVectCntl15	0x00000039	Interrupt source 25 will raise a vector interrupt request

Two FIQ, One Vectored IRQs and One Non-Vectored IRQ

In this illustration, the value 0x00000F00 is written into the VICTrIntSource. This is done by invoking the function ApplyIntr() with the argument as 0. This indicates that sources 8, 9, 10 and 11 are raising interrupts. Since the bits 8 and 10 of the VICIntSelect are 0, they will be raising IRQ interrupts. Now, the contents of VICVectCntl0 is 0x0000002A. The last six bits are 0b101010. This indicates that the interrupt source 10 will raise a vectored interrupt via the 0th vector bank. Hence, the vector address should be the content of the registers VICVectAddr0 i.e., 0x0000A000. Thus, source 8 is raising a Non-Vectored IRQ, source 10 is raising a Vectored IRQ and sources 9 and 11 are raising FIQs.

To test the VIC, the following have to be performed:

- Check the Status registers for expected values.
- Check the nVICIRQ and nVICFIQ pins, whether they had gone low.
- Clear the bit raising the external FIQ. This is the 9th bit of VICTrIntIn.
- Again check the Status registers, whether they contain the required values.

- Check the nVICIRQ and nVICFIQ pins, whether they are low.
- Clear the bit raising the external FIQ. This is the 11th bit of VICTrIntIn.
- Check the nVICIRQ pin, whether it is low.
- Read the VICVectAddr expecting the content of the VICVectAddr0. This marks the beginning of the interrupt service routine of the Vectored interrupt.
- Clear the bit raising the Vectored IRQ (bit 10) in the VICTrIntSource.
- Perform a write access to the VICVectAddr. This marks the end of the interrupt service routine of the Vectored interrupt.
- Again check the Status registers, whether they contain the required values.
- Check the nVICIRQ pin, whether it is low.
- Read the VICVectAddr expecting the content of the VICDefVectAddr. This marks the beginning of the interrupt service routine of the Non-Vectored interrupt.
- Clear the bit raising the Non-Vectored IRQ (bit 8) in the VICTrIntSource.
- Perform a write access to the VICVectAddr. This marks the end of the interrupt service routine of the Non-Vectored interrupt.
- Again check the Status registers, whether they are cleared.

To perform the above tasks, the function `AllIntService()` is invoked. This function is used to service and clear all the interrupts in the order of their priorities. When the argument of this function is 0, it indicates that the interrupts are being raised by the VICTrIntSource register of the Trickbox. When the interrupts are raised via the VICSoftInt register, the argument is 1.

Now the `ReadStatus()` function is called. The VICIRQStatus is read expecting a pattern of 0x00000500. The VICFIQStatus is read expecting a pattern of 0x00000A00. The pattern 0x00000F00 is expected from the VICRawIntr register. To read these three registers, a burst access is done.

Next, the function `CurServReg()` is invoked. The details of the interrupt sources raising vector interrupts are stored in a 16-bit variable called CSR. Here, the interrupt is raised via the 0th vector bank. Hence, the 0th bit of the CSR should be set. Thus, the value stored in the CSR should be 0x0001.

Now, the VICFIQStatus is checked. If there is a non-zero bit, it indicates that there is an active FIQ interrupt. Hence, the function `FIQServ()` is accessed with the argument zero (indicating that it is from the VICTrIntSource). This function clears all the FIQ interrupts. By comparing with the VICFIQStatus register, it is possible to determine which bits of the VICTrIntSource are currently requesting a FIQ. Now, beginning from the 0th position, all the FIQ interrupt sources are cleared by writing 0s in the corresponding positions in the VICTrIntSource register of the Trickbox.

In the present illustration, the status registers are read and the expected data from the VICIRQStatus, VICFIQStatus and VICRawIntr are 0x00000500, 0x00000A00 and 0x00000F00 respectively. The bits of the VICFIQStatus are read from the 0th position. As soon as there is a 1 (here, it is the 9th bit), the VICITOP1 is read with the expected value as 0x00000040. Now, the 9th bit of the VICTrIntSource is cleared. Again the reading of the VICFIQStatus register is continued. Now, at bit 11 again a 1 is present. The VICITOP1 is read with the expected value as 0x00000040. Now, the 11th bit of the VICTrIntSource is cleared.

Now, the VICIRQStatus is checked. If there is a non-zero bit, it indicates that there is an active IRQ interrupt. Now the value of CSR decides whether the IRQ is vectored or non-vectored. Since, the CSR is non-zero (indicating the presence of the active vectored IRQs), the function `VectIRQServ()` is accessed with the argument zero (indicating that it is from the VICTrIntSource). This function clears all the IRQ interrupts in the order of their priorities. The priority of the interrupts goes on decreasing when we move from 0 to 15th bit in the CSR. The CSR is read from the 0th bit. As soon as any non-zero bit is read, the interrupt service routine is started. The routine consists of the reading the VICITOP registers, the address register and then clearing the interrupt and writing to the address register. At the same time the content of the CSR is also changed.

In the present illustration, the status registers are read and the expected data from the VICIRQStatus, VICFIQStatus and VICRawIntr are 0x00000500, 0x00000000 and 0x00000500 respectively. The bits of the CSR are read from the 0th position. As soon as there is a 1 (here, it is the 0th bit), the VICITOP1 is read with the expected value as 0x00000060. The VICITOP2 is read expecting the vector address, 0x0000A000. The VICVectAddr is read expecting the address value as 0x0000A000. This indicates the beginning of the interrupt service routine. Now, the interrupt is to be cleared. For this bit 10 of the VICTrIntSource is made 0. Now, a write access is done to the VICVectAddr to mark the end of the interrupt service routine. The content of the CSR is updated to 0x0000.

Now, the VICIRQStatus is checked. If there is a non-zero bit, it indicates that there is an active IRQ interrupt. Since the CSR is zero (indicating the absence of any active vectored IRQ), the function NonVectIRQServ() is accessed with the argument zero (indicating that it is from the VICTrIntSource). This function clears all the non vectored IRQ interrupts. The VICIRQStatus is read from the 0th bit. As soon as any non-zero bit is read, the VICITOP registers are read. Then the VICVectAddr is read. Now, the bit which is raising this interrupt is cleared in the VICTrIntSource. After that, a write access is conducted on the VICVectAddr register.

In the present illustration, the status registers are read and the expected data from the VICIRQStatus, VICFIQStatus and VICRawIntr are 0x00000800, 0x00000000 and 0x00000100 respectively. The bits of the VICIRQStatus are read from the 0th position. As soon as there is a 1 (here, it is the 8th bit), the VICITOP1 is read with the expected value as 0x00000060. The VICITOP2 is read expecting the vector address, 0x12345678. The VICVectAddr is read expecting the address value as 0x12345678. Now, the interrupt is to be cleared. For this bit 8 of the VICTrIntSource is made 0. Now, a write access is done to the VICVectAddr to mark the end of the interrupt service routine.

Now, the Status registers are read again expecting 0x00000000, 0x00000000 and 0x00000000 from the VICIRQStatus, VICFIQStatus and VICRawIntr registers, respectively.

The other categories of the Four interrupts case are tested in a similar fashion.

Eight Interrupt Test

This test is invoked by the EightAllIntTests() function call. This test can be divided into 45 categories.

In all the categories, the VIC is tested with the external interrupt also. In some cases, certain interrupts are disabled by resetting the bit in the VICIntEnable register. In all, 128 tests are performed for the Eight-Interrupt case. One category is illustrated with an example.

Program the following initialising values in the following registers.

VICIntEnable	0xFFFFFFFF	All the interrupts are enabled
VICSoftInt	0x00000000	None of the interrupts can occur through the VICSoftInt register
VICIntSelect	0x0000FFFF	Decides which of the sources will raise an FIQ
VICTrIntIn	0x00000003	Indicates both the external interrupts
VICTrVectAddrIn	0x00000000	External Vector Address
VICProtection	0x00000000	VIC is in the 'User' mode
VICVectAddr0	0x0000A000	Address for Vector bank 0 is 0x0000A000
VICVectAddr1	0x0000A004	Address for Vector bank 1 is 0x0000A004
...	...	...
VICVectAddr15	0x0000A0C4	Address for Vector bank 15 is 0x0000A0C4
VICVectCntl0	0x0000002E	Interrupt source 14 will raise a vector interrupt request
VICVectCntl1	0x0000002F	Interrupt source 15 will raise a vector interrupt request
...	...	...

VICVectCntl15 0x0000003D Interrupt source 29 will raise a vector interrupt request

Six Vectored IRQs and Two Non-Vectored IRQs

In this illustration, the value 0xFF000000 is written into the VICTrIntSource. This is done by invoking the function ApplyIntr() with the argument as 0. This indicates that sources 24, 25, 26, 27, 28, 29, 30 and 31 are raising interrupts. Since, all of these bits of the VICIntSelect are 0, they will be raising an IRQ interrupt. Now, the last 5 bits of VICVectCntl10, VICVectCntl11, VICVectCntl12, VICVectCntl13, VICVectCntl14 and VICVectCntl15 when converted to decimal gives 24, 25, 26, 27, 28 and 29 respectively. This indicates that the interrupt sources 24, 25, 26, 27, 28 and 29 are raising Vectored IRQs and the sources 30 and 31 are raising Non-Vectored IRQs.

To test the VIC the following have to be performed:

- Check the Status registers, whether they contain the required values.
- Check the nVICIRQ pin, whether it is low.
- Read the VICVectAddr expecting the content of the VICVectAddr10. This marks the beginning of the interrupt service routine of the first interrupt.
- Clear the bit raising the Vectored IRQ (bit 24) in the VICTrIntSource.
- Perform a write access to the VICVectAddr. This marks the end of the interrupt service routine of the first interrupt.
- Again check the Status registers, whether they contain the required values.
- Check the nVICIRQ pin, whether it is low.
- Read the VICVectAddr expecting the content of the VICVectAddr11. This marks the beginning of the interrupt service routine of the second interrupt.
- Clear the bit raising the Vectored IRQ (bit 25) in the VICTrIntSource.
- Perform a write access to the VICVectAddr. This marks the end of the interrupt service routine of the second interrupt.
- Again check the Status registers, whether they contain the required values.
- Read the VICVectAddr expecting the content of the VICVectAddr12. This marks the beginning of the interrupt service routine of the third interrupt.
- Clear the bit raising the Vectored IRQ (bit 26) in the VICTrIntSource.
- Perform a write access to the VICVectAddr. This marks the end of the interrupt service routine of the third interrupt.
- Again check the Status registers, whether they contain the required values.
- Read the VICVectAddr expecting the content of the VICVectAddr13. This marks the beginning of the interrupt service routine of the fourth interrupt.
- Clear the bit raising the Vectored IRQ (bit 27) in the VICTrIntSource.
- Perform a write access to the VICVectAddr. This marks the end of the interrupt service routine of the fourth interrupt.
- Again check the Status registers, whether they contain the required values.
- Read the VICVectAddr expecting the content of the VICVectAddr14. This marks the beginning of the interrupt service routine of the fifth interrupt.
- Clear the bit raising the Vectored IRQ (bit 28) in the VICTrIntSource.

- Perform a write access to the VICVectAddr. This marks the end of the interrupt service routine of the fifth interrupt.
- Again check the Status registers, whether they contain the required values.
- Read the VICVectAddr expecting the content of the VICVectAddr15. This marks the beginning of the interrupt service routine of the sixth interrupt.
- Clear the bit raising the Vectored IRQ (bit 29) in the VICTrIntSource.
- Perform a write access to the VICVectAddr. This marks the end of the interrupt service routine of the sixth interrupt.
- Again check the Status registers, whether they contain the required values.
- Read the VICVectAddr expecting the content of the VICDefVectAddr. This marks the beginning of the interrupt service routine of the first Non-Vectored interrupt.
- Clear the bit raising the Non-Vectored IRQ (bit 30) in the VICTrIntSource.
- Perform a write access to the VICVectAddr. This marks the end of the interrupt service routine of the first Non-Vectored interrupt.
- Again check the Status registers, whether they contain the required values.
- Read the VICVectAddr expecting the content of the VICDefVectAddr. This marks the beginning of the interrupt service routine of the second Non-Vectored interrupt.
- Clear the bit raising the Non-Vectored IRQ (bit 31) in the VICTrIntSource.
- Perform a write access to the VICVectAddr. This marks the end of the interrupt service routine of the second Non-Vectored interrupt.
- Again check the Status registers, whether they contain the required values.

To perform the above tasks, the function `AllIntService()` is invoked. This function is used to service and clear all the interrupts in the order of their priorities. When the argument of this function is 0, it indicates that the interrupts are being raised by the `VICTrIntSource` register of the Trickbox. When the interrupts are raised via the `VICSoftInt` register, the argument is 1.

Now the `ReadStatus()` function is called. Here the `VICIRQStatus` is read expecting a pattern of `0xFF000000`. The `VICFIQStatus` is read expecting a pattern of `0x00000000`. The pattern `0xFF000000` is expected from the `VICRawIntr` register. To read these three registers, a burst access is done.

Next, the function `CurServReg()` is invoked. The details of the interrupt sources raising vectored interrupts are stored in a 16-bit variable called CSR. Here, the interrupts are raised via the 10th, 11th, 12th, 13th, 14th and 15th vector banks. Thus, the value stored in the CSR should be `0xFC00`.

The `VICFIQStatus`, is checked for any FIQ interrupts.

Now, the `VICIRQStatus` is checked. If there is a non-zero bit, it indicates that there is an active IRQ interrupt. Now the value of CSR decides whether the IRQ is vectored or non-vectored. Since, the CSR is non-zero (indicating the presence the active vectored IRQs), the function `VectIRQServ()` is accessed with the argument zero (indicating that it is from the `VICTrIntSource`). This function clears all the IRQ interrupts in the order of their priorities. The priority of the interrupts goes on decreasing when we move from 0 to 15th bit in the CSR. The CSR is read from the 0th bit. As soon as any non-zero bit occur, the interrupt service routine is started. The routine consists of reading the `VICITOP` registers, the address register and then clearing the interrupt and writing to the address register. At the same time the content of the CSR is also changed.

In the present illustration, the status registers are read and the expected data from the `VICIRQStatus`, `VICFIQStatus` and `VICRawIntr` are `0xFF000000`, `0x00000000` and `0xFF000000` respectively. The bits of the CSR are read from the 0th position. As soon as there is a 1 (here, it is the 10th bit), the `VICITOP1` is read with the expected value as `0x00000060`. The `VICITOP2` is read expecting the vector address, `0x0000A028` (the content of the `VICVectAddr10`). The `VICVectAddr` is read expecting the address value as `0x0000A028`. This indicates the beginning of the interrupt service routine. Now, the interrupt is to be cleared. For this, the 24 of the `VICTrIntSource` is made 0. Now, a write

access is done on the VICVectAddr to mark the end of the interrupt service routine. The content of the CSR is updated to 0xF800.

Now, the bits of the CSR are read further from the 11th position. As soon as there is a 1 (here, it is the 11th bit), the VICITOP1 is read with the expected value as 0x00000060. The VICITOP2 is read expecting the address value as the 0x0000A02C (the content of the VICVectAddr11). The VICVectAddr is read expecting the address value as 0x0000A02C. This indicates the beginning of the interrupt service routine. Now, the interrupt is to be cleared. For this bit 25 of the VICTrIntSource is made 0. Now, a write access is done on the VICVectAddr to mark the end of the interrupt service routine. The content of the CSR is updated to 0xF000.

Now, the bits of the CSR are read further from the 12th position. As soon as there is a 1 (here, it is the 12th bit), the VICITOP1 is read with the expected value as 0x00000060. The VICITOP2 is read expecting the address value as the 0x0000A030 (the content of the VICVectAddr12). The VICVectAddr is read expecting the address value as 0x0000A030. This indicates the beginning of the interrupt service routine. Now, the interrupt is to be cleared. For this bit 26 of the VICTrIntSource is made 0. Now, a write access is done on the VICVectAddr to mark the end of the interrupt service routine. The content of the CSR is updated to 0xE000.

Now, the bits of the CSR are read further from the 13th position. As soon as there is a 1 (here, it is the 13th bit), the VICITOP1 is read with the expected value as 0x00000060. The VICITOP2 is read expecting the address value as the 0x0000A034 (the content of the VICVectAddr13). The VICVectAddr is read expecting the address value as 0x0000A034. This indicates the beginning of the interrupt service routine. Now, the interrupt is to be cleared. For this bit 27 of the VICTrIntSource is made 0. Now, a write access is done on the VICVectAddr to mark the end of the interrupt service routine. The content of the CSR is updated to 0xC000.

Now, the bits of the CSR are read further from the 14th position. As soon as there is a 1 (here, it is the 14th bit), the VICITOP1 is read with the expected value as 0x00000060. The VICITOP2 is read expecting the address value as the 0x0000A038 (the content of the VICVectAddr14). The VICVectAddr is read expecting the address value as 0x0000A038. This indicates the beginning of the interrupt service routine. Now, the interrupt is to be cleared. For this bit 28 of the VICTrIntSource is made 0. Now, a write access is done on the VICVectAddr to mark the end of the interrupt service routine. The content of the CSR is updated to 0xF000.

Now, the bits of the CSR are read further from the 15th position. As soon as there is a 1 (here, it is the 15th bit), the VICITOP1 is read with the expected value as 0x00000060. The VICITOP2 is read expecting the address value as the 0x0000A03C (the content of the VICVectAddr15). The VICVectAddr is read expecting the address value as 0x0000A03C. This indicates the beginning of the interrupt service routine. Now, the interrupt is to be cleared. For this bit 29 of the VICTrIntSource is made 0. Now, a write access is done on the VICVectAddr to mark the end of the interrupt service routine. The content of the CSR is updated to 0x0000.

Now, the VICIRQStatus is checked. If there is a non-zero bit, it indicates that there is an active IRQ interrupt. Since, the CSR is zero (indicating the absence of any active vectored IRQ), the function NonVectIRQServ() is accessed with the argument zero (indicating that it is from the VICTrIntSource). This function clears all the non vectored IRQ interrupts. The VICIRQStatus is read from the 0th bit. As soon as any non-zero bit is read, the VICITOP registers are read. Then the VICVectAddr is read. Now, the bit which is raising this interrupt is cleared in the VICTrIntSource.

In the present illustration, the status registers are read and the expected data from the VICIRQStatus, VICFIQStatus and VICRawIntr are 0xC0000000, 0x00000000 and 0xC0000000 respectively. The bits of the VICIRQStatus are read from the 0th position. As soon as, there is a 1 (here, it is the 30th bit), the VICITOP1 is read with the expected value as 0x00000060. The VICITOP2 is read expecting the vector address, 0x12345678. The VICVectAddr is read expecting the address value as 0x12345678. This indicates the beginning of the interrupt service routine. Now, the interrupt is to be cleared. For this the bit 30 of the VICTrIntSource is made 0. Now, a write access is done on the VICVectAddr to mark the end of the interrupt service routine.

Now the bits of the VICIRQStatus are read further from the 31st position. As soon as there is a 1 (here, it is the 31st), the VICITOP1 is read with the expected value as 0x00000060. The VICITOP2 is read expecting the address value as the 0x12345678 (the content of the VICDefVectAddr). This indicates the beginning of the interrupt service routine. Now, the interrupt is to be cleared. For this the bit 31 of the VICTrIntSource is made 0. Now, a write access is done on the VICVectAddr to mark the end of the interrupt service routine.

Now, the Status registers are read again expecting 0x00000000, 0x00000000 and 0x00000000 from the VICIRQStatus, VICFIQStatus and VICRawIntr registers, respectively.

The other categories of the Eight interrupts case are tested in a similar fashion.

Sixteen Interrupt Test

This test is invoked by the EightAllIntTests() function call. This test can be divided into 143 categories.

In all the categories, the VIC is tested with the external interrupt also. In some cases, certain interrupts are disabled by resetting the bit in the VICIntEnable register. In all, 64 tests are performed for the Eight-Interrupt case. One category is illustrated with an example.

Program the following initialising values in the following registers.

VICIntEnable	0xFFFFFFFF	All the interrupts are enabled
VICSoftInt	0x00000000	None of the interrupts can occur through the VICSoftInt register
VICIntSelect	0x0F0F0F0F	Decides which of the sources will raise an FIQ
VICTrIntIn	0x00000003	Indicates both the external interrupts
VICTrVectAddrIn	0x00000000	External Vector Address
VICProtection	0x00000000	VIC is in the 'User' mode
VICVectAddr0	0x0000A000	Address for Vector bank 0 is 0x0000A000
VICVectAddr1	0x0000A004	Address for Vector bank 1 is 0x0000A004
...	...	...
VICVectAddr15	0x0000A0C4	Address for Vector bank 15 is 0x0000A0C4
VICVectCntl0	0x00000026	Interrupt source 6 will raise a vector interrupt request
VICVectCntl1	0x00000027	Interrupt source 7 will raise a vector interrupt request
...	...	...
VICVectCntl15	0x00000035	Interrupt source 21 will raise a vector interrupt request

Eight FIQs, six Vectored IRQs and two Non-Vectored IRQs

In this illustration, the value 0x0000FFFF is written into the VICTrIntSource. This is done by invoking the function ApplyIntr() with the argument as 0. This indicates that sources 0 to 15 are raising interrupts. Since, the bits 0, 1, 2, 3, 8, 9, 10 and 11 of the VICIntSelect are 1, these sources will be raising FIQs. Rest of the interrupts will be raising IRQ interrupts. Now, the last 5 bits of VICVectCntl0, VICVectCntl1, VICVectCntl6, VICVectCntl7, VICVectCntl8 and VICVectCntl9 when converted to decimal give 6, 7, 12, 13, 14 and 15 respectively. This indicates that the interrupt sources 6, 7, 12, 13, 14 and 15 are raising Vectored IRQs and the sources 4 and 5 are raising Non-Vectored IRQs.

To test the VIC the following have to be performed:

- Check the Status registers, whether they contain the required values.
- Check the nVICIRQ and nVICFIQ pins, whether they are low.
- Clear the bit raising the external FIQ. This is the 0th bit of VICTrIntSource.
- Again check the Status registers, whether they contain the required values.
- Check the nVICIRQ and nVICFIQ pins, whether they are low.

- Clear the bit raising the external FIQ. This is the 1st bit of VICTrIntSource.
- Again check the Status registers, whether they contain the required values.
- Check the nVICIRQ and nVICFIQ pins, whether they are low.
- Clear the bit raising the external FIQ. This is the 2nd bit of VICTrIntSource.
- Again check the Status registers, whether they contain the required values.
- Check the nVICIRQ and nVICFIQ pins, whether they are low.
- Clear the bit raising the external FIQ. This is the 3rd bit of VICTrIntSource.
- Again check the Status registers, whether they contain the required values.
- Check the nVICIRQ and nVICFIQ pins, whether they are low.
- Clear the bit raising the external FIQ. This is the 8th bit of VICTrIntSource.
- Again check the Status registers, whether they contain the required values.
- Check the nVICIRQ and nVICFIQ pins, whether they are low.
- Clear the bit raising the external FIQ. This is the 9th bit of VICTrIntSource.
- Again check the Status registers, whether they contain the required values.
- Check the nVICIRQ and nVICFIQ pins, whether they are low.
- Clear the bit raising the external FIQ. This is the 10th bit of VICTrIntSource.
- Again check the Status registers, whether they contain the required values.
- Check the nVICIRQ and nVICFIQ pins, whether they are low.
- Clear the bit raising the external FIQ. This is the 11th bit of VICTrIntSource.
- Again check the Status registers, whether they contain the required values.
- Check the nVICIRQ pin, whether it is low.
- Read the VICVectAddr expecting the content of the VICVectAddr0. This marks the beginning of the interrupt service routine of the first interrupt.
- Clear the bit raising the Vectored IRQ (bit 6) in the VICTrIntSource.
- Perform a write access to the VICVectAddr. This marks the end of the interrupt service routine of the first interrupt.
- Again check the Status registers, whether they contain the required values.
- Read the VICVectAddr expecting the content of the VICVectAddr1. This marks the beginning of the interrupt service routine of the second interrupt.
- Clear the bit raising the Vectored IRQ (bit 7) in the VICTrIntSource.
- Perform a write access to the VICVectAddr. This marks the end of the interrupt service routine of the second interrupt.
- Again check the Status registers, whether they contain the required values.
- Read the VICVectAddr expecting the content of the VICVectAddr6. This marks the beginning of the interrupt service routine of the third interrupt.
- Clear the bit raising the Vectored IRQ (bit 12) in the VICTrIntSource.
- Perform a write access to the VICVectAddr. This marks the end of the interrupt service routine of the third interrupt.
- Again check the Status registers, whether they contain the required values.

- Read the VICVectAddr expecting the content of the VICVectAddr7. This marks the beginning of the interrupt service routine of the fourth interrupt.
- Clear the bit raising the Vectored IRQ (bit 13) in the VICTrIntSource.
- Perform a write access to the VICVectAddr. This marks the end of the interrupt service routine of the fourth interrupt.
- Again check the Status registers, whether they contain the required values.
- Read the VICVectAddr expecting the content of the VICVectAddr8. This marks the beginning of the interrupt service routine of the fifth interrupt.
- Clear the bit raising the Vectored IRQ (bit 14) in the VICTrIntSource.
- Perform a write access to the VICVectAddr. This marks the end of the interrupt service routine of the fifth interrupt.
- Again check the Status registers, whether they contain the required values.
- Read the VICVectAddr expecting the content of the VICVectAddr9. This marks the beginning of the interrupt service routine of the sixth interrupt.
- Clear the bit raising the Vectored IRQ (bit 15) in the VICTrIntSource.
- Perform a write access to the VICVectAddr. This marks the end of the interrupt service routine of the sixth interrupt.
- Again check the Status registers, whether they contain the required values.
- Read the VICVectAddr expecting the content of the VICDefVectAddr. This marks the beginning of the interrupt service routine of the first Non-Vectored interrupt.
- Clear the bit raising the Non-Vectored IRQ (bit 4) in the VICTrIntSource.
- Perform a write access to the VICVectAddr. This marks the end of the interrupt service routine of the first Non-Vectored interrupt.
- Again check the Status registers, whether they contain the required values.
- Read the VICVectAddr expecting the content of the VICDefVectAddr. This marks the beginning of the interrupt service routine of the second Non-Vectored interrupt.
- Clear the bit raising the Non-Vectored IRQ (bit 5) in the VICTrIntSource.
- Perform a write access to the VICVectAddr. This marks the end of the interrupt service routine of the second Non-Vectored interrupt.
- Again check the Status registers, whether they contain the required values.

To perform the above tasks, the function `AllIntService()` is invoked. This function is used to service and clear all the interrupts in the order of their priorities. When the argument of this function is 0, it indicates that the interrupts are being raised by the `VICTrIntSource` register of the `Trickbox`. When the interrupts are raised via the `VICSoftInt` register, the argument is 1.

Now the `ReadStatus()` function is called. Here the `VICIRQStatus` is read expecting a pattern of `0x0000F0F0`. The `VICFIQStatus` is read expecting a pattern of `0x00000F0F`. The pattern `0x0000FFFF` is expected from the `VICRawIntr` register. To read these three registers, a burst access is done.

Next, the function `CurServReg()` is invoked. The details of the interrupt sources raising vectored interrupts are stored in a 16-bit variable called `CSR`. Here, the interrupts are raised via the 0th, 1st, 6th, 7th, 8th and 9th vector banks. Thus, the value stored in the `CSR` should be `0x03C3`.

Now, the `VICFIQStatus` is checked. If there is a non-zero bit, it indicates that there is an active FIQ interrupt. Hence, the function `FIQServ()` is accessed with the argument zero (indication that it is from the `VICTrIntSource`). This function clears all FIQ interrupts. By comparing with the `VICFIQStatus` register, it is possible to determine which bits

of the VICTrIntSource are currently requesting a FIQ. Now, beginning from the 0th position, all the FIQ interrupts sources are cleared by writing 0s in the corresponding positions in the VICTrIntSource register of the Trickbox.

In the present illustration, the status registers are read and the expected data from the VICIRQStatus, VICFIQStatus and VICRawIntr are 0x0000F0F0, 0x0000F0F and 0x0000FFFF respectively. The bits of the VICFIQStatus are read from the 0th position. As soon as, there is a 1 (here, it is the 0th bit), the VICITOP1 is read with the expected value as 0x00000040. Now, the 0th bit of the VICTrIntSource is cleared.

Now, the bits of the VICFIQStatus are further read from the 1st bit position. As soon as, there is a 1 (here, it is the 1st bit), the VICITOP1 is read with the expected value as 0x00000040. Now, the 1st bit of the VICTrIntSource is cleared.

Now, the bits of the VICFIQStatus are further read from the 2nd bit position. As soon as, there is a 1 (here, it is the 2nd bit), the VICITOP1 is read with the expected value as 0x00000040. Now, the 2nd bit of the VICTrIntSource is cleared.

Now, the bits of the VICFIQStatus are further read from the 3rd bit position. As soon as, there is a 1 (here, it is the 3rd bit), the VICITOP1 is read with the expected value as 0x00000040. Now, the 3rd bit of the VICTrIntSource is cleared.

Now, the bits of the VICFIQStatus are further read from the 8th bit position. As soon as, there is a 1 (here, it is the 8th bit), the VICITOP1 is read with the expected value as 0x00000040. Now, the 8th bit of the VICTrIntSource is cleared.

Now, the bits of the VICFIQStatus are further read from the 9th bit position. As soon as, there is a 1 (here, it is the 9th bit), the VICITOP1 is read with the expected value as 0x00000040. Now, the 9th bit of the VICTrIntSource is cleared.

Now, the bits of the VICFIQStatus are further read from the 10th bit position. As soon as, there is a 1 (here, it is the 10th bit), the VICITOP1 is read with the expected value as 0x00000040. Now, the 10th bit of the VICTrIntSource is cleared.

Now, the bits of the VICFIQStatus are further read from the 11th bit position. As soon as, there is a 1 (here, it is the 11th bit), the VICITOP1 is read with the expected value as 0x00000040. Now, the 11th bit of the VICTrIntSource is cleared.

Now, the bits of the VICFIQStatus are further read from the 12th position.

Now, the VICIRQStatus is checked. If there is a non-zero bit, it indicates that there is an active IRQ interrupt. Now the value of CSR decides whether the IRQ is vectored or non-vectored. Since, the CSR is non-zero (indicating the presence of active vectored IRQs), the function VectIRQServ() is accessed with the argument zero (indicating that it is from the VICTrIntSource). This function clears all the IRQ interrupts in the order of their priorities. The priority of the interrupts goes on decreasing when we move from 0 to 15th bit in the CSR. The CSR is read from the 0th bit. As soon as any non-zero bit is read, the interrupt service routine is started. The routine consists of the reading the VICITOP registers, the address register and then clearing the interrupt and writing to the address register. At the same time the content of the CSR is also changed.

In the present illustration, the status registers are read and the expected data from the VICIRQStatus, VICFIQStatus and VICRawIntr are 0x0000F0F0, 0x00000000 and 0x0000F0F0 respectively. The bits of the CSR are read from the 0th position. As soon as, there is a 1 (here, it is the 0th bit), the VICITOP1 is read with the expected value as 0x00000060. The VICITOP2 is read expecting the vector address, 0x0000A000 (the content of the VICVectAddr0). The VICVectAddr is read expecting the address value as 0x0000A000. This indicates the beginning of the interrupt service routine. Now, the interrupt is to be cleared. For this the bit 6 of the VICTrIntSource is made 0. Now, a write access is done on the VICVectAddr to mark the end of the interrupt service routine. The content of the CSR is updated to 0x03C2.

Now, the bits of the CSR are read further from the 1st position. As soon as there is a 1 (here, it is the 1st bit), the VICITOP1 is read with the expected value as 0x00000060. The VICITOP2 is read expecting the address value as the 0x0000A004 (the content of the VICVectAddr1). The VICVectAddr is read expecting the address value as 0x0000A004. This indicates the beginning of the interrupt service routine. Now, the interrupt is to be cleared. For this bit 7 of the VICTrIntSource is made 0. Now, a write access is done on the VICVectAddr to mark the end of the interrupt service routine. The content of the CSR is updated to 0x03C0.

Now, the bits of the CSR are read further from the 2nd position. As soon as there is a 1 (here, it is the 6th bit), the VICITOP1 is read with the expected value as 0x00000060. The VICITOP2 is read expecting the address value as the

0x0000A018 (the content of the VICVectAddr6). The VICVectAddr is read expecting the address value as 0x0000A018. This indicates the beginning of the interrupt service routine. Now, the interrupt is to be cleared. For this bit 12 of the VICTrIntSource is made 0. Now, a write access is done on the VICVectAddr to mark the end of the interrupt service routine. The content of the CSR is updated to 0x0380.

Now, the bits of the CSR are read further from the 7th position. As soon as there is a 1 (here, it is the 7th bit), the VICITOP1 is read with the expected value as 0x00000060. The VICITOP2 is read expecting the address value as the 0x0000A01C (the content of the VICVectAddr7). The VICVectAddr is read expecting the address value as 0x0000A01C. This indicates the beginning of the interrupt service routine. Now, the interrupt is to be cleared. For this bit 13 of the VICTrIntSource is made 0. Now, a write access is done on the VICVectAddr to mark the end of the interrupt service routine. The content of the CSR is updated to 0x0300.

Now, the bits of the CSR are read further from the 8th position. As soon as there is a 1 (here, it is the 8th bit), the VICITOP1 is read with the expected value as 0x00000060. The VICITOP2 is read expecting the address value as the 0x0000A020 (the content of the VICVectAddr8). The VICVectAddr is read expecting the address value as 0x0000A020. This indicates the beginning of the interrupt service routine. Now, the interrupt is to be cleared. For this bit 14 of the VICTrIntSource is made 0. Now, a write access is done on the VICVectAddr to mark the end of the interrupt service routine. The content of the CSR is updated to 0x0200.

Now, the bits of the CSR are read further from the 9th position. As soon as there is a 1 (here, it is the 9th bit), the VICITOP1 is read with the expected value as 0x00000060. The VICITOP2 is read expecting the address value as the 0x0000A024 (the content of the VICVectAddr9). The VICVectAddr is read expecting the address value as 0x0000A024. This indicates the beginning of the interrupt service routine. Now, the interrupt is to be cleared. For this bit 15 of the VICTrIntSource is made 0. Now, a write access is done on the VICVectAddr to mark the end of the interrupt service routine. The content of the CSR is updated to 0x0000.

Now, the bits of the CSR are further read from the 16th position.

Now, the VICIRQStatus is checked. If there is a non-zero bit, it indicates that there is an active IRQ interrupt. Since, the CSR is zero (indicating the absence of any active vector IRQ), the function NonVectIRQServ() is accessed with the argument zero (indicating that it is from the VICTrIntSource). This function clears all the non vectored IRQ interrupts. The VICIRQStatus is read from the 0th bit. As soon as any non-zero bit is read, the VICITOP registers are read. Then the VICVectAddr is read. Now, the bit which is raising this interrupt is cleared in the VICTrIntSource. To mark the end of the service routine, the VICVectAddr is written with any value.

In the present illustration, the status registers are read and the expected data from the VICIRQStatus, VICFIQStatus and VICRawIntr are 0x00000030, 0x00000000 and 0x00000030 respectively. The bits of the VICIRQStatus are read from the 0th position. As soon as, there is a 1 (here, it is the 4th bit), the VICITOP1 is read with the expected value as 0x00000060. The VICITOP2 is read expecting the vector address, 0x12345678. The VICVectAddr is read expecting the address value as 0x12345678. This indicates the beginning of the ISR. Now, the interrupt is to be cleared. For this bit 4 of the VICTrIntSource is made 0. Now, a write access is done on the VICVectAddr to mark the end of the interrupt service routine.

Now the bits of the VICIRQStatus are read further from the 5th position. As soon as there is a 1 (here, it is the 5th), the VICITOP1 is read with the expected value as 0x00000060. The VICITOP2 is read expecting the address value as the 0x12345678 (the content of the VICDefVectAddr). This indicates the beginning of the ISR. Now, the interrupt is to be cleared. For this bit 5 of the VICTrIntSource is made 0. Now, a write access is done on the VICVectAddr to mark the end of the interrupt service routine.

Now the bits of the VICIRQStatus are read further from the 6th position.

Now, the Status registers are read again expecting 0x00000000, 0x00000000 and 0x00000000 from the VICIRQStatus, VICFIQStatus and VICRawIntr registers, respectively.

The other categories of the Sixteen interrupts case are tested in a similar fashion.

Thirty-Two Interrupt Test

This test is performed by EightAllIntTests() function call. This test can be divided into 561 categories.

In all the categories, the VIC is tested with the external interrupt also. In some cases, certain interrupts are disabled by resetting the bit in the VICIntEnable register. In all, 32 tests are performed for Eight-Interrupt. One category is illustrated with an example.

Program the following initialising values in the following registers.

VICIntEnable	0xFFFFFFFF	All the interrupts are enabled
VICSoftInt	0x00000000	None of the interrupts can occur through the VICSoftInt register
VICIntSelect	0xFA5005AF	Decides which of the sources will raise an FIQ
VICTrIntIn	0x00000003	Indicates both the external interrupts
VICTrVectAddrIn	0x00000000	External Vector Address
VICProtection	0x00000000	VIC is in the 'User' mode
VICVectAddr0	0x0000A000	Address for Vector bank 0 is 0x0000A000
VICVectAddr1	0x0000A004	Address for Vector bank 1 is 0x0000A004
...	...	...
VICVectAddr15	0x0000A0C4	Address for Vector bank 15 is 0x0000A0C4
VICVectCntl0	0x00000029	Interrupt source 9 will raise a vector interrupt request
VICVectCntl1	0x0000002A	Interrupt source 10 will raise a vector interrupt request
...	...	...
VICVectCntl15	0x00000038	Interrupt source 24 will raise a vector interrupt request

Sixteen FIQs, eleven Vectored IRQs and five Non-Vectored IRQs

In this illustration, the value 0xFFFFFFFF is written into the VICTrIntSource. This is done by invoking the function ApplyIntr() with the argument as 0. This indicates that sources 0 to 31 are raising interrupts. Since, the bits 0, 1, 2, 3, 4, 6, 9, 11, 21, 23, 24, 26, 28, 29, 30 and 31 of the VICIntSelect are 1, these sources will be raising FIQs. Rest of the interrupts will be raising IRQ interrupts. Now, the last 5 bits of VICVectCntl1, VICVectCntl3, VICVectCntl4, VICVectCntl5, VICVectCntl6, VICVectCntl7, VICVectCntl8, VICVectCntl9, VICVectCntl10, VICVectCntl11 and VICVectCntl13 when converted to decimal gives 10, 12, 13, 14, 15, 16, 17, 18, 19, 20 and 22 respectively. This indicates that the interrupt sources 10, 12, 13, 14, 15, 16, 17, 18, 19, 20 and 22 are raising Vectored IRQs and the sources 5, 7, 8, 25 and 27 are raising Non-Vectored IRQs.

To test the VIC, the following have to be performed:

- Check the Status registers, whether they contain the required values.
- Check the nVICIRQ and nVICFIQ pins, whether they are low.
- Clear the bit raising the external FIQ. This is the 0th bit of VICTrIntSource.
- Again check the Status registers, whether they contain the required values.
- Check the nVICIRQ and nVICFIQ pins, whether they are low.
- Clear the bit raising the external FIQ. This is the 1st bit of VICTrIntSource.
- Again check the Status registers, whether they contain the required values.
- Check the nVICIRQ and nVICFIQ pins, whether they are low.

- Clear the bit raising the external FIQ. This is the 2nd bit of VICTrIntSource.
- Again check the Status registers, whether they contain the required values.
- Check the nVICIRQ and nVICFIQ pins, whether they are low.
- Clear the bit raising the external FIQ. This is the 3rd bit of VICTrIntSource.
- Again check the Status registers, whether they contain the required values.
- Check the nVICIRQ and nVICFIQ pins, whether they are low.
- Clear the bit raising the external FIQ. This is the 4th bit of VICTrIntSource.
- Again check the Status registers, whether they contain the required values.
- Check the nVICIRQ and nVICFIQ pins, whether they are low.
- Clear the bit raising the external FIQ. This is the 6th bit of VICTrIntSource.
- Again check the Status registers, whether they contain the required values.
- Check the nVICIRQ and nVICFIQ pins, whether they are low.
- Clear the bit raising the external FIQ. This is the 9th bit of VICTrIntSource.
- Again check the Status registers, whether they contain the required values.
- Check the nVICIRQ and nVICFIQ pins, whether they are low.
- Clear the bit raising the external FIQ. This is the 11th bit of VICTrIntSource.
- Again check the Status registers, whether they contain the required values.
- Check the nVICIRQ and nVICFIQ pins, whether they are low.
- Clear the bit raising the external FIQ. This is the 21st bit of VICTrIntSource.
- Again check the Status registers, whether they contain the required values.
- Check the nVICIRQ and nVICFIQ pins, whether they are low.
- Clear the bit raising the external FIQ. This is the 23rd bit of VICTrIntSource.
- Again check the Status registers, whether they contain the required values.
- Check the nVICIRQ and nVICFIQ pins, whether they are low.
- Clear the bit raising the external FIQ. This is the 24th bit of VICTrIntSource.
- Again check the Status registers, whether they contain the required values.
- Check the nVICIRQ and nVICFIQ pins, whether they are low.
- Clear the bit raising the external FIQ. This is the 26th bit of VICTrIntSource.
- Again check the Status registers, whether they contain the required values.
- Check the nVICIRQ and nVICFIQ pins, whether they are low.
- Clear the bit raising the external FIQ. This is the 28th bit of VICTrIntSource.
- Again check the Status registers, whether they contain the required values.
- Check the nVICIRQ and nVICFIQ pins, whether they are low.
- Clear the bit raising the external FIQ. This is the 29th bit of VICTrIntSource.
- Again check the Status registers, whether they contain the required values.
- Check the nVICIRQ and nVICFIQ pins, whether they are low.
- Clear the bit raising the external FIQ. This is the 30th bit of VICTrIntSource.
- Again check the Status registers, whether they contain the required values.

- Check the nVICIRQ and nVICFIQ pins, whether they are low.
 - Clear the bit raising the external FIQ. This is the 31st bit of VICTrIntSource.
 - Again check the Status registers, whether they contain the required values.
 - Check the nVICIRQ pin, whether it is low.
 - Read the VICVectAddr expecting the content of the VICVectAddr1. This marks the beginning of the interrupt service routine of the first interrupt.
 - Clear the bit raising the Vectored IRQ (bit 10) in the VICTrIntSource.
 - Perform a write access to the VICVectAddr. This marks the end of the interrupt service routine of the first interrupt.
 - Again check the Status registers, whether they contain the required values.
 - Read the VICVectAddr expecting the content of the VICVectAddr3. This marks the beginning of the interrupt service routine of the second interrupt.
 - Clear the bit raising the Vectored IRQ (bit 12) in the VICTrIntSource.
 - Perform a write access to the VICVectAddr. This marks the end of the interrupt service routine of the second interrupt.
 - Again check the Status registers, whether they contain the required values.
 - Read the VICVectAddr expecting the content of the VICVectAddr4. This marks the beginning of the interrupt service routine of the third interrupt.
 - Clear the bit raising the Vectored IRQ (bit 13) in the VICTrIntSource.
 - Perform a write access to the VICVectAddr. This marks the end of the interrupt service routine of the third interrupt.
 - Again check the Status registers, whether they contain the required values.
 - Read the VICVectAddr expecting the content of the VICVectAddr5. This marks the beginning of the interrupt service routine of the fourth interrupt.
 - Clear the bit raising the Vectored IRQ (bit 14) in the VICTrIntSource.
 - Perform a write access to the VICVectAddr. This marks the end of the interrupt service routine of the fourth interrupt.
 - Again check the Status registers, whether they contain the required values.
 - Read the VICVectAddr expecting the content of the VICVectAddr6. This marks the beginning of the interrupt service routine of the fifth interrupt.
 - Clear the bit raising the Vectored IRQ (bit 15) in the VICTrIntSource.
 - Perform a write access to the VICVectAddr. This marks the end of the interrupt service routine of the fifth interrupt.
 - Again check the Status registers, whether they contain the required values.
 - Read the VICVectAddr expecting the content of the VICVectAddr7. This marks the beginning of the interrupt service routine of the sixth interrupt.
 - Clear the bit raising the Vectored IRQ (bit 16) in the VICTrIntSource.
 - Perform a write access to the VICVectAddr. This marks the end of the interrupt service routine of the sixth interrupt.
 - Again check the Status registers, whether they contain the required values.
 - Read the VICVectAddr expecting the content of the VICVectAddr8. This marks the beginning of the interrupt service routine of the seventh interrupt.
 - Clear the bit raising the Vectored IRQ (bit 17) in the VICTrIntSource.
-

- Perform a write access to the VICVectAddr. This marks the end of the interrupt service routine of the seventh interrupt.
- Again check the Status registers, whether they contain the required values.
- Read the VICVectAddr expecting the content of the VICVectAddr9. This marks the beginning of the interrupt service routine of the eighth interrupt.
- Clear the bit raising the Vectored IRQ (bit 18) in the VICTrIntSource.
- Perform a write access to the VICVectAddr. This marks the end of the interrupt service routine of the eighth interrupt.
- Again check the Status registers, whether they contain the required values.
- Read the VICVectAddr expecting the content of the VICVectAddr10. This marks the beginning of the interrupt service routine of the ninth interrupt.
- Clear the bit raising the Vectored IRQ (bit 19) in the VICTrIntSource.
- Perform a write access to the VICVectAddr. This marks the end of the interrupt service routine of the ninth interrupt.
- Again check the Status registers, whether they contain the required values.
- Read the VICVectAddr expecting the content of the VICVectAddr11. This marks the beginning of the interrupt service routine of the tenth interrupt.
- Clear the bit raising the Vectored IRQ (bit 20) in the VICTrIntSource.
- Perform a write access to the VICVectAddr. This marks the end of the interrupt service routine of the tenth interrupt.
- Again check the Status registers, whether they contain the required values.
- Read the VICVectAddr expecting the content of the VICVectAddr13. This marks the beginning of the interrupt service routine of the eleventh interrupt.
- Clear the bit raising the Vectored IRQ (bit 22) in the VICTrIntSource.
- Perform a write access to the VICVectAddr. This marks the end of the interrupt service routine of the eleventh interrupt.
- Again check the Status registers, whether they contain the required values.
- Read the VICVectAddr expecting the content of the VICDefVectAddr. This marks the beginning of the interrupt service routing of the first Non-Vectored interrupt.
- Clear the bit raising the Non-Vectored IRQ (bit 5) in the VICTrIntSource.
- Perform a write access to the VICVectAddr. This marks the end of the interrupt service routine of the first Non-Vectored interrupt.
- Again check the Status registers, whether they contain the required values.
- Read the VICVectAddr expecting the content of the VICDefVectAddr. This marks the beginning of the interrupt service routing of the second Non-Vectored interrupt.
- Clear the bit raising the Non-Vectored IRQ (bit 6) in the VICTrIntSource.
- Perform a write access to the VICVectAddr. This marks the end of the interrupt service routine of the second Non-Vectored interrupt.
- Again check the Status registers, whether they contain the required values.
- Read the VICVectAddr expecting the content of the VICDefVectAddr. This marks the beginning of the interrupt service routing of the third Non-Vectored interrupt.
- Clear the bit raising the Non-Vectored IRQ (bit 7) in the VICTrIntSource.

- Perform a write access to the VICVectAddr. This marks the end of the interrupt service routine of the third Non-Vectored interrupt.
- Again check the Status registers, whether they contain the required values.
- Read the VICVectAddr expecting the content of the VICDefVectAddr. This marks the beginning of the interrupt service routing of the fourth Non-Vectored interrupt.
- Clear the bit raising the Non-Vectored IRQ (bit 25) in the VICTrIntSource.
- Perform a write access to the VICVectAddr. This marks the end of the interrupt service routine of the fourth Non-Vectored interrupt.
- Again check the Status registers, whether they contain the required values.
- Read the VICVectAddr expecting the content of the VICDefVectAddr. This marks the beginning of the interrupt service routing of the fifth Non-Vectored interrupt.
- Clear the bit raising the Non-Vectored IRQ (bit 27) in the VICTrIntSource.
- Perform a write access to the VICVectAddr. This marks the end of the interrupt service routine of the fifth Non-Vectored interrupt.
- Again check the Status registers, whether they contain the required values.

To perform the above tasks, the function `AllIntService()` is invoked. This function is used to service and clear all the interrupts in the order of their priorities. When the argument of this function is 0, it indicates that the interrupts are being raised by the `VICTrIntSource` register of the Trickbox. When the interrupts are raised via the `VICSoftInt` register, the argument is 1.

Now the `ReadStatus()` function is called. Here the `VICIRQStatus` is read expecting a pattern `0xFA5005AF`. The `VICFIQStatus` is read expecting a pattern `0x05AFFA50`. Pattern `0xFFFFFFFF` is expected from the `VICRawIntr` register. To read these three registers, a burst access is done.

Next, the function `CurServReg()` is invoked. The details of the interrupt sources raising vectored interrupts are stored in a 16-bit variable called CSR. Here, the interrupts are raised via the 1st, 3rd, 4th, 5th, 6th, 7th, 8th, 9th, 10th, 11th and 13th vector banks. Thus, the value stored in the CSR should be `0x2FFA`.

Now, the `VICFIQStatus` is checked. If there is a non-zero bit, it indicates that there is an active FIQ interrupt. Hence, the function `FIQServ()` is accessed with the argument zero (indication that it is from the `VICTrIntSource`). This function clears all FIQ interrupts. By comparing with the `VICFIQStatus` register, it is possible to determine which bits of the `VICTrIntSource` are currently requesting a FIQ. Now, beginning from the 0th position, all the FIQ interrupt sources are cleared by writing 0s in the corresponding positions in the `VICTrIntSource` register of the Trickbox.

In the present illustration, the status registers are read. Expected data from the `VICIRQStatus`, `VICFIQStatus` and `VICRawIntr` are `0xFA5005AF`, `0x05AFFA50` and `0xFFFFFFFF` respectively. The bits of the `VICFIQStatus` are read from the 0th position. As soon as, there is a 1 (here, it is the 0th bit), the `VICITOP1` is read with the expected value as `0x00000040`. Now, the 0th bit of the `VICTrIntSource` is cleared.

Now, the bits of the `VICFIQStatus` are further read from the 1st bit position. As soon as, there is a 1 (here, it is the 1st bit), the `VICITOP1` is read with the expected value as `0x00000040`. Now, the 1st bit of the `VICTrIntSource` is cleared.

Now, the bits of the `VICFIQStatus` are further read from the 2nd bit position. As soon as, there is a 1 (here, it is the 2nd bit), the `VICITOP1` is read with the expected value as `0x00000040`. Now, the 2nd bit of the `VICTrIntSource` is cleared.

Now, the bits of the `VICFIQStatus` are further read from the 3rd bit position. As soon as, there is a 1 (here, it is the 3rd bit), the `VICITOP1` is read with the expected value as `0x00000040`. Now, the 3rd bit of the `VICTrIntSource` is cleared.

Now, the bits of the `VICFIQStatus` are further read from the 4th bit position. As soon as, there is a 1 (here, it is the 4th bit), the `VICITOP1` is read with the expected value as `0x00000040`. Now, the 4th bit of the `VICTrIntSource` is cleared.

Now, the bits of the `VICFIQStatus` are further read from the 5th bit position. As soon as, there is a 1 (here, it is the 5th bit), the `VICITOP1` is read with the expected value as `0x00000040`. Now, the 5th bit of the `VICTrIntSource` is cleared.

Now, the bits of the VICFIQStatus are further read from the 7th bit position. As soon as, there is a 1 (here, it is the 9th bit), the VICITOP1 is read with the expected value as 0x00000040. Now, the 9th bit of the VICTrIntSource is cleared.

Now, the bits of the VICFIQStatus are further read from the 10th bit position. As soon as, there is a 1 (here, it is the 11th bit), the VICITOP1 is read with the expected value as 0x00000040. Now, the 11th bit of the VICTrIntSource is cleared.

Now, the bits of the VICFIQStatus are further read from the 12th bit position. As soon as, there is a 1 (here, it is the 21st bit), the VICITOP1 is read with the expected value as 0x00000040. Now, the 21st bit of the VICTrIntSource is cleared.

Now, the bits of the VICFIQStatus are further read from the 22nd bit position. As soon as, there is a 1 (here, it is the 23rd bit), the VICITOP1 is read with the expected value as 0x00000040. Now, the 23rd bit of the VICTrIntSource is cleared.

Now, the bits of the VICFIQStatus are further read from the 24th bit position. As soon as, there is a 1 (here, it is the 24th bit), the VICITOP1 is read with the expected value as 0x00000040. Now, the 24th bit of the VICTrIntSource is cleared.

Now, the bits of the VICFIQStatus are further read from the 25th bit position. As soon as, there is a 1 (here, it is the 26th bit), the VICITOP1 is read with the expected value as 0x00000040. Now, the 26th bit of the VICTrIntSource is cleared.

Now, the bits of the VICFIQStatus are further read from the 27th bit position. As soon as, there is a 1 (here, it is the 28th bit), the VICITOP1 is read with the expected value as 0x00000040. Now, the 28th bit of the VICTrIntSource is cleared.

Now, the bits of the VICFIQStatus are further read from the 29th bit position. As soon as, there is a 1 (here, it is the 29th bit), the VICITOP1 is read with the expected value as 0x00000040. Now, the 29th bit of the VICTrIntSource is cleared.

Now, the bits of the VICFIQStatus are further read from the 30th bit position. As soon as, there is a 1 (here, it is the 30th bit), the VICITOP1 is read with the expected value as 0x00000040. Now, the 30th bit of the VICTrIntSource is cleared.

Now, the bits of the VICFIQStatus are further read from the 31st bit position. As soon as, there is a 1 (here, it is the 31st bit), the VICITOP1 is read with the expected value as 0x00000040. Now, the 31st bit of the VICTrIntSource is cleared.

Now, the VICIRQStatus is checked. If there is a non-zero bit, it indicates that there is an active IRQ interrupt. Now the value of CSR decides whether the IRQ is vectored or non-vectored. Since, the CSR is non-zero (indicating the presence of active vectored IRQs), the function VectIRQServ() is accessed with the argument zero (indicating that it is from the VICTrIntSource). This function clears all the IRQ interrupts in the order of their priorities. The priority of the interrupts goes on decreasing when we move from 0 to 15th bit in the CSR. The CSR is read from the 0th bit. As soon as any non-zero bit occur, the interrupt service routine is started. The routine consists of the reading the VICITOP registers, the address register and then clearing the interrupt and writing to the address register. At the same time the content of the CSR is also changed.

In the present illustration, the status registers are read. Expected data from the VICIRQStatus, VICFIQStatus and VICRawIntr are 0x05AFFA50, 0x00000000 and 0x05AFFA50 respectively. The bits of the CSR are read from the 0th position. As soon as, there is a 1 (here, it is the 1st bit), the VICITOP1 is read with the expected value as 0x00000060. The VICITOP2 is read expecting the vector address, 0x0000A004 (the content of the VICVectAddr1). The VICVectAddr is read expecting the address value as 0x0000A004. This indicates the beginning of the interrupt service routine. Now, the interrupt is to be cleared. For this the bit 10 of the VICTrIntSource is made 0. Now, a write access is done on the VICVectAddr to mark the end of the interrupt service routine. The content of the CSR is updated to 0x2FF8.

Now, the bits of the CSR are read further from the 2nd position. As soon as there is a 1 (here, it is the 3rd bit), the VICITOP1 is read with the expected value as 0x00000060. The VICITOP2 is read expecting the address value as the 0x0000A008 (the content of the VICVectAddr3). The VICVectAddr is read expecting the address value as

0x0000A008. This indicates the beginning of the interrupt service routine. Now, the interrupt is to be cleared. For this bit 12 of the VICTrIntSource is made 0. Now, a write access is done on the VICVectAddr to mark the end of the interrupt service routine. The content of the CSR is updated to 0x2FF0.

Now, the bits of the CSR are read further from the 4th position. As soon as there is a 1 (here, it is the 4th bit), the VICITOP1 is read with the expected value as 0x00000060. The VICITOP2 is read expecting the address value as the 0x0000A00C (the content of the VICVectAddr4). The VICVectAddr is read expecting the address value as 0x0000A00C. This indicates the beginning of the interrupt service routine. Now, the interrupt is to be cleared. For this bit 13 of the VICTrIntSource is made 0. Now, a write access is done on the VICVectAddr to mark the end of the interrupt service routine. The content of the CSR is updated to 0x2FE0.

Now, the bits of the CSR are read further from the 5th position. As soon as there is a 1 (here, it is the 5th bit), the VICITOP1 is read with the expected value as 0x00000060. The VICITOP2 is read expecting the address value as the 0x0000A010 (the content of the VICVectAddr5). The VICVectAddr is read expecting the address value as 0x0000A010. This indicates the beginning of the interrupt service routine. Now, the interrupt is to be cleared. For this bit 14 of the VICTrIntSource is made 0. Now, a write access is done on the VICVectAddr to mark the end of the interrupt service routine. The content of the CSR is updated to 0x2FC0.

Now, the bits of the CSR are read further from the 6th position. As soon as there is a 1 (here, it is the 6th bit), the VICITOP1 is read with the expected value as 0x00000060. The VICITOP2 is read expecting the address value as the 0x0000A014 (the content of the VICVectAddr6). The VICVectAddr is read expecting the address value as 0x0000A014. This indicates the beginning of the interrupt service routine. Now, the interrupt is to be cleared. For this bit 15 of the VICTrIntSource is made 0. Now, a write access is done on the VICVectAddr to mark the end of the interrupt service routine. The content of the CSR is updated to 0x2F80.

Now, the bits of the CSR are read further from the 7th position. As soon as there is a 1 (here, it is the 7th bit), the VICITOP1 is read with the expected value as 0x00000060. The VICITOP2 is read expecting the address value as the 0x0000A018 (the content of the VICVectAddr7). The VICVectAddr is read expecting the address value as 0x0000A018. This indicates the beginning of the interrupt service routine. Now, the interrupt is to be cleared. For this bit 16 of the VICTrIntSource is made 0. Now, a write access is done on the VICVectAddr to mark the end of the interrupt service routine. The content of the CSR is updated to 0x2F00.

Now, the bits of the CSR are read further from the 8th position. As soon as there is a 1 (here, it is the 8th bit), the VICITOP1 is read with the expected value as 0x00000060. The VICITOP2 is read expecting the address value as the 0x0000A01C (the content of the VICVectAddr8). The VICVectAddr is read expecting the address value as 0x0000A01C. This indicates the beginning of the interrupt service routine. Now, the interrupt is to be cleared. For this bit 17 of the VICTrIntSource is made 0. Now, a write access is done on the VICVectAddr to mark the end of the interrupt service routine. The content of the CSR is updated to 0x2E00.

Now, the bits of the CSR are read further from the 9th position. As soon as there is a 1 (here, it is the 9th bit), the VICITOP1 is read with the expected value as 0x00000060. The VICITOP2 is read expecting the address value as the 0x0000A020 (the content of the VICVectAddr9). The VICVectAddr is read expecting the address value as 0x0000A020. This indicates the beginning of the interrupt service routine. Now, the interrupt is to be cleared. For this bit 18 of the VICTrIntSource is made 0. Now, a write access is done on the VICVectAddr to mark the end of the interrupt service routine. The content of the CSR is updated to 0x2C00.

Now, the bits of the CSR are read further from the 10th position. As soon as there is a 1 (here, it is the 10th bit), the VICITOP1 is read with the expected value as 0x00000060. The VICITOP2 is read expecting the address value as the 0x0000A024 (the content of the VICVectAddr10). The VICVectAddr is read expecting the address value as 0x0000A024. This indicates the beginning of the interrupt service routine. Now, the interrupt is to be cleared. For this bit 19 of the VICTrIntSource is made 0. Now, a write access is done on the VICVectAddr to mark the end of the interrupt service routine. The content of the CSR is updated to 0x2800.

Now, the bits of the CSR are read further from the 11th position. As soon as there is a 1 (here, it is the 11th bit), the VICITOP1 is read with the expected value as 0x00000060. The VICITOP2 is read expecting the address value as the 0x0000A028 (the content of the VICVectAddr11). The VICVectAddr is read expecting the address value as 0x0000A028. This indicates the beginning of the interrupt service routine. Now, the interrupt is to be cleared. For

this bit 20 of the VICTrIntSource is made 0. Now, a write access is done on the VICVectAddr to mark the end of the interrupt service routine. The content of the CSR is updated to 0x2000.

Now, the bits of the CSR are read further from the 12th position. As soon as there is a 1 (here, it is the 13th bit), the VICITOP1 is read with the expected value as 0x00000060. The VICITOP2 is read expecting the address value as the 0x0000A030 (the content of the VICVectAddr13). The VICVectAddr is read expecting the address value as 0x0000A030. This indicates the beginning of the interrupt service routine. Now, the interrupt is to be cleared. For this bit 22 of the VICTrIntSource is made 0. Now, a write access is done on the VICVectAddr to mark the end of the interrupt service routine. The content of the CSR is updated to 0x0000.

Now, the bits of the CSR are further read from the 14th position.

Now, the VICIRQStatus is checked. If there is a non-zero bit, it indicates that there is an active IRQ interrupt. Since, the CSR is zero (indicating the absence of any active vector IRQ), the function NonVectIRQServ() is accessed with the argument zero (indicating that it is from the VICTrIntSource). This function clears all the non vectored IRQ interrupts. The VICIRQStatus is read from the 0th bit. As soon as any non-zero bit occur, the ITOP registers are read. Then the VICVectAddr is read. Now, the bit which is raising this interrupt is cleared in the VICTrIntSource. Then, a write access is done on the VICVectAddr register.

In the present illustration, the status registers are read. Expected data from the VICIRQStatus, VICFIQStatus and VICRawIntr are 0x0A0000E0, 0x00000000 and 0x0A0000E0 respectively. The bits of the VICIRQStatus are read from the 0th position. As soon as, there is a 1 (here, it is the 5th bit), the VICITOP1 is read with the expected value as 0x00000060. The VICITOP2 is read expecting the vector address, 0x12345678. The VICVectAddr is read expecting the address value as 0x12345678. This indicates the beginning of the interrupt service routine. Now, the interrupt is to be cleared. For this bit 5 of the VICTrIntSource is made 0. Now, a write access is done on the VICVectAddr to mark the end of the interrupt service routine.

Now the bits of the VICIRQStatus are read further from the 6th position. As soon as there is a 1 (here, it is the 6th), the VICITOP1 is read with the expected value as 0x00000060. The VICITOP2 is read expecting the address value as the 0x12345678 (the content of the VICDefVectAddr). This indicates the beginning of the interrupt service routine. Now, the interrupt is to be cleared. For this the bit 6 of the VICTrIntSource is made 0. Now, a write access is done on the VICVectAddr to mark the end of the interrupt service routine.

Now the bits of the VICIRQStatus are read further from the 7th position. As soon as there is a 1 (here, it is the 7th), the VICITOP1 is read with the expected value as 0x00000060. The VICITOP2 is read expecting the address value as the 0x12345678 (the content of the VICDefVectAddr). This indicates the beginning of the interrupt service routine. Now, the interrupt is to be cleared. For this bit 7 of the VICTrIntSource is made 0. Now, a write access is done on the VICVectAddr to mark the end of the interrupt service routine.

Now the bits of the VICIRQStatus are read further from the 8th position. As soon as there is a 1 (here, it is the 25th), the VICITOP1 is read with the expected value as 0x00000060. The VICITOP2 is read expecting the address value as the 0x12345678 (the content of the VICDefVectAddr). This indicates the beginning of the interrupt service routine. Now, the interrupt is to be cleared. For this bit 25 of the VICTrIntSource is made 0. Now, a write access is done on the VICVectAddr to mark the end of the interrupt service routine.

Now the bits of the VICIRQStatus are read further from the 26th position. As soon as there is a 1 (here, it is the 27th), the VICITOP1 is read with the expected value as 0x00000060. The VICITOP2 is read expecting the address value as the 0x12345678 (the content of the VICDefVectAddr). This indicates the beginning of the interrupt service routine. Now, the interrupt is to be cleared. For this bit 27 of the VICTrIntSource is made 0. Now, a write access is done on the VICVectAddr to mark the end of the interrupt service routine.

Now the bits of the VICIRQStatus are read further from the 28th position.

Now, the Status registers are read again expecting 0x00000000, 0x00000000 and 0x00000000 from the VICIRQStatus, VICFIQStatus and VICRawIntr registers, respectively.

The other categories of the Sixteen interrupts case are tested in a similar fashion.

6.4.8 Special Interrupt Tests

Apart from the Basic Interrupt tests, where the interrupts are raised simultaneously and cleared one-by-one according to the priority of the interrupts, three other cases are considered.

- Interrupts are raised simultaneously, but the order of servicing and clearing is predefined.
- Interrupts are raised non-simultaneously.
- Interrupts are cleared simultaneously.

Interrupts serviced and cleared in a predefined order

This test is invoked by the UserOneIntTests() function call.

Four cases each from the One-Interrupt, Two-Interrupt, Four-Interrupt, Eight-Interrupt, Sixteen-Interrupt and Thirty-Two-Interrupt are considered. In all, 640 tests are performed of this type. One test is illustrated with an example.

Program the following initialising values in the following registers.

VICIntEnable	0xFFFFFFFF	All the interrupts are enabled
VICSoftInt	0x00000000	None of the interrupts can occur through the VICSoftInt register
VICIntSelect	0x00000000	Decides which of the sources will raise an FIQ
VICTrIntIn	0x00000003	Indicates both the external interrupts
VICTrVectAddrIn	0x00000000	External Vector Address
VICProtection	0x00000000	VIC is in the 'User' mode
VICVectAddr0	0x0000A000	Address for Vector bank 0 is 0x0000A000
VICVectAddr1	0x0000A004	Address for Vector bank 1 is 0x0000A004
...	...	...
VICVectAddr15	0x0000A03C	Address for Vector bank 15 is 0x0000A03C
VICVectCntl0	0x00000024	Interrupt source 4 will raise a vector interrupt request
VICVectCntl1	0x00000025	Interrupt source 5 will raise a vector interrupt request
...	...	...
VICVectCntl15	0x00000033	Interrupt source 19 will raise a vector interrupt request

In this illustration interrupt sources 12 and 16 are made active by invoking the function ApplyIntr() with the argument as 0. Since none of the bits of the VICIntSelect are 1, these sources will be raising IRQs. The rest of the interrupts will be IRQ interrupts. Now, the last 5 bits of VICVectCntl8 and VICVectCntl12 when converted to decimal gives 12 and 16 respectively. This indicates that the interrupt sources 12 and 16 are raising Vectored IRQs. Now the function ServiceTwo() is called with the arguments 16 followed by 12. This indicates that the interrupt source 16 is to be cleared first followed by the source 12.

To test the VIC, the following have to be performed:

- Check the Status registers, whether they contain the required values.
- Check the nVICIRQ pin, whether it is low.
- Read the VICVectAddr expecting the content of the VICVectAddr8. This marks the beginning of the interrupt service routine of the first interrupt.
- Clear the 16th bit of the interrupt source.

- Check the Status registers, whether they contain the required values.
- Clear the 12th bit of the interrupt source.
- Perform a write access to the VICVectAddr. This marks the end of the interrupt service routine of the first interrupt.
- Again check the Status registers, whether they contain the required values.

To perform the above tasks, the function `ServiceTwo()` is invoked. This function is used to clear the interrupts in the order the arguments are passed to it.

Since, the status registers are clocked, the change in these registers are visible only after one clock cycle. Hence, in the function `ServiceTwo()`, an IDLE cycle is added to compensate for this one clock discrepancy.

Now the `ReadStatus()` function is called. Here the `VICIRQStatus` is read expecting a pattern of 0x00011000. The `VICFIQStatus` is read expecting a pattern of 0x00000000. The pattern 0x00011000 is expected from the `VICRawIntr` register. To read these three registers, a burst access is done.

Next, the function `CurServReg()` is invoked. The details of the interrupt sources raising vectored interrupts are stored in a 16-bit variable called CSR. Here, the interrupts are raised via the 8th and 12th vector banks. Thus, the value stored in the CSR should be 0x1100.

Now, another function called `UserIntService()` is called with the argument 16. This indicates that the interrupt to be cleared is 16. But, the interrupt to be serviced will be the highest priority one (here, it is source 12).

While clearing the Vectored interrupt, once the `VICVectAddr` is read it should not be read until this particular interrupt bit is cleared. To track this a 16-bit global variable called `servreg` is used. Here, the `VICVectAddr` will be the content of `VICVectAddr8` (corresponding to the interrupt from 12th bit). But, the interrupt to be cleared is that from the 16th bit. Hence, as soon as the `VICVectAddr` is read, the 8th bit of the `servreg` is set. Hence, `servreg` is now, 0x0100. Now, the 16th interrupt is cleared. Now, interrupt number 12 is to be cleared. But, before reading the `VICVectAddr` register, the `servreg` is checked whether it is zero. If it is non-zero, then the expected address-out will be the default address. Now, after clearing the 12th interrupt, the `servreg` is cleared. When the interrupt is an FIQ, the address is not read.

In this illustration, the 16th bit of the `VICFIQStatus` is checked for any FIQ at the 16th bit position.

Then the 16th bit of `VICIRQStatus` is checked for any IRQ. The CSR is read from the 0th bit. As soon as a 1 is read (in this case, it is at bit 8) the `VICVectAddr` is read expecting the `VICVectAddr8`. Since, this is not the interrupt to be cleared, the 8th bit of the `servbit` is set. The CSR updated to 0100.

Now, the program control returns to the `ServiceTwo()` function. There another `UserIntService()` function call is made with argument as 12.

The 12th bit of the `VICFIQStatus` is checked for any FIQ at the 12th bit position.

Then the 12th bit of `VICIRQStatus` is checked for any IRQ. The CSR is read from the 0th bit. As soon as a 1 is read (in this case, it is at bit 12) the `VICVectAddr` is read. This time, the expected address is the the content of `VICDefVectAddr`. This is because `servreg` is not zero. Now, the CSR is updated to 0x0000 and the `servreg` is changed to 0x0000.

Interrupts are raised non-simultaneously

This test is invoked by the `UserOneIntTests()` function call.

In all, 128 tests are performed of this type. One test is illustrated as an example.

Program the following initialising values in the following registers.

<code>VICIntEnable</code>	0xFFFFFFFF	All the interrupts are enabled
<code>VICSoftInt</code>	0x00000000	None of the interrupts can occur through the <code>VICSoftInt</code> register
<code>VICIntSelect</code>	0x00000000	Decides which of the sources will raise an FIQ

VICTrIntIn	0x00000003	Indicates both the external interrupts
VICTrVectAddrIn	0x00000000	External Vector Address
VICProtection	0x00000000	VIC is in the 'User' mode
VICVectAddr0	0x0000A000	Address for Vector bank 0 is 0x0000A000
VICVectAddr1	0x0000A004	Address for Vector bank 1 is 0x0000A004
...	...	...
VICVectAddr15	0x0000A03C	Address for Vector bank 15 is 0x0000A03C
VICVectCntl0	0x00000024	Interrupt source 4 will raise a vector interrupt request
VICVectCntl1	0x00000025	Interrupt source 5 will raise a vector interrupt request
...	...	...
VICVectCntl15	0x00000033	Interrupt source 19 will raise a vector interrupt request

In this illustration, interrupt source 8 is made active by invoking the function `ApplyIntr()` with the argument as 0. Since, none of the bits of the `VICIntSelect` are 1, this source will be raising IRQ. Now, the last 5 bits of `VICVectCntl4` when converted to decimal gives 8. This indicates that the interrupt source 8 is raising Vectored IRQs. The servicing of this interrupt is started. Now, interrupt source 20 is made active by invoking the function `ApplyIntr()`. This interrupt is a non-vectored IRQ. Hence, the `VICVectAddr` won't change. Now, the 8th bit of the interrupt source is cleared. Then the 20th bit is cleared.

To test the VIC, the following have to be performed:

- Check the Status registers, whether they contain the required values.
- Check the `nVICIRQ` pin, whether it is low.
- Read the `VICVectAddr` expecting the content of the `VICVectAddr4`. This marks the beginning of the interrupt service routine of the first interrupt.
- The interrupt from the 20th bit is applied.
- Check the Status registers, whether they contain the required values.
- Clear the 8th bit of the interrupt source.
- Perform a write access to the `VICVectAddr`. This marks the end of the interrupt service routine of the first interrupt.
- Again check the Status registers, whether they contain the required values.
- Read the `VICVectAddr` expecting the content of the `DefVICVectAddr`. This marks the beginning of the interrupt service routine of the second interrupt.
- Clear the 20th bit of the interrupt source.
- Perform a write access to the `VICVectAddr`. This marks the end of the interrupt service routine of the second interrupt.

To perform the above tasks, the function `TwoDiffInt()` is invoked with the arguments as 8 and 20. This function is used to raise the interrupts in the order the arguments are passed. The interrupt from the 8th bit is applied.

Now the `ReadStatus()` function is called. Here the `VICIRQStatus` is read expecting a pattern of 0x00000100. The `VICFIQStatus` is read expecting a pattern of 0x00000000. The pattern 0x00000100 is expected from the `VICRawIntr` register. To read these three registers, a burst access is done.

Next, the function `CurServReg()` is invoked. The details of the interrupt sources raising vectored interrupts are stored in a 16-bit variable called CSR. Here, the interrupts are raised via the 4th vector bank. Thus, the value stored in the CSR should be 0x0010.

This value of CSR is stored in a variable called CSR1. Hence, the value of the CSR1 is 0x0010. If the first interrupt is a Vectored IRQ then the vector address is read. If it is a Non-Vectored IRQ, the vector address is read expecting the default value. In the illustration, the value expected is the content of `VICVectAddr4`.

At this point, the second interrupt from bit 20 is applied. This interrupt source is raising a Non-Vectored IRQ.

Now the `ReadStatus()` function is called. Here the `VICIRQStatus` is read expecting a pattern of 0x00100100. The `VICFIQStatus` is read expecting a pattern of 0x00000000. The pattern 0x00100100 is expected from the `VICRawIntr` register. To read these three registers, a burst access is done.

Next, the function `CurServReg()` is invoked. The details of the interrupt sources raising vectored interrupts are stored in a 16-bit variable called CSR. Here, the interrupts are raised via the 4th vector bank. Thus, the value stored in the CSR should be 0x0010.

Now, to compare the priority of the second interrupt compared to the first one, a new variable CSR2 is introduced. It holds the value of the CSR if only the second interrupt was active. In the present illustration, CSR2 is 0x0000. This is because the second interrupt is a Non-Vectored IRQ.

If any of the interrupts applied is a FIQ, then it is just cleared. In the present illustration, this case is not applicable.

Now, in the present illustration CSR1 is non-zero and CSR2 is zero. Hence, the first interrupt is cleared. This is done by writing a 0 in the 8th bit of the `VICTrIntSource`. To mark the end of the interrupt service routine, a write access is performed to the `VICVectAddr` register. The CSR is updated to 0x0000.

Any pending IRQs are checked. If there is one, then the value of CSR is read. If the CSR is zero (as is the case here), it looks whether the interrupt service routine of this interrupt has already been started. Here, the `VICITOP` registers are read. Then the `VICVectAddr` is read to indicate the beginning of the interrupt service routine. Then the 0th bit in the `VICTrIntSource` is cleared to clear the interrupt source. To mark the end of the service routine, a write is conducted on `VICVectAddr` register.

Interrupts Cleared Simultaneously

This test is invoked by the `UserOneIntTests()` function call.

In all, 128 tests are performed of this type. One test is illustrated as an example.

Program the following initialising values in the following registers.

<code>VICIntEnable</code>	0xFFFFFFFF	All the interrupts are enabled
<code>VICSoftInt</code>	0x00000000	None of the interrupts can occur through the <code>VICSoftInt</code> register
<code>VICIntSelect</code>	0x00000000	Decides which of the sources will raise an FIQ
<code>VICTrIntIn</code>	0x00000003	Indicates both the external interrupts
<code>VICTrVectAddrIn</code>	0x00000000	External Vector Address
<code>VICProtection</code>	0x00000000	VIC is in the 'User' mode
<code>VICVectAddr0</code>	0x0000A000	Address for Vector bank 0 is 0x0000A000
<code>VICVectAddr1</code>	0x0000A004	Address for Vector bank 1 is 0x0000A004
...	...	...
<code>VICVectAddr15</code>	0x0000A03C	Address for Vector bank 15 is 0x0000A03C
<code>VICVectCntl0</code>	0x00000024	Interrupt source 4 will raise a vector interrupt request

VICVectCntl1	0x00000025	Interrupt source 5 will raise a vector interrupt request
...	...	...
VICVectCntl15	0x00000033	Interrupt source 19 will raise a vector interrupt request

In this illustration, interrupt sources 6, 8, 22 and 24 are made active by invoking the function `ApplyIntr()` with the argument as 0. Since, none of the bits of the `VICIntSelect` are 1, these sources will be raising IRQs. Now, the last 5 bits of `VICVectCntl2` and `VICVectCntl4` when converted to decimal give 6 and 8 respectively. This indicates that the interrupt sources 6 and 8 are raising Vectored IRQs. And the interrupt from the sources 22 and 24 are raising Non-Vectored IRQs. Now, the function `SimulClear()` is invoked with arguments 6, 8, 22 and 24. The function `SimulClear()` services the highest priority interrupt first. In the present case, it is interrupt number 6. But, while clearing the interrupt number 6, the second highest priority interrupt is also cleared. In this case, the interrupt numbers 6 and 8 are cleared simultaneously. The rest of the interrupts are serviced and cleared according to their priorities.

To test the VIC, the following have to be performed:

- Check the Status registers, whether they contain the required values.
- Check the `nVICIRQ` pin, whether it is low.
- Read the `VICVectAddr` expecting the content of the `VICVectAddr2`. This marks the beginning of the interrupt service routine of the interrupt from source 6.
- Identify the next highest priority interrupt, i.e., source 8.
- Clear both the interrupts viz., source 6 and source 8.
- Check the Status registers, whether they contain the required values.
- Read the `VICVectAddr` expecting the content of the `VICDefVectAddr`.
- Clear the interrupt source number 22.
- Check the Status registers, whether they contain the required values.
- Read the `VICVectAddr` expecting the content of the `VICDefVectAddr`.
- Clear the interrupt source number 24.
- Again check the Status registers, whether they contain the required values.

To perform the above tasks, the function `SimulClear()` is invoked with the arguments as 6, 8, 22 and 24. This function is used to raise the interrupts through the interrupt source 6, 8, 22 and 24.

Now the `ReadStatus()` function is called. Here the `VICIRQStatus` is read expecting a pattern of 0x08400840. The `VICFIQStatus` is read expecting a pattern of 0x00000000. The pattern 0x08400840 is expected from the `VICRawIntr` register. To read these three registers, a burst access is done.

Next, the function `CurServReg()` is invoked. The details of the interrupt sources raising vectored interrupts are stored in a 16-bit variable called CSR. Here, the interrupts are raised via the 2nd and 4th vector bank. Thus, the value stored in the CSR should be 0x0014.

Two variables `intnum` and `vectnum` are initialised. These keep track of the number of interrupts cleared and the number of vectored interrupts cleared. The fundamental idea is that, when the value of `intnum` is 1, no read or write transactions take place. When the next highest priority interrupt is determined, the value of `intnum` is incremented by 1. Thus, two bits - the highest and second highest priority interrupts - are cleared simultaneously.

First, the `VICFIQStatus` is checked for any FIQ.

If the `intnum` is not equal to 1 (at present it is 0), the status registers are again read. The values expected are 0x08400840, 0x00000000 and 0x08400840 from the `VICIRQStatus`, `VICFIQStatus` and `VICRawIntr` respectively.

The CSR is checked from the 0th position. At the 2nd bit position, a 1 is encountered. Now, the vector address of the highest priority interrupt is read from the `VICVectAddr`, i.e., 0x0000A008. This is the beginning of the servicing of the interrupt from source 6.

The CSR is updated to 0x0010.

Now, the `intnum` and `vectnum` are made 1. The variable which is used to store the data in the `D_VICTrIntSource` is changed. The 6th bit in it is made 0.

Since, the value of `intnum` is 1, no write is made on the `VICTrIntSource`.

Again the checking of the CSR is continued from the 3rd position onwards. The 4th bit is 1. Since, `vectnum` is 1, the address is not read.

Now, `vectnum` and `intnum` are incremented to 2. The variable which is used to store the data in the `D_VICTrIntSource` is changed. The 8th bit is also made 0. Since, the value of `intnum` is not 1, the value stored by the `D_VICTrIntSource` is written to the `VICTrIntSource` register. The data in the `VICTrIntSource` is 0x08400000.

To mark the end of the servicing of source 6, a write access is done to the `VICVectAddr` register.

The CSR is updated to 0x0000.

Since no Vectored IRQs are now pending, the Non-Vectored IRQs are looked for. This is done by reading the `VICIRQStatus` from the 0th bit. In this case, first the 22nd bit in the `VICTrIntSource` is cleared and then the 24th bit. After clearing one bit, the `VICVectAddr` is accessed for a write.

6.5 Verification Test Vector Generation

A ***make sourcefilename*** (without the .c extension) in the ***verification/bustests*** directory will create ***.bif*** formatted and ***.sim*** formatted vectors in the ***verification/bustest/invec*** directory. The makefile in ***verification/bustest*** directory can be referred to for the make options.

The ***make all*** option can be used to create a test vector file including all the tests.

7 VIC INTEGRATION TEST DETAILS

The VIC integration test vector suite allows the integrator to verify that the VIC has been connected into the target system correctly. These vectors test 2 classes of signals:

- a) AMBA signals (VIC signals connected directly to the AHB)
- b) Intra-Chip signals (VIC signals connected to the ARM or to other daisy chained VICs)

For more details on Integration testing, please refer to the PrimeCell Integration Vector Strategy document [Ref 6.].

The VIC Integration testbench is based on an AHB TICTalk testbench. The VHDL and Verilog TICTalk testbenches are used to verify that the VIC has been connected into the target system correctly. The VHDL and Verilog TICTalk testbench files reside in the *integration/vhdl* and *integration/verilog* directories respectively. The test vectors that are applied to the TICTalk testbenches are in the *integration/bustest* directory. The testvectors for integration testing are written in BusTalk and then converted into *.tif* format to be applied to the TICTalk integration testbench.

7.1 VIC VHDL Integration testbench

The AMBA TICTalk VHDL testbench used for integration testing of the VIC is located in the *integration/vhdl/tbench* directory. The Integration test vectors are located in the *integration/bustest* directory.

The test vectors used with this testbench can be applied to the VIC when it is part of an AMBA system design.

Figure 7.1-1 illustrates the VHDL testbench to apply TICTalk test vectors.

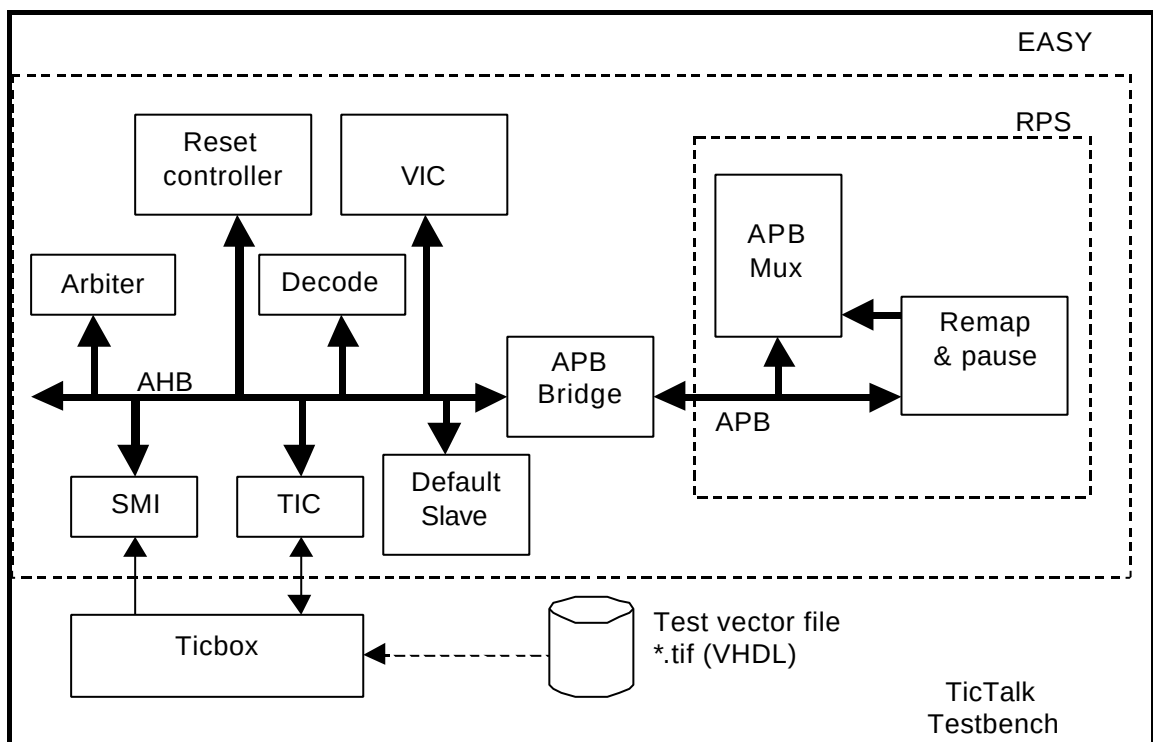


Figure 7.1-1 VHDL Testbench for simulating TICTalk test vectors

The VIC VHDL Integration testbench is a stripped down version of the standard EASY system testbench. For further information refer to the Example AMBA System user guide which is part of ARM Micropack documentation.

Figure 7.1-2 illustrates the hierarchy for the TICTalk VHDL testbench.

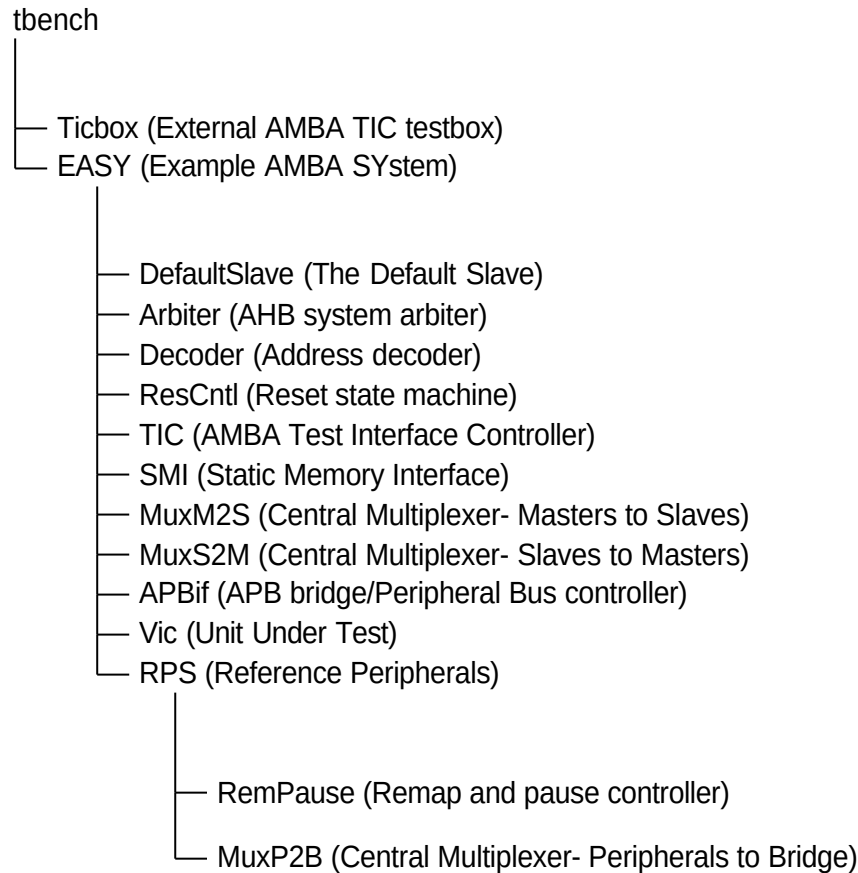


Figure 7.1-2 VHDL Testbench hierarchy for simulating TICTalk test vectors

The test vectors are applied to **tbench.vhd**, the top-level of the VIC Integration testbench. This module instantiates the model of the EASY microcontroller and the Ticbox. The VIC is instantiated in the EASY.vhd. All the Scan input pins are tied low to keep them from interfering with the test process. The VICINTSOURCE, VICVECTADDRIN, nVICFIQIN and nVICIRQIN lines are tied to their inactive levels. [During system integration, these lines will have to be re-wired].

7.1.1 Unit Under Test

The Unit Under Test may be one of the following

- **VIC VHDL RTL**
- **VIC scan-inserted VHDL netlist from VHDL Source**

One out of these two versions may be referenced by creating a link [named **uut**] in the **integration/vhdl** directory to the corresponding source directory. [Note that this link will be created automatically by the simulation makefiles].

7.1.2 Test Vectors

The test vectors for the integration testing of the VIC are coded into separate C files for testing accesses to all the VIC registers and for Integration testing. Make options are provided to run the tests together or individually.

7.1.3 Running Simulations

The **README** file in the *integration/vhdl/tbench* directory gives step by step instructions for running simulations using the VIC Integration test vectors on the VHDL source code.

Toggle coverage numbers (for the block level I/Os) when the integration test vectors are run on the VHDL RTL and netlist forms of the VIC are available in the *integration/vhdl/Vic_rtl* and *integration/vhdl/Vic_scaninsert_vhdl_net_max* directories.

See files: *Vic_rtl_modelsim.untog*, *Vic_scaninsert_vhdl_vhdl_net_max_modelsim.untog*.

A sample toggle report (*.untog) file is shown below:

Toggle Report	Node	-> 0	->1
	Signal 1	x	y
	Signal 2	a	b
Total Node Count	= C		
Toggled Node Count	= T		
Untoggled Node Count	C-T = U		
Toggle Coverage	T/C = P %		

The column "Node" in the .untog file lists the individual nodes that are not toggled when the test vectors are run on the VHDL in RTL or netlist forms. The column "->0" indicates the number of times that particular node has toggled to 0 and the column "->1" indicates the number of times it has toggled to 1. In the above example, the Node "Signal 1" toggles to 0 "x" times and it toggles to 1 "y" times. The "Total Node Count", indicates the total number of nodes in the block and the "Toggled Node Count" indicates the number of these nodes that are toggled when the test vectors are run. The number "Untoggled Node Count" indicates the number of nodes in the block that are not toggled when the test vectors are run and the "Toggle Coverage" gives the percentage of the total number of nodes in the block that are toggled when the test vectors are run.

Integration tests are done with the VIC in stand-alone mode with the internal test paths from the intra-chip inputs pins to the VIC core logic being selected. Therefore the intra-chip input pins may be listed as untoggled in the .untog file. To toggle these, configure the VICITCR register (as mentioned in the .c files in the *integration/bustest* directory) so that the normal paths from the intra-chip pins to the VIC core logic is selected. The VIC supports only WORD accesses (HSIZE = 0x2) and gives only OKAY and ERROR responses (HRESP = 0x0 and 0x1). Hence the AHB lines, HSIZE[2] and HRESP[1], may be listed as untoggled.

Run time logs for all the combinations are available in the following files in the *integration/vhdl/Vic_rtl* and *integration/vhdl/Vic_scaninsert_vhdl_net_max* directories.

See files: *Vic_rtl_modelsim.log*, *Vic_scaninsert_vhdl_vhdl_net_max_modelsim.log*.

7.2 VIC Verilog Integration testbench

The AMBA TICTalk Verilog testbench used for integration testing of the VIC is located in the *integration/verilog/tbench* directory. The Integration test vectors are located in the *integration/bustest/invec* directory.

The test vectors used with this testbench can be applied to the VIC when it is part of an AMBA system design.

Figure 7.2-1 illustrates the Verilog testbench to apply TICTalk test vectors.

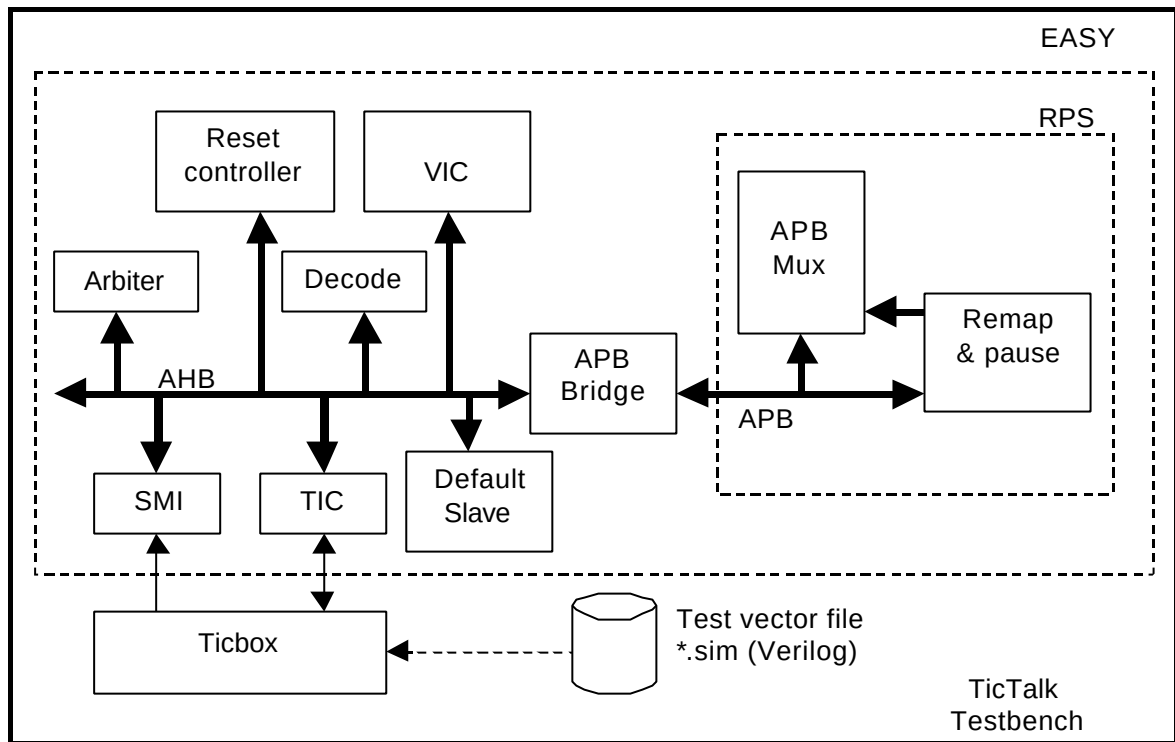


Figure 7.2-1 Verilog Testbench for simulating TICTalk test vectors

The VIC Verilog Integration testbench is a stripped down version of the standard EASY system testbench. For further information refer to the Example AMBA System user guide which is part of ARM Micropack documentation.

Figure 7.2-2 illustrates the hierarchy for the TICTalk Verilog testbench.

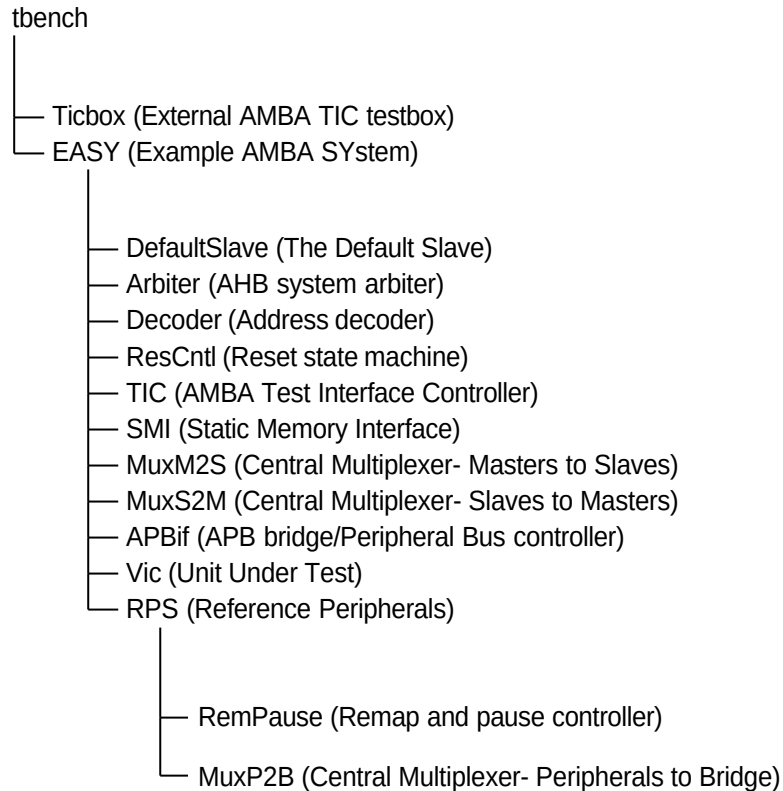


Figure 7.2-2 Verilog Testbench hierarchy for simulating TICTalk test vectors

The test vectors are applied to **tbench.v**, the top-level of the VIC Integration testbench. This module instantiates the model of the EASY microcontroller and the Ticbox. The VIC is instantiated in the EASY.v. All the Scan input pins are tied low to keep them from interfering with the test process. The VICINTSOURCE, VICVECTADDRIN, nVICFIQIN and nVICIRQIN lines are tied to their inactive levels. [During system integration, these lines will have to be re-wired].

7.2.1 Unit Under Test

The Unit Under Test may be one of the following

- **VIC Verilog RTL**
- **VIC scan-ready Verilog netlist from Verilog Source**
- **VIC non-scan Verilog netlist from VHDL Source**

One out of these two versions may be referenced by creating a link [named **uut**] in the *integration/verilog* directory to the corresponding source directory. [Note that this link will be created automatically by the simulation makefiles].

7.2.2 Test Vectors

The test vectors for the integration testing of the VIC are coded into separate C files for testing accesses to all the VIC registers and for Integration testing. Make options are provided to run the tests together or individually.

7.2.3 Running Simulations

The **README** file in the *integration/verilog/tbench* directory gives step by step instructions for running simulations using the VIC Integration test vectors on the VERILOG source code.

Toggle coverage numbers (for the block level I/Os) when the integration test vectors are run on the Verilog RTL and netlist forms of the VIC are available in the *integration/verilog/Vic_rtl* and *integration/verilog/Vic_noscan_vhdl_net_min* directories.

See files: *Vic_rtl_modelsim.untog*, *Vic_noscan_vhdl_verilog_net_min_modelsim.untog*.

A sample toggle report (*.untog) file is shown below:

Toggle Report	Node	-> 0	->1
	Signal 1	x	y
	Signal 2	a	b
Total Node Count	= C		
Toggled Node Count	= T		
Untoggled Node Count	C-T = U		
Toggle Coverage	T/C = P %		

The column "Node" in the .untog file lists the individual nodes that are not toggled when the test vectors are run on the Verilog in RTL or netlist forms. The column "->0" indicates the number of times that particular node has toggled to 0 and the column "->1" indicates the number of times it has toggled to 1. In the above example, the Node "Signal 1" toggles to 0 "x" times and it toggles to 1 "y" times. The "Total Node Count", indicates the total number of nodes in the block and the "Toggled Node Count" indicates the number of these nodes that are toggled when the test vectors are run. The number "Untoggled Node Count" indicates the number of nodes in the block that are not toggled when the test vectors are run and the "Toggle Coverage" gives the percentage of the total number of nodes in the block that are toggled when the test vectors are run.

Integration tests are done with the VIC in stand-alone mode with the internal test paths from the intra-chip inputs pins to the VIC core logic being selected. Therefore the intra-chip input pins may be listed as untoggled in the .untog file. To toggle these, configure the VICITCR register (as mentioned in the .c files in the *integration/bustest* directory) so that the normal paths from the intra-chip pins to the VIC core logic is selected. The VIC supports only WORD accesses (HSIZE = 0x2) and gives only OKAY and ERROR responses (HRESP = 0x0 and 0x1). Hence the AHB lines, HSIZE[2] and HRESP[1], may be listed as untoggled.

Run time logs for all the combinations are available in the following files in the *integration/verilog/Vic_rtl*, *integration/verilog/Vic_noscan_vhdl_net_min* and *integration/verilog/Vic_scanready_verilog_net_typ* directories.

See files: *Vic_rtl_modelsim.log*, *Vic_rtl_LDV.log*, *Vic_noscan_vhdl_verilog_net_min_modelsim.log* and *Vic_scanready_verilog_verilog_net_typ_LDV.log*.

7.3 Integration Tests

When the VIC is used in the AMBA system, it must be ensured that all the I/O pins are correctly connected. Integration vectors allow the user to verify that the VIC has been wired into the system correctly.

Integration testing is performed by the RegAccessTests() and the IntegrationTest() function calls.

7.3.1 Integration Testing of AHB pins

The `RegAccessTests()` function call is used to test the connectivity of the AHB signals via a sequence of register accesses.

- HCLK and HRESETn are verified to be toggling (implicitly) by verifying that the other tests which follow pass. Note that the test sequence starts with an assertion and then a deassertion of the HRESETn line.
- HADDR: 1's and 0's are made to appear on these signals by performing a read access to all the internal registers, following the deassertion of HRESETn.
- HTRANS[1]: A '1' and a '0' are made to appear on this AHB signal by initiating an NSEQ (HTRANS[1] = 1) and an IDLE (HTRANS[1] = 0) transfer.
- HWRITE: Read and Write accesses are performed to the VIC to toggle this signal.
- HSIZE: The VIC supports only WORD (HSIZE = 2) accesses. 1's and 0's are made to appear on these lines by performing WORD and HALFWORD accesses to the internal registers.
- HPROT: A '1' and a '0' is made to appear on this AHB signal by making 'User' and 'Supervisor' mode accesses. Toggling of this line is verified using the following steps. Configure the VIC in 'PROTECTED' mode. Then access the VIC in Supervisor mode for writing into a register (VICDefVectAddr) with a data value of 0x55555555. Access the VIC in User mode to modify this register. Check the register contents to prove that it remains unmodified.
- HWDATA and HRDATA: These lines are toggled by performing write and read accesses with different data patterns (0x55555555 and 0xAAAAAAAA) and their connectivity is verified by validating those accesses.
- HSELVIC: A '1' and a '0' is made to appear on this line by writing to a location with HADDR[11:2] same as the offset address of a VIC register (VICIntSelect) and with a base address different from that of the VIC. The connectivity of HSELVIC is proven by checking for unmodified data in the above register.

7.3.2 Integration Testing of intra-chip inputs

Signals : nVICFIQIN, nVICIRQIN, VICVECTADDRIN, VICINTSOURCE

When Integration tests are run with the VIC in a stand-alone test setup:

- Write a '1' to the ITEN bit in the VICITCR register. This selects the test path from the VICITIP registers to the internal signals.
- Write 1's and then 0's to the VICITIP registers and read the same register to ensure that the value written is read out.

When Integration tests are run with the VIC as part of an integrated system:

- Write a '0' to the ITEN bit in the VICITCR register. This selects the normal path from the intra-chip inputs to the internal VIC signals.
- '1's and '0's are made to appear on these inputs by programming their sources appropriately.
- For the nVICFIQIN, nVICIRQIN and VICVECTADDRIN inputs in a system where multiple VICs are connected in a Daisy Chain, these signals are controlled by writing into the internal registers of the first VIC in the Daisy Chain and the connection is verified by reading out from the VICITIP1-2 registers. In case the source of these signals is not another VIC, the integrator will have to suitably program the relevant controller.
- For the VICINTSOURCE inputs, these sources typically are other interrupt capable peripherals in the system. The integrator will have to suitably program these peripherals and read back the expected values from the VICRawIntr register.

The ***IntegrationTest.c*** file in the ***integration/bustest*** directory is embedded with NOTES which direct the system integrator to those commands that need to be replaced with suitable equivalents.

7.3.3 Integration Testing of intra-chip outputs

Signals : nVICFIQ, nVICIRQ, VICVECTADDRROUT

When Integration tests are run with the VIC in a stand-alone test setup:

- Program the VIC such that the all intra-chip outputs are set to both '0' and '1'.
- Read the expected values back from the VICITOP1 and VICITOP2 registers. Note that for the nVICIRQ and the nVICFIQ outputs, a '1' on the output pin will be reflected as a '0' in the VICITOP1 register [and vice versa]

When Integration tests are run with the VIC as part of an integrated system:

- Program the VIC such that the all intra-chip outputs are set to both 0's and then '1's.
- In a system where multiple VICs are connected in a Daisy Chain, the connection is verified by reading out the expected values from the next VIC's internal registers (VICITIP1-2). Note that for the nVICIRQ and the nVICFIQ outputs, a '1' on the output pin will be reflected as a '1' in the next VIC's internal registers.

7.4 Integration Test Vector Generation

A **make sourcefilename** (without the .c extension) in the **integration/bustest** directory will create .tif formatted and .sim formatted vectors in the **integration/bustest/invec** directory. The makefile in the **integration/bustest** directory can be referred to for the make options.

The **make all** option can be used to create a test vector file including all the tests.